

SKALA

Full-Stack Engineering

HTML, CSS, JavaScript

2026.7

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



1. Web 개요
2. HTML 기초
3. HTML Form
4. HTML 심화
5. CSS 기초
6. CSS 심화
7. JavaScript 기초
8. JavaScript 심화

교수 소개

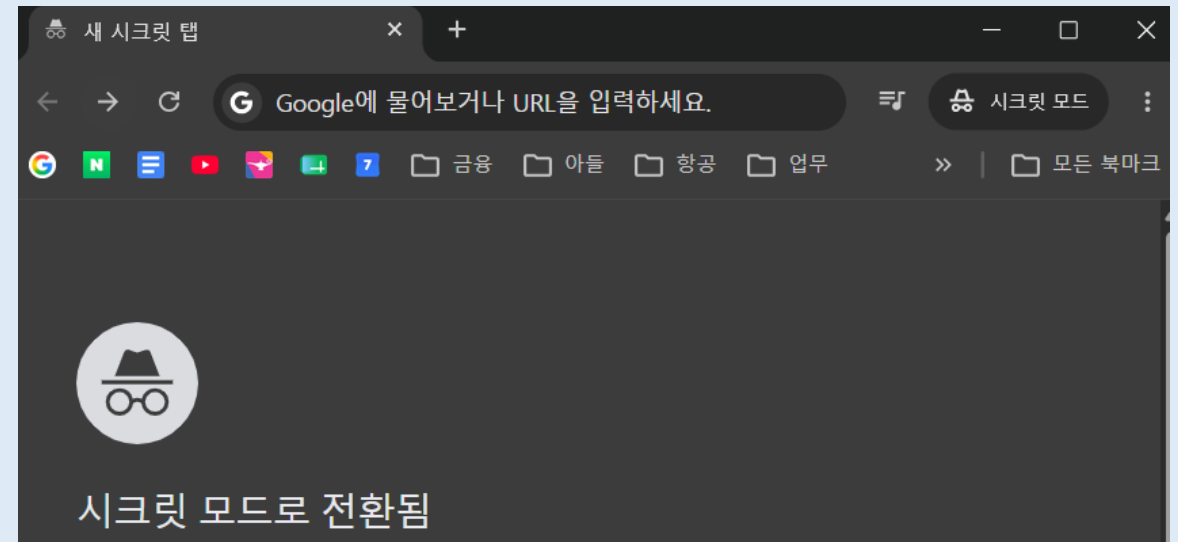


[소스 코드]

- <https://github.com/bottletiger/skala-front>
 - Git Clone
 - Download ZIP
 - GitHub Fork

[과제 제출]

- 실습 내용을 제출한다.
 - 본인의 GitHub 계정에 **Public** 저장소를 생성한다. (Private 설정 시 채점 불가).
 - 저장소 주소 예: <https://github.com/교육생GitHub계정/skala-front>
 - 제출 전 확인
 - Browser 시크릿 창을 켜고 저장소 주소로 접속했을 때, 로그인 요구 없이 소스코드가 정상적으로 잘 보이는지 더블 체크 후 제출한다.
 - Chrome Secret Mode 켜는 법
- Windows / Linux: Ctrl + Shift + N
- macOS (Mac): Command(⌘) + Shift + N

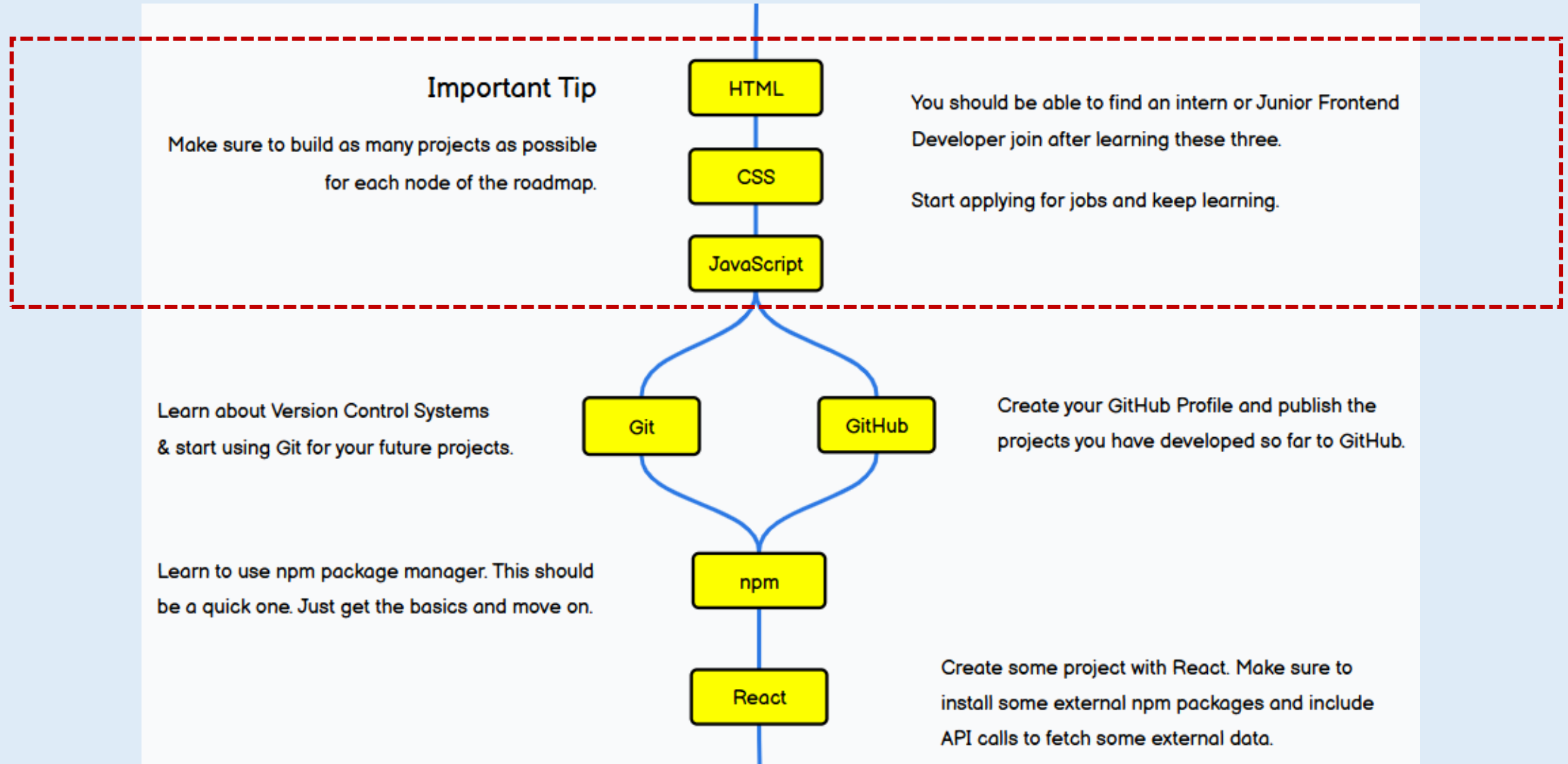


[과제 제출]

■ 종합과제 평가 기준표 (Rubric)

점수 구간	평가 등급	기술적 및 실무적 평가 기준
91 ~ 100점	Excellent	- 기본 요구사항을 완벽히 충족 - 지시되지 않은 내용을 추가로 실습한 경우
81 ~ 90점	Good	- 기본 요구사항을 완벽히 충족 - 단순 복사-붙여넣기가 아닌 충분한 개인 실습 흔적이 보이는 경우
71 ~ 80점	Fair	- 안내된 기본 요구사항(최소 스펙)을 부분 충족 - 지정된 형식에 맞춰 GitHub Public Repository URL을 올바르게 제출한 경우
61 ~ 70점	Poor	- 과제를 제출했으나 요구사항의 핵심 기능 중 상당 부분이 누락된 경우 - 제출한 GitHub URL이 비공개(Public) 설정되어 있어 소스코드 확인 및 채점이 불가능한 경우

Frontend Beginner Roadmap



Full-Stack Engineering - HTML, CSS, JavaScript

1. Web 개요

Internet - History

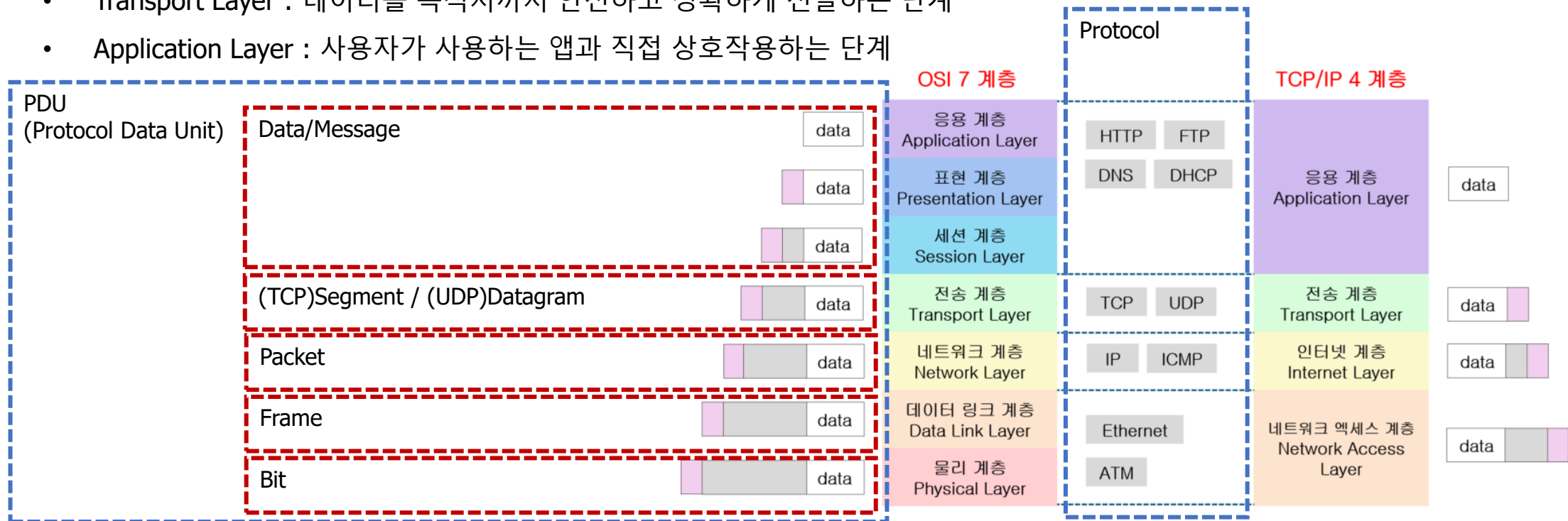
- 전 세계적으로 **연결**된 컴퓨터 네트워크
- 데이터를 주고받을 수 있도록 표준화된 프로토콜을 사용 : 웹, 이메일, 파일 등
- 인터넷의 역사
 - 1960년대: 미국 국방부 ARPANET(Advanced Research Projects Agency Network) 프로젝트에서 시작
 - 1970~1980년대: TCP/IP 프로토콜 개발, ARPANET의 성장
 - 1990년대: WWW(World Wide Web)의 등장 ➔ 대중화
 - 2000년대: 모바일 인터넷, Cloud, IoT(Internet of Things) 등으로 발전

Internet - Physical Infrastructure

- Client & Server (End Systems, 종단시스템)
 - 네트워크의 가장자리(Edge)에 위치하며, 프로토콜을 직접 실행하고 데이터를 생성하거나 최종적으로 소비하는 단말 기기
 - Client: 서비스를 요청하는 단말기. 스마트폰, PC, 노트북 등이 이에 해당하며, 웹 브라우저 등을 통해 데이터를 요청한다.
 - Server: 요청에 응답하여 서비스를 제공하는 고성능 컴퓨터. Web Server, Database Server 같이 서비스를 제공한다.
 - 고유한 IP 주소를 할당받아 네트워크 식별자로 사용되며, 웹 브라우저나 앱 같은 네트워크 애플리케이션 수준의 프로세스를 구동한다.
- 전송 매체 (Transmission Media)
 - 종단 시스템과 네트워크 장비, 혹은 장비와 장비 사이를 물리적으로 연결하여 데이터 신호가 물리적 층 위에서 이동할 수 있는 채널을 제공하는 매체
 - 유선 매체: 광케이블(Optical Fiber Cable), Ethernet Cable(LAN선), 해저 광케이블(대륙간 연결)
 - 무선 매체: Wi-Fi, 5G/LTE, 위성통신 등으로 공기 중으로 신호를 전달한다.
- 네트워크 장비(Network Equipment)
 - 종단 시스템이 전송한 데이터 조각(Packet)의 헤더 정보를 파싱하여, 최적의 경로를 설정하고 다음 목적지로 데이터를 포워딩(Forwarding)하는 네트워크 노드 장비
 - Router: 서로 다른 네트워크를 연결하고, 데이터 패킷이 가장 효율적인 경로로 이동할 수 있도록 경로를 배정(라우팅)하는 장치
 - Switch: 하나의 네트워크 안에서 여러 컴퓨터와 장비들을 유선으로 연결하고 데이터를 분배하는 장치입니다

Internet - Protocol

- 인터넷에서 데이터를 주고 받기 위한 일정한 규칙과 약속으로 네트워크 기능을 계층(Layer)별로 정의한다.
 - OSI 7 Layers: 학술적이고 이론적인 모델 표준 (네트워크 구조를 이해하기 좋음)
 - TCP/IP 4 Layers: 실무적이고 실용적인 모델 (인터넷의 사실상 표준)
- TCP/IP (인터넷 표준 Protocol Suite)
 - Network Access Layer : 물리적인 케이블, 랜카드 등을 통해 데이터를 비트(0,1) 단위로 전달
 - Internet Layer : 데이터가 이동할 최적의 경로를 찾고 IP 주소를 부여하는 단계
 - Transport Layer : 데이터를 목적지까지 안전하고 정확하게 전달하는 단계
 - Application Layer : 사용자가 사용하는 앱과 직접 상호작용하는 단계



Internet - Network Access Layer Protocol

■ Network Access Layer

- OSI 모델의 물리 계층(Physical Layer)와 데이터 링크 계층(Data Link Layer)을 통합한 계층이다.
- 물리적인 네트워크 매체(광케이블, LAN선, 무선 주파수)를 통해 데이터를 전기 신호로 변환하여 실제로 전송하는 계층
- 데이터 단위: Frame

■ 핵심 Protocol

- Ethernet : 가장 널리 쓰이는 유선 네트워크 규격
- Wi-Fi(IEEE 802.11) : 무선 로컬 네트워크 규격

■ MAC (Media Access Control) Address

- IP 주소가 컴퓨터의 논리적 위치인 반면에 MAC 주소는 하드웨어 고유한 물리적 식별자
- 하드웨어 내부에 영구적으로 기록되는 주소로 값이 변경되지 않는다.
- 48비트(6바이트)로 16진수 12자리로 표기

■ 내 컴퓨터의 상세 네트워크 Hardware/IP 정보 확인

- Windows : ipconfig /all
- macOS : ifconfig

```
C:\Users\03295>ipconfig /all

이더넷 어댑터 이더넷:

   미디어 상태 . . . . . : 미디어 연결 끊김
   연결별 DNS 접미사 . . . . :
   설명 . . . . . : Realtek PCIe GbE Family Controller
   물리적 주소 . . . . . : 8C-B0-E9-D8-30-A8
   DHCP 사용 . . . . . : 예
   자동 구성 사용 . . . . . : 예

무선 LAN 어댑터 로컬 영역 연결* 1:

   미디어 상태 . . . . . : 미디어 연결 끊김
   연결별 DNS 접미사 . . . . :
   설명 . . . . . : Microsoft Wi-Fi Direct Virtual Adapter
   물리적 주소 . . . . . : BC-F1-71-88-B9-1E
   DHCP 사용 . . . . . : 예
   자동 구성 사용 . . . . . : 예
```

MAC

MAC

Internet - Internet Layer Protocol

■ Internet Layer

- 논리적인 주소인 IP를 사용하여 서로 다른 네트워크 간의 통신을 가능하게 한다.
- 핵심 Protocol: IP(Internet Protocol), ICMP(Internet Control Message Protocol)
- 데이터 단위: Packet

■ IP Address

- 인터넷에서 각 장치를 식별하는 고유 주소
- IPv4
 - 192.168.0.1 처럼 3개의 점으로 구분된 4개의 숫자로 표현
 - 약 43억 개의 주소, 사용량이 급증하면서 현재 주소가 거의 고갈
- IPv6
 - IPv4의 주소 부족 문제를 해결하기 위해 등장
 - 주소 예) 2001:0db8:85a3:0000:0000:8a2e:0370:7334
 - 사실상 무한대에 가까운 주소를 제공하므로 주소 고갈 걱정이 없다.

Private IP

```
C:\Users\03295>ipconfig /all

무선 LAN 어댑터 Wi-Fi:

연결별 DNS 접미사 . . . . . :
설명 . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
물리적 주소 . . . . . : BC-F1-71-88-B9-1D
DHCP 사용 . . . . . : 예
자동 구성 사용 . . . . . : 예
IPv4 주소 . . . . . : 172.16.20.59(기본 설정)
서브넷 마스크 . . . . . : 255.255.254.0
임대 시작 날짜 . . . . . : 2026년 7월 9일 목요일 오전 9:06:08
임대 만료 날짜 . . . . . : 2026년 7월 9일 목요일 오후 7:06:08
기본 게이트웨이 . . . . . : 172.16.20.1
DHCP 서버 . . . . . : 172.16.20.3
DNS 서버 . . . . . : 168.126.63.1
                        8.8.8.8
Tcpip를 통한 NetBIOS . . . . : 사용
```

■ 공인 IP (Public IP)와 사설 IP (Private IP)

- Public IP: 전 세계 인터넷 망(WAN, Wide Area Network)에서 유일무이하게 식별되는 식별자
- Private IP: 폐쇄된 내부 네트워크(LAN, Local Area Network) 안에서만 사용되는 식별자
- NAT(Network Address Translation): 하나의 공인IP 주소를 여러 개의 사설IP 주소로 변환하거나 그 반대로 변환하는 기술

Internet - Transport Layer Protocol

■ Transport Layer

- 데이터 전송의 신뢰성을 보장한다.
- 핵심 Protocol: TCP, UDP
- 데이터 단위: Segment

■ TCP (Transmission Control Protocol)

- 연결 지향적 프로토콜 : 데이터를 주고받기 전에 송신측과 수신측이 서로 연결을 확립하는 과정(3-Way Handshake)을 거친다.
- 높은 신뢰성 및 순서 보장: 데이터가 중간에 유실되면 다시 재전송을 요청하여 목적지에 도착하도록 보장한다.
- 단점: 연결을 설정하고 확인하는 과정이 반복되므로 UDP에 비해 속도가 느리고 오버헤드가 크다.
- 주요 용도: 신뢰성이 절대적으로 중요한 서비스 - 웹(HTTP), 파일 전송(FTP), 이메일(SMTP)

■ UDP (User Datagram Protocol)

- 비연결형 프로토콜: 연결 확립 과정 없이, 목적지 주소만 확인하면 확인 신호도 받지 않고 데이터를 일방적으로 던지듯 보낸다.
- 장점: 수신측이 데이터를 잘 받았는지, 순서가 맞는지 신경 쓰지 않기 때문에 헤더가 가볍고 전송 속도가 매우 빠르다.
- 단점: 데이터가 중간에 유실되거나 순서가 뒤바뀌더라도 이를 해결하지 않는 비신뢰성 구조이다.
- 주요 용도: 실시간성이 중요하고 약간의 데이터 손실은 허용되는 서비스 - 실시간 스트리밍, 온라인 게임

■ 내 컴퓨터의 네트워크 연결 및 열려 있는 포트 전체 현황 확인 명령어

- Windows : netstat -an
- macOS : lsof -I TCP -P -n

```
C:\Users\03295>netstat -an

활성 연결

프로토콜 로컬 주소          외부 주소          상태
TCP      0.0.0.0:135      0.0.0.0:0          LISTENING
TCP      0.0.0.0:445      0.0.0.0:0          LISTENING
TCP      0.0.0.0:5040     0.0.0.0:0          LISTENING
TCP      0.0.0.0:48664    0.0.0.0:0          LISTENING
```

Internet - Application Layer Protocol

▪ Application Layer 주요 Protocol

- HTTP/HTTPS - 웹 브라우저와 서버간의 자원(HTML, CSS, JS, 이미지 등) 전송을 담당하는 가장 대표적인 프로토콜
- DNS (Domain Name System) - 도메인 네임을 IP 주소로 변환해주는 역할
- FTP (File Transfer Protocol) / SSH (Secure Shell) - 파일 전송 및 보안이 강화된 원격 접속 서비스를 제공하는 프로토콜
- SMTP/POP3/IMAP - 이메일 송수신을 위한 프로토콜

▪ nslookup

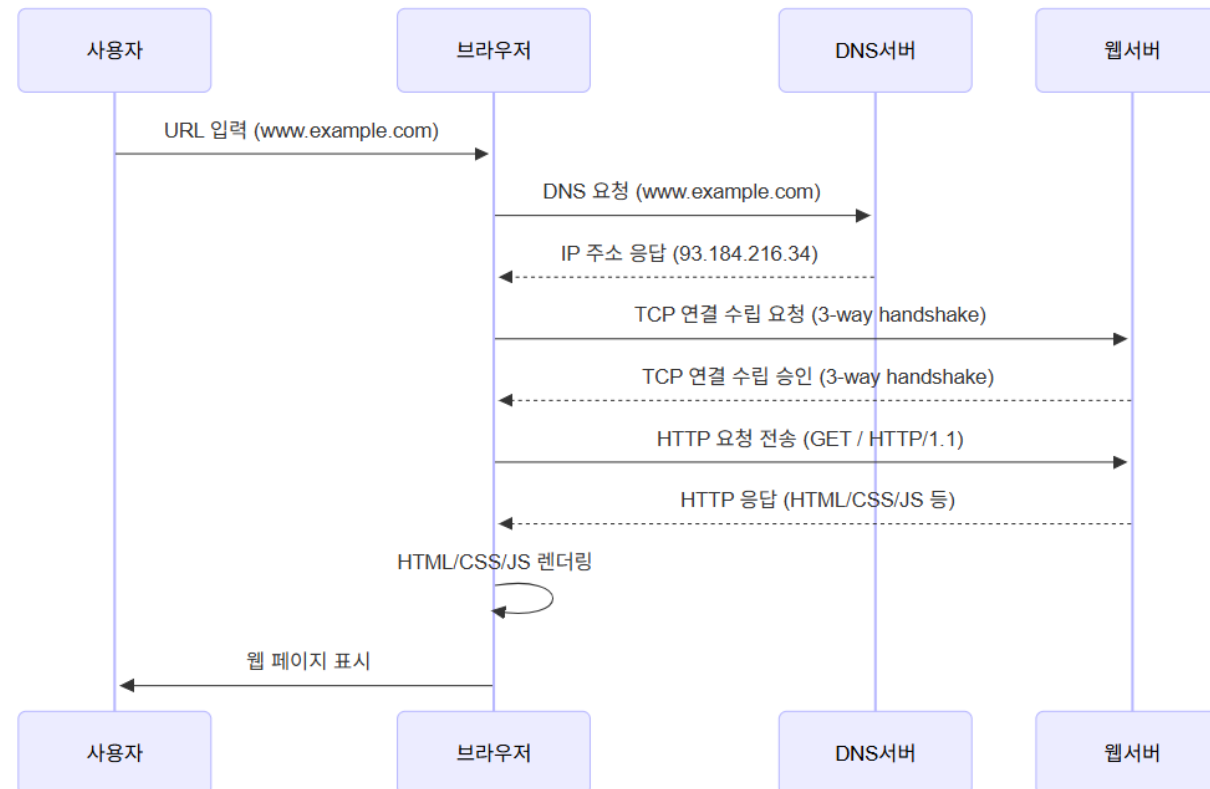
- Nslookup은 DNS에 질의하여 특정 도메인의 IP 주소를 확인할 수 있는 명령어 (Windows, macOS 동일)

```
C:\Users\03295>nslookup naver.com
서버:      dns.google
Address:  8.8.8.8

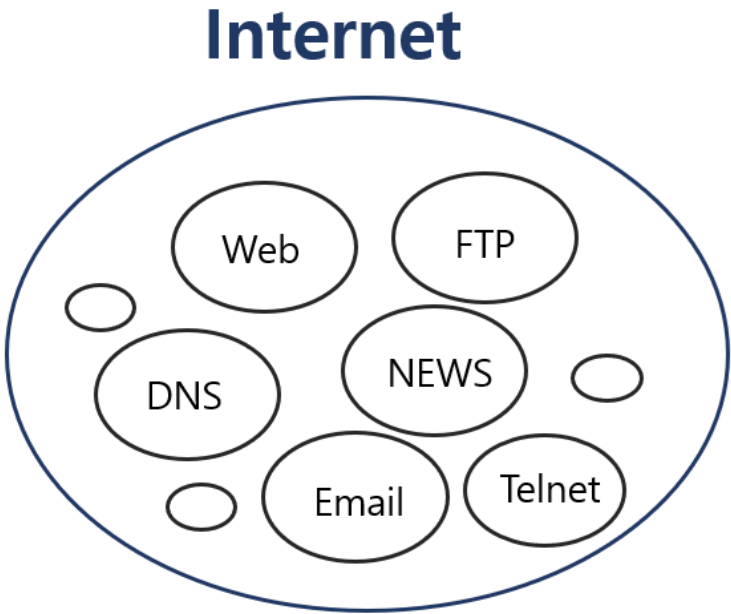
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
권한 없는 응답:
이름:      naver.com
Addresses: 223.130.192.247
           223.130.200.236
           223.130.200.219
           223.130.192.248
```

Internet - Web Connection Workflow

1. DNS 서버에 IP 주소를 문의
2. TCP 3-way Handshake 과정을 통해 두 컴퓨터(클라이언트와 서버)가 서로 신뢰할 수 있는 통신 통로를 수립
 - TCP 3-Way Handshake : SYN (연결요청) → SYN+ACK (수락응답+연결요청) → ACK (연결확립)의 3단계 과정
3. 통신 통로가 수립되면 안전하게 필요 데이터를 주고받기 시작한다.



- WEB(=WWW, World Wide Web)의 정의
 - 인터넷 상에서 정보를 주고받는 서비스의 한 형태
 - 웹 페이지를 통해 정보를 사용자에게 제공하는 서비스
 - 웹 페이지는 HTML로 작성되며 웹 브라우저를 통해 열람 가능
- WEB의 3대 개념
 - 어떻게 - HTTP (HyperText Transfer Protocol)
 - 어디로 - URL (Uniform Resource Locator)
 - 무엇을 - HTML (HyperText Markup Language)
- Internet vs World Wide Web



구분	인터넷 (Internet)	웹 (World Wide Web, WWW)
정의	전 세계 컴퓨터 네트워크를 연결한 물리적인 통신망 (인프라)	인터넷망 위에서 정보를 공유할 수 있도록 만든 정보 공간 (서비스)
핵심 구성 요소	케이블, 라우터, 스위치, 컴퓨터(하드웨어)	HTML, HTTP, URL, 웹 브라우저(소프트웨어)
기준 계층 (TCP/IP)	하위 계층 (인터넷 계층, 전송 계층 중심)	최상위 계층 (응용 계층)
포함 관계	전체 인프라 (웹을 포함한 모든 서비스의 기반)	인터넷이라는 인프라 위에 구동되는 수많은 서비스 중 하나
다른 서비스 예시	이메일(SMTP), 파일 전송(FTP), 원격 접속(SSH) 등	-

WWW - Client와 Server

▪ Client

- 웹 페이지를 요청하는 사용자의 컴퓨터 또는 장치
- 웹 브라우저는 대표적인 클라이언트 소프트웨어
- 대표 웹 브라우저: Chrome, Edge, Safari

▪ Server

- 클라이언트의 요청을 받아서 처리하고 데이터를 제공하는 컴퓨터
- 웹 서버는 HTML 페이지, 이미지, CSS, JavaScript 등을 저장하고 클라이언트에 전달
- 대표 웹 서버

Web Server	설명
Apache	• Apache Software Foundation (1995년) • 오픈소스, 모든 운영체제에서 동작 가능
Nginx	• 오픈소스 + 상용, 모든 운영체제에서 동작 가능 • 빠른 속도와 가벼운 리소스, 이벤트 기반 비동기 처리 방식
Microsoft IIS	• Microsoft (1995년) • Windows 전용으로 Windows 생태계에 최적화

URL (Uniform Resource Locator)

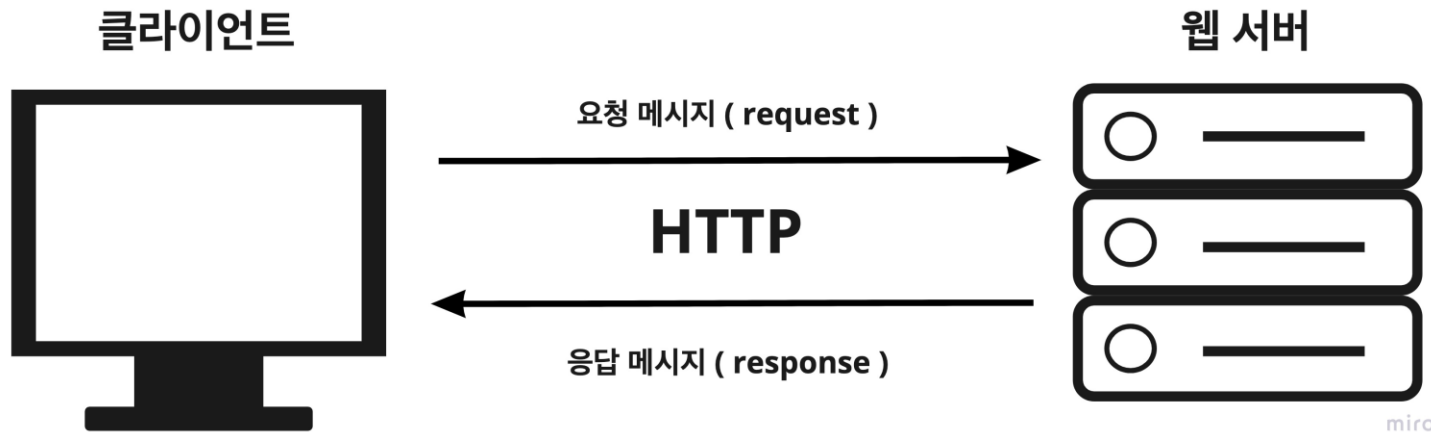
■ URL (Uniform Resource Locator)

- 네트워크 상에서 자원이 어디 있는지를 알려주기 위한 규약으로 리소스의 위치와 액세스에 사용되는 주소를 나타낸다.
- URL은 아래의 그림 예와 같이 통신 프로토콜과 서버의 위치를 알려주는 도메인 주소 또는 IP주소, 포트 번호, 디렉터리 경로, 파일문서 명, 매개변수 파라미터와 쿼리 등이 포함된 구조로 이루어져 있다



HTTP

- HTTP (**H**yper **T**ext **T**ransfer **P**rotocol)
 - Communication between client computers and web servers is done by sending **HTTP Requests** and receiving **HTTP Responses**
- HTTP Request / Response
 1. A client (a browser) sends an HTTP request to the web
 2. A web server receives the request
 3. The server runs an application to process the request
 4. The server returns an HTTP response (output) to the browser
 5. The client (the browser) receives the response



HTTP - Request

- HTTP Request 구성요소
 1. Request Line : **Request Method**, URL, HTTP 버전
 2. Request Header : User-Agent, Content-Type 등
 3. Request Body : Server로 보낼 Data
- HTTP Methods
 - **GET**: request data from a specified resource.
 - **POST**: send data to a server to create/update a resource.
 - PUT, HEAD, DELETE, PATCH, OPTIONS, CONNECT, TRACE

Request Line	<code>POST /test/demo_form.php HTTP/1.1</code> <code>Host: w3schools.com</code>	<code>GET /posts/1 HTTP/1.1</code> <code>Host: jsonplaceholder.typicode.com</code> <code>Connection: keep-alive</code> <code>User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...</code> <code>Accept: application/json, text/plain, */*</code> <code>Accept-Encoding: gzip, deflate, br</code> <code>Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7</code>
Request Body	<code>name1=value1&name2=value2</code>	

HTTP - Response

■ HTTP Response 구성요소

1. Status Line : HTTP Version, Status Code, Status Message
2. Response Header : Content-Type, Server 등
3. Response Body : HTML, JSON 등

■ HTTP Status Codes

- 1xx(Informational)
- 2xx(Success): **200 OK**, 201 Created
- 3xx(Redirection): **304 Not Modified**
- 4xx(Client Error): 400 Bad Request, **404 Not Found**
- 5xx(Server Error): **500 Internal Server Error**

Status Line	HTTP/1.1 200 OK
Response Header	Date: Mon, 08 Jun 2026 16:50:00 GMT Content-Type: application/json; charset=utf-8 Connection: keep-alive
Response Body	{ "userId": 1, "id": 1, "title": "delectus aut autem", "completed": false }

HTTP - HTTPS (Hypertext Transfer Protocol Secure)

- HTTP의 보안 버전으로 SSL(Secure Sockets Layer)/TLS(Transport Layer Security) 암호화를 사용하여 데이터 보호
 - HTTPS 암호화를 사용하면 Request Headers, Request Body, Response Headers, Response Body가 암호화된다.
 - 목적지 주소 (도메인과 IP)는 암호화 되지 않는다.

```
POST /login HTTP/1.1
Host: my-shopping-mall.com
Content-Type: application/json
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...)
Cookie: session_id=abc123xyz

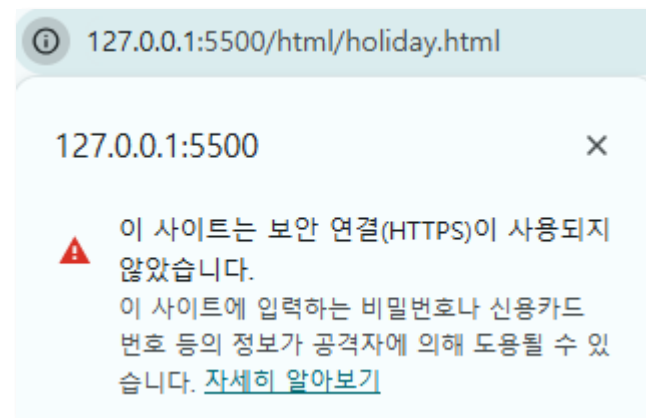
{
  "id": "admin",
  "pw": "1234"
}
```



```
POST /???????? HTTP/1.1
Host: my-shopping-mall.com (👉 목적지 배달을 위해 이 주소만 유일하게 노출됨)
????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????

🔒 [여기서부터 바디까지 통째로 암호화된 데이터 구역]
exK9x8dB4mP9z2vR7wJq/sG5tN1Y3hL6uA8cE0oI2fX4rV3pM
bC5zN2qW1eY8uI3o0pA9sD4fG7hJ1kL3zX5cV2bN8mQ0wE4r
T7yU1iI3o0pA2sD4fG6hJ8kL0zX2cV4bN6mQ8wE0rT2yU4iI
```

- HTTPS는 현재 웹사이트의 기본 표준이며, Google 크롬 같은 브라우저는 HTTPS를 사용하지 않는 웹사이트에 대해 "보안 위험 경고"를 표시



HTML - 개요

- HTML(Hyper-text Markup Language) is the standard markup language for creating Web pages
 - Hyper-text: 링크를 통해 웹 페이지 간의 이동이 가능
 - Markup Language: 태그를 사용해 문서의 구조와 스타일을 정의
- 웹 브라우저는 HTML 문서를 해석하여 사용자가 볼 수 있는 웹 페이지로 렌더링(Rendering)된다.
- HTML은 텍스트, 이미지, 링크, 폼, 멀티미디어 등 다양한 콘텐츠를 포함하여 구조화한다.
- HTML문서는 다양한 기기와 브라우저에서 작동한다.
- HTML문서는 CSS와 JavaScript를 통해 스타일링과 동작을 추가할 수 있다.

[참고 - Markup vs. Markdown]

구분	Markup	Markdown
정의	문서의 구조와 스타일을 태그로 지정하는 언어 HTML, XML	간단한 문서 작성용 경량 마크업 언어 - README, 블로그 작성: (.md)
목적	콘텐츠의 구조화 및 표현 제어 (스타일 포함)	텍스트 중심의 간단한 서식 표현
문법	비교적 복잡하고 태그 기반 예) 제목	직관적이고 간단한 기호 기반 예) # 제목

HTML - History

- 1989년 - Tim Berners-Lee 의해 처음 www 개발
- 1991년 - Tim Berners-Lee 의해 처음 HTML 개발
- 1995년 - HTML Working Group에 의해 표준화 (HTML 2.0)
 - 기본적인 폼과 테이블 요소가 추가
- 1997년 - W3C(World Wide Web Consortium)에서 관리 시작 (HTML 3.2)
 - JavaScript 및 CSS가 도입됨
- 1999년 - W3C에서 HTML 4.01 발표
- 2000년 - W3C에서 XHTML 1.0 발표
- 2014년 - W3C에서 HTML5 발표
 - 멀티미디어 콘텐츠를 기본적으로 포함하고 Semantic 태그 구조 명확화 → 브라우저 호환성
- 2017년 - W3C에서 HTML 5.2 발표

HTML - Page Structure



- 1 DTD : 문서형 정의 (<!DOCTYPE html>는 HTML5 문서라는 것을 의미한다.)
- 2 html : HTML 문서의 시작과 끝
- 3 head : 문서 정보 정의
- 4 body : 웹 브라우저에 표시할 내용

HTML - 주요 태그 (문서 구조와 Metadata)

카테고리	태그	설명
문서 구조	<html>	HTML 문서의 루트 요소로, 문서의 시작과 끝을 표시
	<head>	메타데이터(문서 제목, 스타일, 스크립트 등)를 정의
	<title>	브라우저 탭에 표시되는 문서의 제목
	<body>	문서의 본문 콘텐츠를 정의
Metadata	<meta>	문서의 메타데이터를 정의하며, SEO 및 브라우저 동작에 영향을 줌
	<link>	외부 리소스(CSS 파일 등)와의 관계를 정의
	<script>	JavaScript 코드를 삽입하거나 외부 스크립트를 연결

HTML - 주요 태그 (Text/Contents)

카테고리	태그	설명
Text/Contents	<h1> ~ <h6>	제목(Heading)을 정의하며, <h1>이 가장 크고 <h6>이 가장 작음
	<p>	단락을 정의
	 	줄바꿈을 삽입
		인라인 요소, 텍스트를 묶어 스타일링이나 스크립트를 적용할 때 사용
		중요성을 나타내는 텍스트를 굵게 표시
		강조된 텍스트를 기울임꼴로 표시
	<a>	하이퍼링크를 생성하며, href 속성을 통해 링크 대상을 지정
	<hr>	수평선
	<mark>	형광펜 강조
	<small>	작은 글씨
	<blockquote>	인용 문단
	<cite>	출처 인용
	<code>	코드 조각 표현
	<pre>	공백/줄바꿈 유지한 채 코드나 텍스트 표시
	<div>	의미 없는 구획(block), 스타일/스크립트 용도

HTML - 주요 태그 (List, Table)

카테고리	태그	설명
List Tag		순서 없는 목록(Unordered List)을 정의
		순서 있는 목록(Ordered List)을 정의
		목록의 항목을 정의: 및 에서 모두 사용
	<dl>	설명 목록
	<dt>	설명 제목
	<dd>	설명 내용
Table Tag	<table>	테이블(표 양식)을 정의
	<thead>	표의 머리글 부분
	<tbody>	표의 본문
	<tfoot>	표의 바닥글
	<tr>	테이블 행(Row)을 정의
	<th>	테이블의 헤더 셀을 정의하며, 기본적으로 텍스트를 굵게 표시하고 가운데 정렬
	<td>	테이블 데이터 셀을 정의

HTML - 주요 태그 (Form)

카테고리	태그	설명
Form Tag	<form>	사용자가 데이터를 입력하여 서버로 전송할 수 있는 폼을 정의
	<input>	다양한 입력 필드를 정의(텍스트, 비밀번호, 체크박스 등)
	<button>	버튼
	<label>	입력 필드에 대한 설명을 제공
	<textarea>	여러 줄의 텍스트 입력 필드
	<select>	드롭다운 선택 필드
	<option>	선택 항목
	<fieldset>	입력 항목 그룹화
	<legend>	그룹 설명 제목

HTML - 주요 태그 (Media)

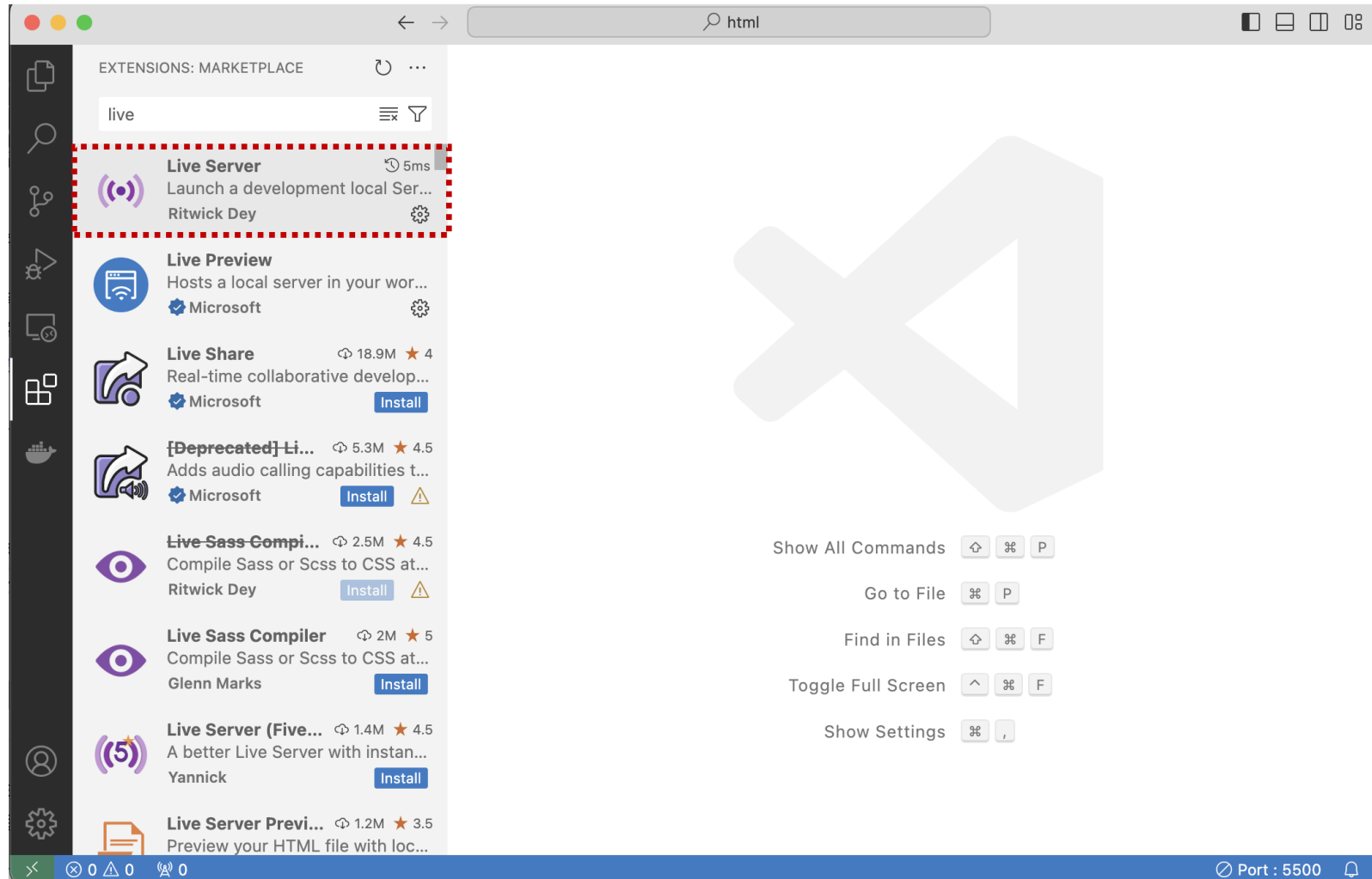
카테고리	태그	설명
Media Tag		이미지를 삽입하며, src 속성으로 이미지 경로를, alt 속성으로 대체 텍스트를 지정
	<audio>	오디오 콘텐츠를 삽입
	<video>	비디오 콘텐츠를 삽입
	<source>	<audio>나 <video>에 미디어 소스를 지정
	<track>	자막 트랙
	<figure>	미디어와 캡션 묶음
	<figcaption>	설명 캡션

HTML - 주요 태그 (Semantic)

카테고리	태그	설명
Semantic Tag	<header>	문서나 섹션의 머리글을 정의
	<footer>	문서나 섹션의 바닥글을 정의
	<main>	문서의 주 콘텐츠
	<article>	독립적인 콘텐츠를 정의
	<section>	문서의 주제를 정의하는 섹션
	<aside>	부가적인 콘텐츠(예: 사이드바)를 정의.
	<nav>	네비게이션 링크를 포함하는 섹션을 정의

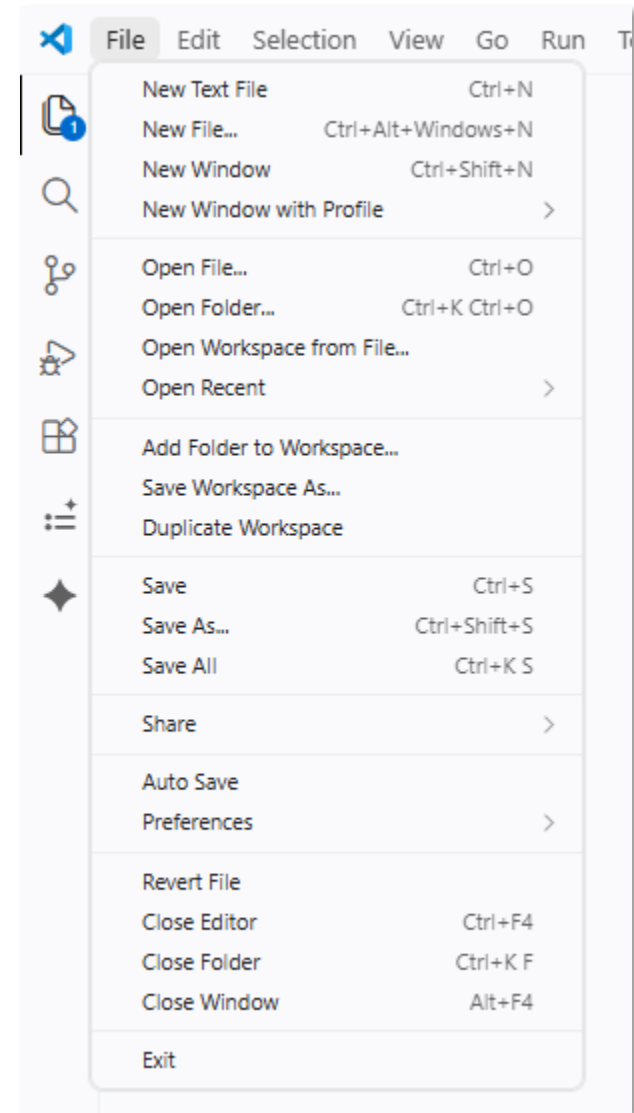
VS Code - Live Server Installation

- VS Code Live Server는 웹 개발(HTML, CSS, JavaScript)을 할 때 내가 코드를 수정하고 저장(Ctrl + S)하면, 브라우저가 알아서 새로고침 되어 실행 결과를 실시간으로 바로 확인하게 해주는 확장 프로그램(Extension)이다.



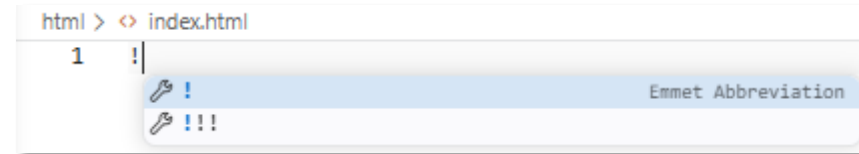
VS Code - 프로젝트 폴더 및 파일 생성

- 프로젝트 폴더 생성
- 파일생성 (index.html)



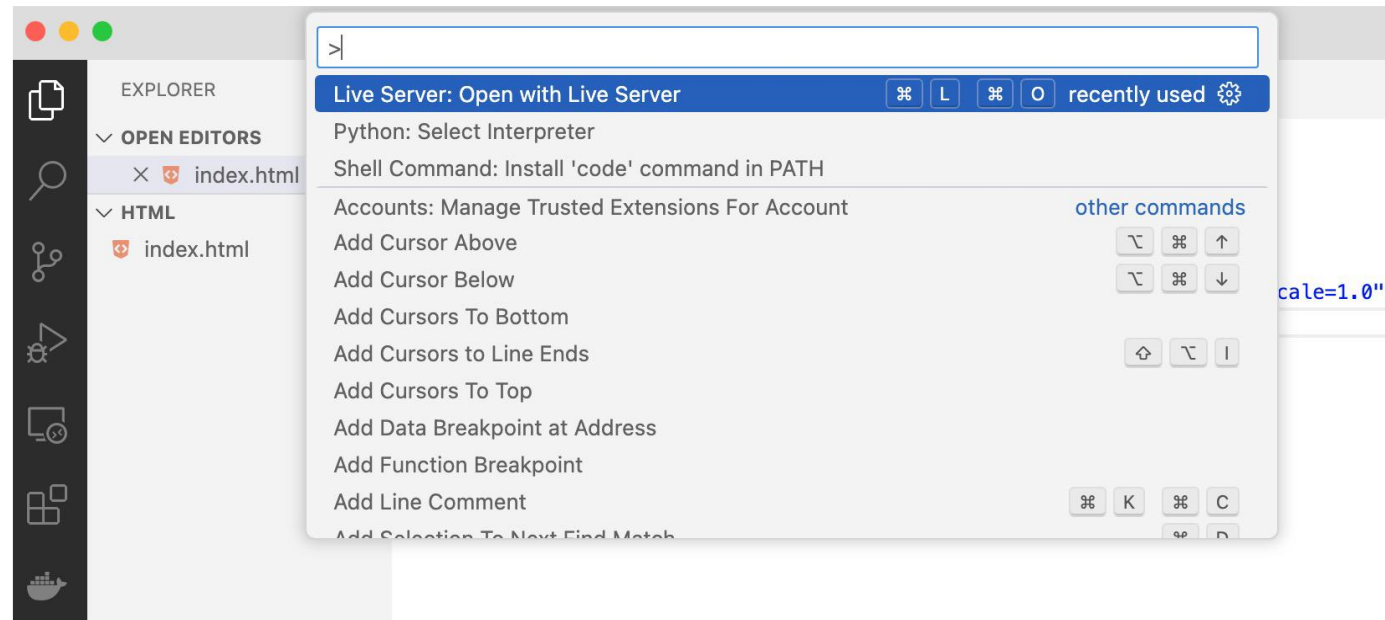
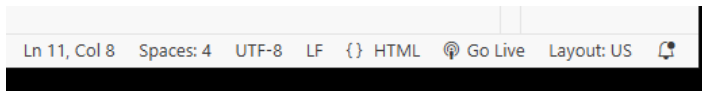
VS Code - 코드작성 Emmet(에밋)

- Emmet은 HTML, CSS 등의 코드를 빠르게 작성할 수 있도록 도와주는 코드 자동 완성 기능(Code Snippet Engine)이다.
- Emmet은 VS Code에 탑재(내장)되어 있어 별도 Extention으로 설치할 필요는 없다.
- 현재 편집 중인 파일의 이름이 .html로 끝나야지 Emmet이 동작한다.
- Emmet 활용하기
 - 뼈대코드 작성: !
 - 자식노드 생성: >
 - 형제노드 생성: +
 - 반복노드 생성: *
 - 클래스명(.) 또는 ID(#) 지정과 동시에 만들기



VS Code - 문서보기 (Open with Live Server)

- 방법1) 오른쪽 하단 "Go Live" 영역 선택
- 방법2) Ctrl + Shift + P를 클릭 (또는 검색창에 ">" 입력) Live Server: Open with Live Server 선택
- 코드를 수정하고 저장(Ctrl + S)하면, 브라우저가 알아서 새로 고침된다.

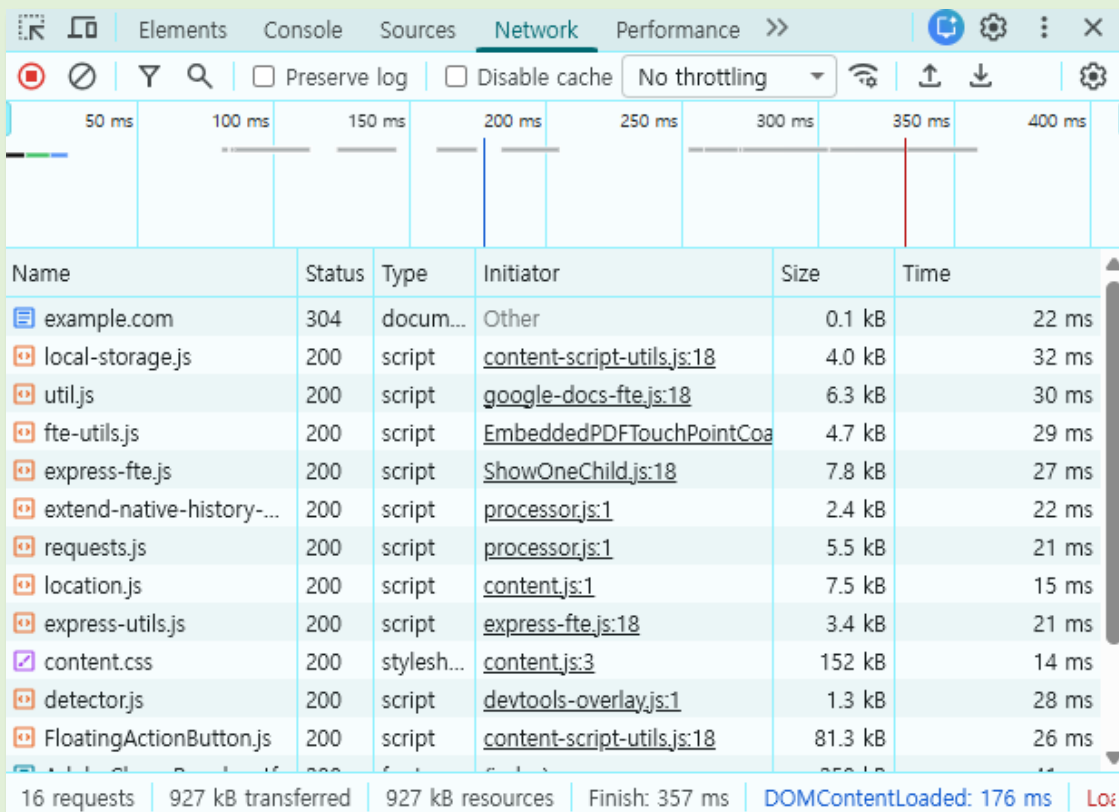


[실습] What is my IP?

- 윈도우 터미널 "ipconfig /all" 또는 맥OS 터미널 "ifconfig" 명령을 통해 내 IP를 확인
→ Private IP
- 웹 사이트 "what is my ip"
→ Public IP
- (참고) NAT (Network Address Translation, 네트워크 주소 변환)
 - NAT는 IP 패킷의 헤더에 있는 IP 주소를 다른 주소로 변환하여 라우팅하는 기술이다.
 - 주로 기관 내부의 Private IP를 인터넷상의 Public IP로 변환할 때 사용하며, 라우터 같은 네트워크 장비에서 수행된다.
 - 변환된 내역은 NAT 매핑 테이블에 기록된다.

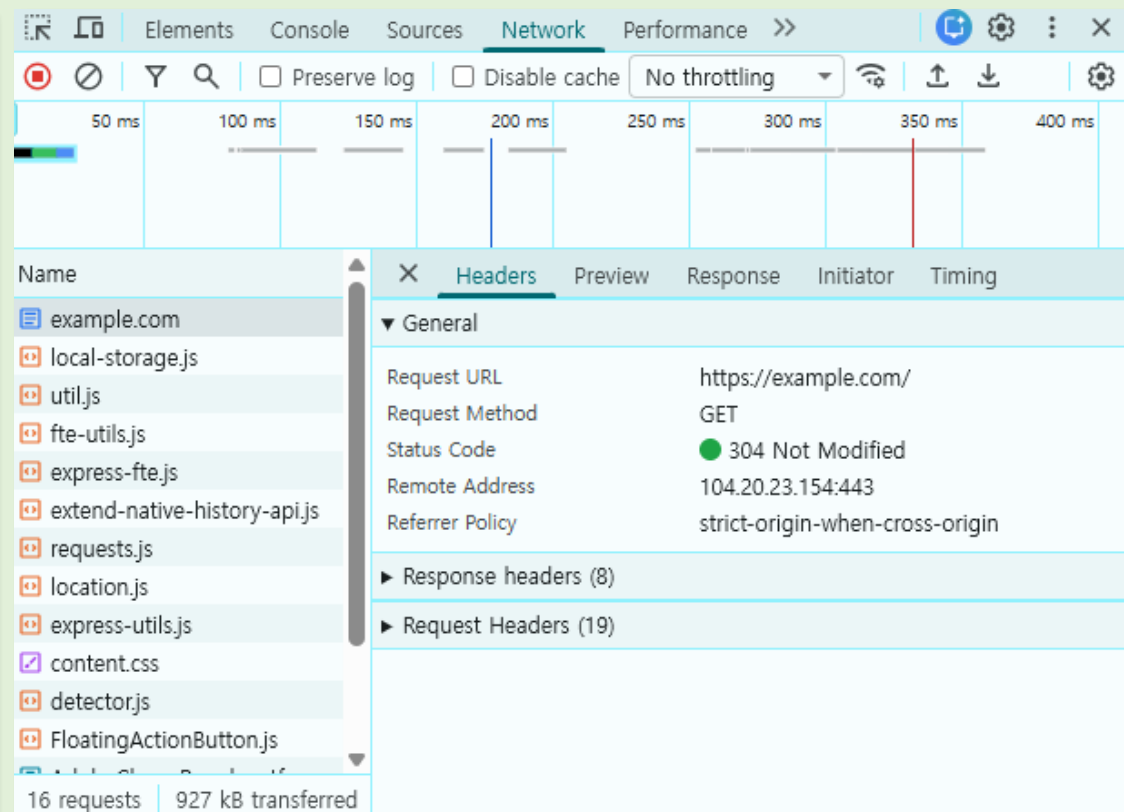
[실습] HTTP Request Circle

- Chrome 개발자 도구(F12 or Ctrl+Shift+I)를 연다.
- Network Tab으로 이동해서 Clear Networking Log를 한다.
- 주소창에 <https://example.com/> 입력한다.
- Network Tab에서 목록을 확인한다.
- Network Tab에서 example.com을 클릭한 후, Header, Preview, Response, Initiator, Timing을 확인한다.



Name	Status	Type	Initiator	Size	Time
example.com	304	docum...	Other	0.1 kB	22 ms
local-storage.js	200	script	content-script-utils.js:18	4.0 kB	32 ms
util.js	200	script	google-docs-fte.js:18	6.3 kB	30 ms
fte-utils.js	200	script	EmbeddedPDFTouchPointCoa	4.7 kB	29 ms
express-fte.js	200	script	ShowOneChild.js:18	7.8 kB	27 ms
extend-native-history-...	200	script	processor.js:1	2.4 kB	22 ms
requests.js	200	script	processor.js:1	5.5 kB	21 ms
location.js	200	script	content.js:1	7.5 kB	15 ms
express-utils.js	200	script	express-fte.js:18	3.4 kB	21 ms
content.css	200	stylesh...	content.js:3	152 kB	14 ms
detector.js	200	script	devtools-overlay.js:1	1.3 kB	28 ms
FloatingActionButton.js	200	script	content-script-utils.js:18	81.3 kB	26 ms

16 requests | 927 kB transferred | 927 kB resources | Finish: 357 ms | DOMContentLoaded: 176 ms | Load: 357 ms



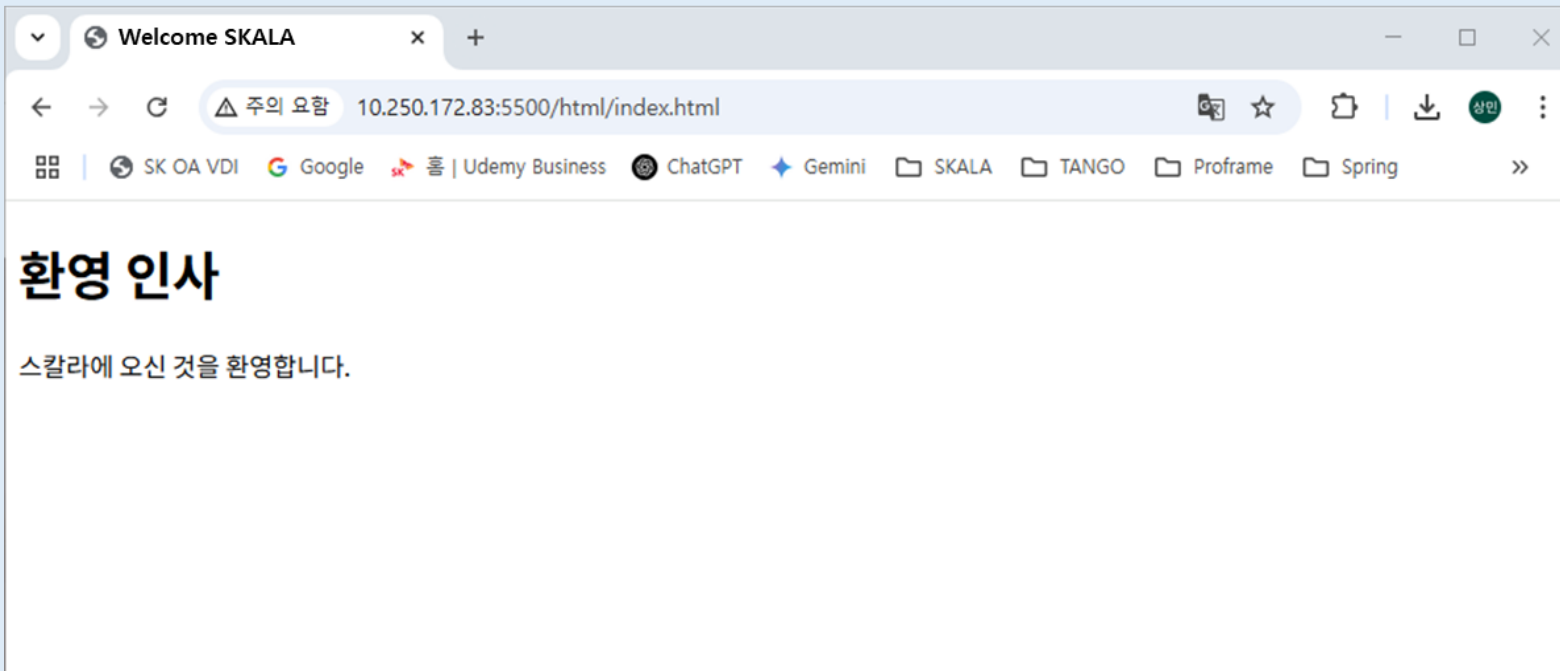
Name	Status	Type	Initiator	Size	Time
example.com	304	docum...	Other	0.1 kB	22 ms
local-storage.js	200	script	content-script-utils.js:18	4.0 kB	32 ms
util.js	200	script	google-docs-fte.js:18	6.3 kB	30 ms
fte-utils.js	200	script	EmbeddedPDFTouchPointCoa	4.7 kB	29 ms
express-fte.js	200	script	ShowOneChild.js:18	7.8 kB	27 ms
extend-native-history-...	200	script	processor.js:1	2.4 kB	22 ms
requests.js	200	script	processor.js:1	5.5 kB	21 ms
location.js	200	script	content.js:1	7.5 kB	15 ms
express-utils.js	200	script	express-fte.js:18	3.4 kB	21 ms
content.css	200	stylesh...	content.js:3	152 kB	14 ms
detector.js	200	script	devtools-overlay.js:1	1.3 kB	28 ms
FloatingActionButton.js	200	script	content-script-utils.js:18	81.3 kB	26 ms

16 requests | 927 kB transferred

Headers	
▼ General	
Request URL	https://example.com/
Request Method	GET
Status Code	● 304 Not Modified
Remote Address	104.20.23.154:443
Referrer Policy	strict-origin-when-cross-origin
▶ Response headers (8)	
▶ Request Headers (19)	

[과제] Project 구성과 index.html 생성

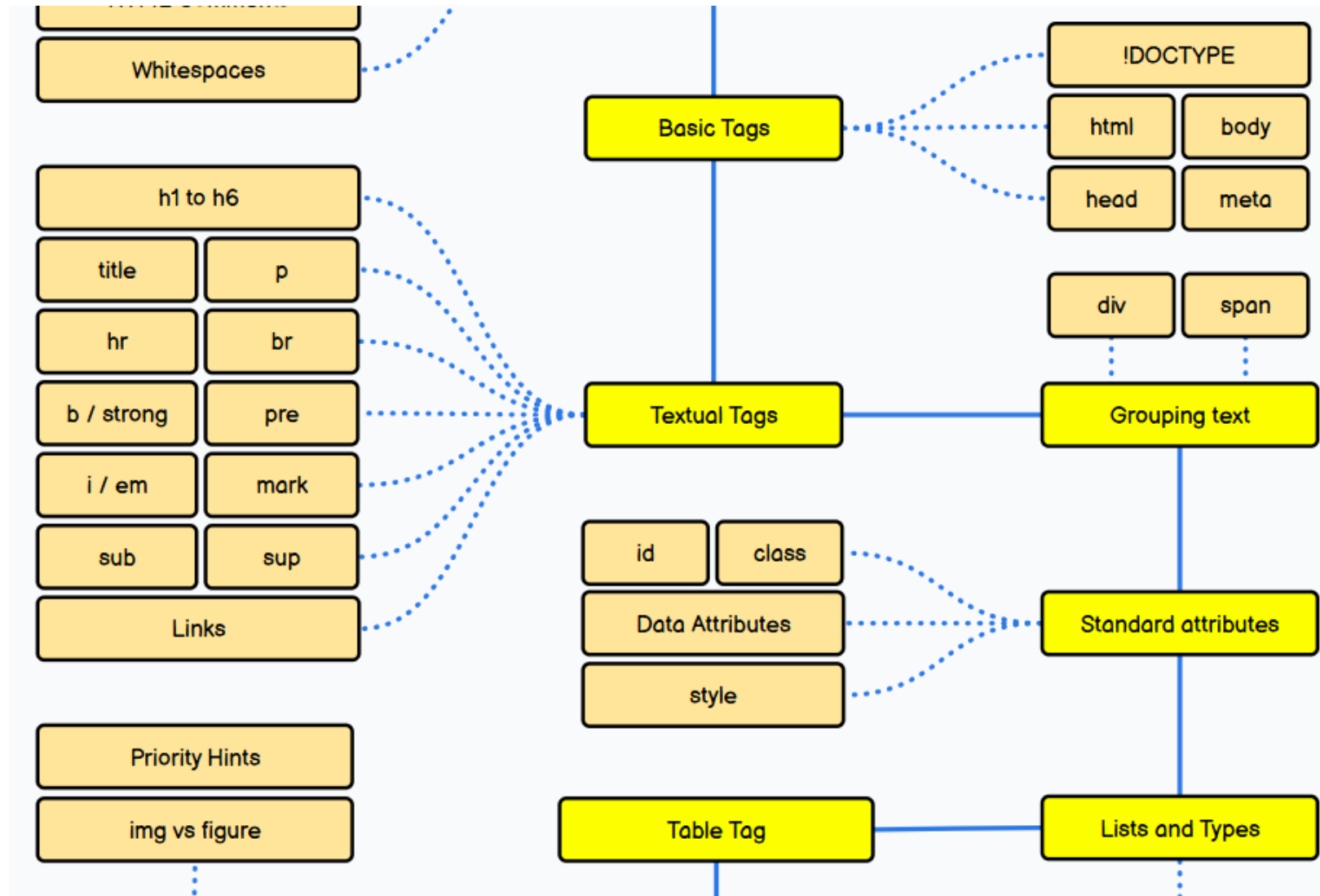
- Project Name: SKALA-FRONT
- Folder: html
- File: index.html
 - 브라우저 타이틀(title): "Welcome SKALA"
 - 본문 타이틀(h1) : "환영 인사 "
 - 본문(p): "스칼라에 오신 것을 환영합니다."
- Live Server로 확인



Full-Stack Engineering - HTML, CSS, JavaScript

2. HTML 기초

HTML Roadmap



HTML Elements

- An HTML element is defined by a start tag, some content, and an end tag
 - `<tagname>` Content goes here... `</tagname>`
- The HTML element is everything from the start tag to the end tag

Start Tag	Element Content	End Tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>
<code>
</code>	<i>none</i>	<i>none</i>

- HTML elements with no content are called empty elements.
- HTML elements can be nested (this means that elements can contain other elements).

```
<html>
  <body>
    <h1>My First Heading</h1>
    <p>My first paragraph.</p>
  </body>
</html>
```

- HTML is Not Case Sensitive

HTML Attributes

- HTML attributes provide additional information about HTML elements.

- All HTML elements can have attributes
- Attributes provide additional information about elements
- Attributes are always specified in the start tag
- Attributes usually come in name/value pairs like: name="value"

```
<a href="https://www.w3schools.com">Visit W3Schools</a>  
  
<p style="color:red;">This is a red paragraph.</p>
```

- Double quotes around attribute values are the most common in HTML, but single quotes can also be used.

```
<p title='John "ShotGun" Nelson'>  
<p title="John 'ShotGun' Nelson">
```


HTML Attributes - Global attributes

- 다음 속성은 모든 HTML Element에서 사용되는 속성이다.

Attribute	설명	예시
id	요소의 고유 식별자 - JS, CSS 등에서 참조할 때 사용	<code><div id="title">...</div></code>
class	CSS나 JS에서 참조 가능한 클래스 이름 (여러 개 가능)	<code><p class="red-color">...</p></code>
style	인라인 스타일을 지정	<code><h1 style="color:red;"></h1></code>
title	엘리먼트의 추가정보 (마우스를 위에 올리면 툴팁으로 표시)	<code></code>
lang	엘리먼트에 사용한 텍스트의 언어정보를 지정	<code><html lang="ko"></code>
hidden	요소를 화면에 표시하지 않음 (렌더링 X)	<code><div hidden>비밀 정보</div></code>
tabindex	키보드 탭 순서 지정 (작은 숫자가 먼저)	<code><button tabindex="1">확인</button></code>
draggable	요소를 드래그할 수 있는지 여부 설정 (true / false)	<code></code>
spellcheck	입력된 텍스트의 맞춤법 검사 여부 (true / false)	<code><textarea spellcheck="true" lang="kr"></textarea></code>
contenteditable	사용자가 콘텐츠를 편집할 수 있도록 설정 (true / false)	<code><div contenteditable="true">편집 가능</div></code>
dir	텍스트 방향 설정 (ltr, rtl, auto)	<code><p dir="rtl">שלום</p></code>
lang	콘텐츠 언어 지정 (en, ko, fr 등)	<code><p lang="en">Hello</p></code>
data-*	사용자가 임의의 속성을 생성	<code><p data-name="spiderMan" data-hero="true">...</p></code>

HTML Attributes - Id

- The **id** attribute specifies a **unique id** for an HTML element.
- The value of the id attribute **must be unique** within the HTML document.

```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>

<h2>The id Attribute</h2>
<p>Use CSS to style an element with the id "myHeader":</p>

<h1 id="myHeader">My Header</h1>

</body>
</html>
```

The id Attribute

Use CSS to style an element with the id "myHeader":



My Header

HTML Attributes - class

- The **class** attribute is often used to point to a class name in a style sheet.
- It can also be used by JavaScript to access and manipulate elements with the specific class name.

```
<!DOCTYPE html>
<html>
<head>
<style>
.note {
  font-size: 120%;
  color: red;
}
</style>
</head>
<body>

<h1>My <span class="note">Important</span> Heading</h1>
<p>This is some <span class="note">important</span> text.</p>

</body>
</html>
```

My Important Heading

This is some important text.

HTML Attributes - style

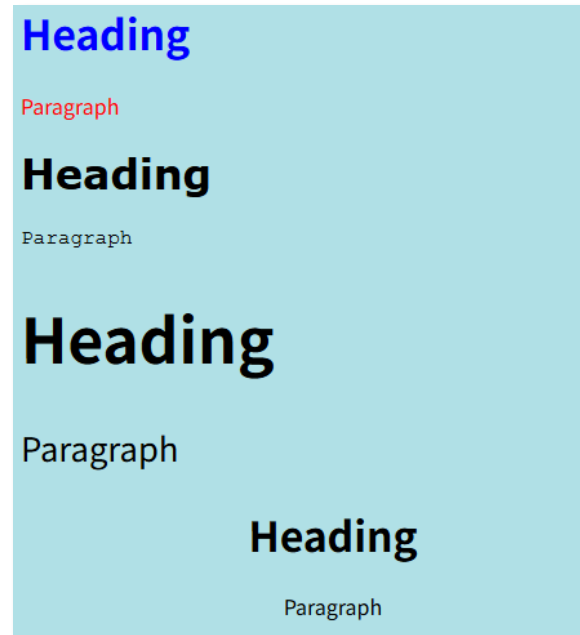
- The HTML **style** attribute is used to add styles to an element, such as color, font, size, and more.
 - `<tagname style="property:value;">`
 - The property is a CSS property. The value is a CSS value.
- The CSS **background-color** property defines the background color for an HTML element.

```
<body style="background-color:powderblue;">
  <h1 style="background-color:lightgray;">제목</h1>
  <p style="background-color:tomato;">문단</p>
</body>
```



- CSS의 color, font-family, font-size, text-align 같은 property를 통해 HTML을 꾸밀 수 있다.

```
<h1 style="color:blue;">Heading</h1>
<p style="color:red;">Paragraph</p>
<h1 style="font-family:verdana;">Heading</h1>
<p style="font-family:courier;">Paragraph</p>
<h1 style="font-size:300%;">Heading</h1>
<p style="font-size:160%;">Paragraph</p>
<h1 style="text-align:center;">Heading</h1>
<p style="text-align:center;">Paragraph</p>
```



HTML Headings - <h1>~<h6>

- **HTML headings are titles or subtitles** that you want to display on a webpage.
- 검색엔진(Search engines)이 웹페이지의 구조와 내용을 파악하는데도 사용한다.

```
<!DOCTYPE html>
<html>
<body>

<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>

</body>
</html>
```

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

HTML Paragraphs - <p>

- The HTML `<p>` element defines a **paragraph**.
- A paragraph always starts on a new line.

```
<p>This is a paragraph.</p>  
<p>This is another paragraph.</p>
```

This is a paragraph.

This is another paragraph.

- The browser will automatically remove any extra spaces and lines when the page is displayed

```
<p>  
  이 문단은  
  소스 코드에서 많은 줄 바꿈이 있지만  
  브라우저는 이를 무시합니다.  
</p>  
<p>  
  이 문단은          소스 코드에서 많은 공백이 있지만  
  브라우저는 이를    무시합니다.  
</p>
```

이 문단은 소스 코드에서 많은 줄 바꿈이 있지만 브라우저는 이를 무시합니다.

이 문단은 소스 코드에서 많은 공백이 있지만 브라우저는 이를 무시합니다.

HTML Paragraphs - <hr>/

- The <hr> tag defines a **thematic break** in an HTML page
 - 대부분 수평선을 그려서 시각적으로 구분된다.
- The HTML
 element defines a **line break**.
- <hr>과
 태그는 empty tag로 end tag가 필요하지 않다.

```
<!DOCTYPE html>
<html>
<body>
  <h1>This is heading 1</h1>
  <p>This is some text.</p>
  <hr>
  <h2>This is heading 2</h2>
  <p>This is some other text.</p>
  <hr>
  <p>This is<br>a paragraph<br>with line breaks.</p>
</body>
</html>
```

This is heading 1

This is some text.

This is heading 2

This is some other text.

This is
a paragraph
with line breaks.

HTML Block & Inline

- 모든 HTML Element는 기본 표시(display) 값을 가지고 있다.

구분	Block Elements	Inline Elements
디스플레이	화면에 새 줄에서 시작하며, 가로 폭을 가능한 한 최대로 차지	텍스트 또는 다른 인라인 요소와 같은 줄에서 표시
영역 크기	기본적으로 전체 폭을 차지	콘텐츠의 크기만큼 공간을 차지
배치	위/아래로 쌓이며(수직 정렬) 다른 요소와 겹치지 않음	다른 인라인 요소와 가로로 나란히 배치
주요 태그	<div>, <p>, <h1> ~ <h6>, , , , <table>, <form>, <section>, <article>, <header>, <footer>, <nav> 등	, <a>, , , , <i>, , <input>, <label>, <button>, <small>, <code>, <mark> 등
CSS 적용	width, height, margin, padding 모두 적용 가능	width와 height는 무시, margin과 padding은 수평 방향만 적용
용도	레이아웃 구성 및 큰 영역을 나타낼 때 사용	텍스트나 소규모 콘텐츠 강조, 링크, 스타일 적용 시 사용

HTML Inline Container -

- element는 텍스트 또는 문서의 일부를 표시하는 데 사용되는 inline container이다.
- 요소에는 필수 속성이 없지만 style, class, id 속성이 일반적으로 사용되며, CSS와 함께 사용하면서 텍스트의 특정 부분에 스타일을 지정할 수 있다.

```
<!DOCTYPE html>
<html>
<body>
  <h1>The span element</h1>
  <p>My mother has <span style="color:blue;font-weight:bold;">blue</span> eyes and my father has <span
style="color:darkolivegreen;font-weight:bold;">dark green</span> eyes.</p>
</body>
</html>
```

The span element

My mother has blue eyes and my father has dark green eyes.

HTML Div - <div>

- The `<div>` element is used as a **container for other HTML elements**.
- The `<div>` element is by default a **block element**, meaning that it takes all available width, and comes with line breaks before and after.

```
<!DOCTYPE html>
<html>
<body>
<h1>Multiple DIV Elements</h1>
<div style="background-color:#FFF4A3;">
  <h2>London</h2>
  <p>London is the capital city of England.</p>
  <p>London has over 9 million inhabitants.</p>
</div>
<div style="background-color:#FFC0C7;">
  <h2>Oslo</h2>
  <p>Oslo is the capital city of Norway.</p>
  <p>Oslo has over 700,000 inhabitants.</p>
</div>
<div style="background-color:#D9EEE1;">
  <h2>Rome</h2>
  <p>Rome is the capital city of Italy.</p>
  <p>Rome has over 4 million inhabitants.</p>
</div>
<p>CSS styles are added to make it easier to separate the divs,
and to make them more pretty:</p>
</body>
</html>
```

Multiple DIV Elements

London

London is the capital city of England.

London has over 9 million inhabitants.

Oslo

Oslo is the capital city of Norway.

Oslo has over 700,000 inhabitants.

Rome

Rome is the capital city of Italy.

Rome has over 4 million inhabitants.

CSS styles are added to make it easier to separate the divs, and to make them more pretty:)

HTML Links - <a>

- The HTML <a> tag defines a **hyperlink**.
- href 속성은 링크의 목적지 URL을 지정한다.

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```

[Visit W3Schools.com!](https://www.w3schools.com/)

- target 속성은 링크된 문서가 열리는 공간을 지정한다.
 - _self - Default. Opens the document in the same window/tab as it was clicked
 - _blank - Opens the document in a new window or tab
 - _parent - Opens the document in the parent frame
 - _top - Opens the document in the full body of the window

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

- Link Bookmarks
 - Bookmarks can be useful if a web page is very long.

```
<a href="#C4">Jump to Chapter 4</a>
...
<h2 id="C4">Chapter 4</h2>
...
```

HTML Text Formatting - ```<strong>``<i>``<em>``<small>``<mark>`...

- Formatting elements were designed to display special types of text

```
<body>
  <p><b>This text is bold</b></p>
  <p><strong>This text is important!</strong></p>
  <p><i>This text is italic</i></p>
  <p><em>This text is emphasized</em></p>
  <p><small>This is some smaller text.</small></p>
  <p>Do not forget to buy <mark>milk</mark> today.</p>
  <p>My favorite color is <del>blue</del> red.</p>
  <p>My favorite color is <del>blue</del> <ins>red</ins>.</p>
  <p>This is <sub>subscripted</sub> text.</p>
  <p>This is <sup>superscripted</sup> text.</p>
</body>
```

This text is bold

This text is important!

This text is italic

This text is emphasized

This is some smaller text.

Do not forget to buy **milk** today.

My favorite color is ~~blue~~ red.

My favorite color is ~~blue~~ red.

This is subscripted text.

This is superscripted text.

HTML Lists - Unordered Lists

- An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag.
- The CSS `list-style-type` property is used to define the style of the list item marker.

```
<ul style="list-style-type:disc;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
<ul style="list-style-type:circle;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
<ul style="list-style-type:square;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
<ul style="list-style-type:none;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

- Coffee
- Tea
- Milk

- Coffee
- Tea
- Milk

- Coffee
- Tea
- Milk

Coffee
Tea
Milk

HTML Lists - ordered Lists

- An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.
- The `type` attribute of the `<ol>` tag, defines the type of the list item marker:

```
<ol type="1">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
<ol type="A">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
<ol type="a">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
<ol type="I">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
<ol type="i">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

1. Coffee
2. Tea
3. Milk

A. Coffee
B. Tea
C. Milk

a. Coffee
b. Tea
c. Milk

I. Coffee
II. Tea
III. Milk

i. Coffee
ii. Tea
iii. Milk

HTML Lists - Nested HTML Lists

- Lists can be nested (list inside list)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document</title>
</head>
<body>
<ol>
  <li>Coffee</li>
  <li>Tea
    <ul>
      <li>Black tea</li>
      <li>Green tea</li>
    </ul>
  </li>
  <li>Milk</li>
</ol>
</body>
</html>
```

1. Coffee
2. Tea
 - Black tea
 - Green tea
3. Milk

HTML Lists - Description Lists <dl><dt><dd>

- <dl>: 정의 리스트 전체를 감싸는 태그로 정의할 항목들의 목록을 포함
- <dt>: 목록 항목의 제목으로 주로 용어나 항목 이름을 입력
- <dd>: dt에서 정의된 항목에 대한 상세 설명

```
<body>
<dl>
  <dt>HTML</dt>
  <dd>Hypertext Markup Language, 웹 페이지를 만드는 데 사용되는 마크업 언어입니다.</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets, 웹 페이지의 스타일을 정의하는 언어입니다.</dd>
  <dt>JavaScript</dt>
  <dd>웹 페이지의 동적인 기능을 구현하는 프로그래밍 언어입니다.</dd>
</dl>
</body>
```

HTML

Hypertext Markup Language, 웹 페이지를 만드는 데 사용되는 마크업 언어입니다.

CSS

Cascading Style Sheets, 웹 페이지의 스타일을 정의하는 언어입니다.

JavaScript

웹 페이지의 동적인 기능을 구현하는 프로그래밍 언어입니다.

HTML Tables

- A table in HTML consists of table cells inside rows and columns.

<table>	표 전체를 감싸는 태그
<thead>	표의 머리글(제목 행)을 정의합니다. <tr>과 <th>로 구성
<tbody>	실제 데이터 행을 담는 부분입니다. <tr>과 <td>로 구성됩니다.
<tfoot>	표의 바닥글을 정의합니다. 합계나 요약 정보를 넣는 데 사용됩니다.
<tr>	표의 한 행(Row)을 의미합니다.
<th>	머리글 셀(Header cell)을 정의하며, 기본적으로 굵은 글씨와 가운데 정렬입니다.
<td>	일반 셀(Data cell)을 정의합니다.

```
<h2>TR elements define table rows</h2>
<table style="width:100%">
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
  <tr>
    <td>16</td>
    <td>14</td>
    <td>10</td>
  </tr>
</table>
```

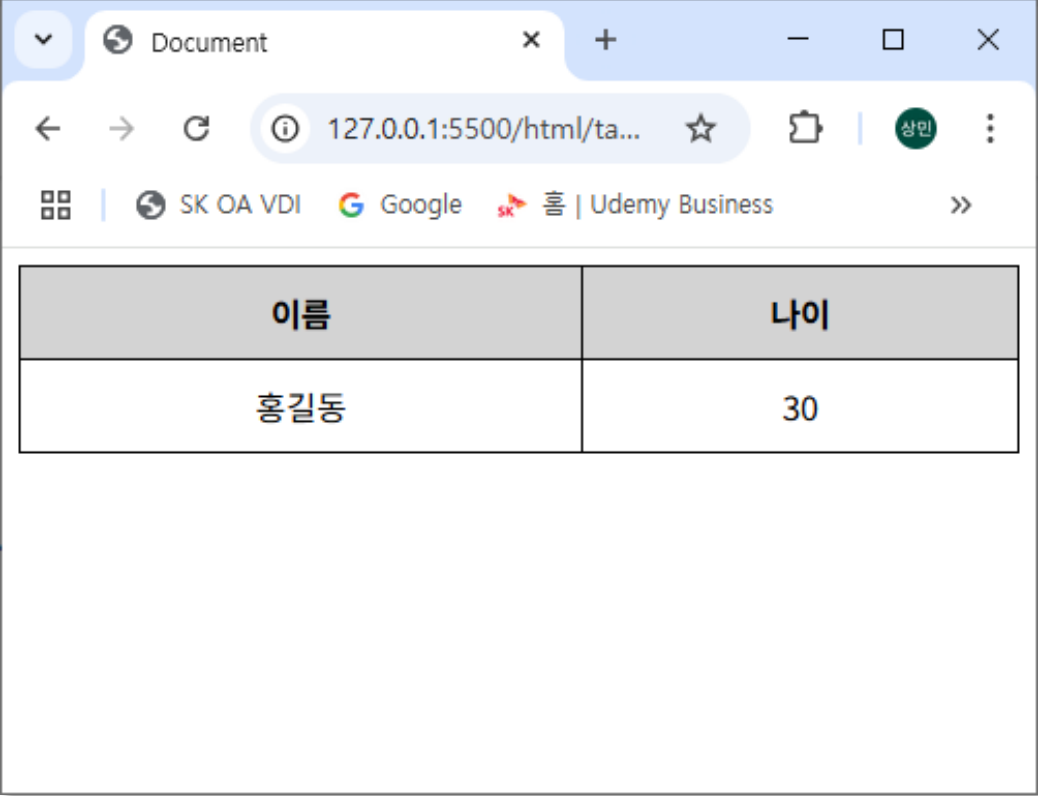
TR elements define table rows

Emil	Tobias	Linus
16	14	10

HTML Tables - <thead><tbody><tfoot>

- 단순 예제나 소규모 표에서는 <tr><th>만으로 사용 가능.
- 접근성, 유지보수, 스타일링, 상호작용 구현 측면에서는 반드시 <thead>, <tbody>, <tfoot>을 구분해서 사용

```
<body>
  <table>
    <thead>
      <tr>
        <th>이름</th>
        <th>나이</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>홍길동</td>
        <td>30</td>
      </tr>
    </tbody>
  </table>
</body>
```



A screenshot of a web browser window showing the rendered HTML table. The browser's address bar displays the URL '127.0.0.1:5500/html/ta...'. The table has two columns: '이름' (Name) and '나이' (Age). The first row contains the values '홍길동' (Hong Gildong) and '30'.

이름	나이
홍길동	30

HTML Tables - Styling

- Use CSS to make your tables look better.

주요 항목	HTML 속성 사용 (지양)	CSS 사용 (권장)
테두리	border	border, border-collapse
셀 간격	cellspacing	border-spacing
셀 안쪽 여백	cellpadding	padding
정렬	align	margin, text-align
배경색	bgcolor	background-color

```
<head>
<style>
table {border-collapse: collapse; width: 100%;}
th,td {text-align: left; padding: 8px;}
tr:nth-child(even) {background-color: #D6EEEE;}
</style>
</head>
<body>
<h2>Zebra Striped Table</h2>
<table>
  <tr>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Points</th>
  </tr>
  ...
```

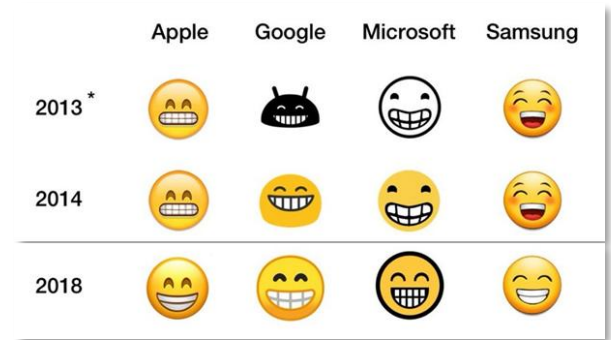
Zebra Striped Table

First Name	Last Name	Points
Peter	Griffin	\$100
Lois	Griffin	\$150
Joe	Swanson	\$300
Cleveland	Brown	\$250

HTML에서 Emoji 활용

- Emojis are **letters (characters)** from the UTF-8 (Unicode) character set
 - 이모지(Emoji): 감정, 사물, 개념 등을 나타내는 그림문자
 - 유니코드(Unicode, 국제 문자 인코딩 표준의 일부)로 관리
 - 일반 텍스트처럼 전송되며 유니코드 코드포인트(U+1F600형식)로 표현
- Emoji 역사

1999년	일본 통신사 NTT 도코모가 세계 최초로 휴대폰용 이모지 도입 (176개)
2010년	유니코드 6.0에 처음으로 722개의 이모지가 공식 포함
2016년	성별(남성/여성/성중립), 직업 관련 이모지 추가
2025년	3,800개 이상의 이모지가 등록되어 있으며, 매년 유니코드에서 새로운 이모지 승인



- Emoji는 문자임으로 그냥 복사하고 붙여넣기하면 다른 문자들처럼 HTML에서 사용될 수 있다.

```
<!DOCTYPE html>
<html>
<meta charset="UTF-8">
<body>
  <h1>Sized Emojis</h1>
  <p style="font-size:48px">&#128512; &#128516; &#128525; &#128151;</p>
</body>
</html>
```

Sized Emojis



[과제] 나의 휴일 일과

- 나의 휴일 일과를 “myHoliday.html” 문서로 작성한다.
 - Folder: html
 - File: myHoliday.html
- 필수 사용 Element
 - <h1>
 - <h2>
 -

 - <p>
 - <mark>



나의 휴일 일과



느긋한 아침

09:00 - 🛌 늦잠 자는 행복

10:00 - 🔍 여유 있게 브런치 즐기기

11:00 - ☕ 커피 한 잔과 음악 감상 🎵



오후 활동

13:00 - 🌳 공원 산책하며 마음 정리

14:30 - 📷 사진 찍으며 일상의 순간 기록

16:00 - 🛍️ 충동구매 대신 창의적인 쇼핑



저녁과 밤

18:00 - 🍲 맛있는 저녁 만들기

20:00 - 🎬 영화 감상 (코미디 추천 😄)

23:00 - 📖 짧은 독서 후 힐링 타임 🛀

📅 휴일 버전 작성일: 2026년 5월

[과제] 나의 소개

- 나의 소개를 “myProfile.html” 문서로 작성한다.
 - Folder: html
 - File: myProfile.html
- 필수 사용 Element
 - 내가 좋아하는 음식
 - 올 해 할 일
 - <dl> 나를 설명하는 단어들
 - CSS는 사용하지 말 것

안녕하세요! 저를 소개합니다

아래는 제가 좋아하는 것들과 올해의 계획, 그리고 저를 표현하는 단어들이입니다.

내가 좋아하는 음식

- 아이스 아메리카노 
- 매콤한 떡볶이 
- 달콤한 초콜릿 케이크 

올해 할 일

1. 일주일에 책 한 권씩 읽기 
2. 꾸준히 운동해서 체력 키우기 
3. 새로운 프로그래밍 언어 배우기 

나를 설명하는 단어들

호기심

새로운 기술이나 흥미로운 트렌드를 보면 직접 경험해보고 배워야 직성이 풀립니다.

꾸준함

매일 작은 계획이라도 세우고 그것을 지키기 위해 노력하는 성실함을 가지고 있습니다.

크리에이티브

단순한 결과물보다는 사용자에게 더 나은 경험과 시각적인 즐거움을 주는 것을 좋아합니다.

[과제] 나의 강의 일정

- 나의 소개를 "myClass.html" 문서로 작성한다.
 - Folder: html
 - File: myClass.html
- 필수 사용 Element
 - <table>
 - <thead> - 시간, 요일
 - <tbody>
 - <td> - 2시간 이상의 강의나, 점심시간은 셀을 합쳐서 표시
 - CSS는 사용하지 말 것

나의 강의 시간표

시간	월요일	화요일	수요일	목요일	금요일
09:00 - 10:00	Vue.js 웹 프레임워크 🖥️	네트워크	웹 시스템 설계 및 분석 🌸	IT 트렌드 특강 💡	자바스크립트 심화 실습 🚀
10:00 - 11:00		보안 기초 🛡️		데이터베이스	
11:00 - 12:00		알고리즘 문제풀이 🗯️		SQL 실습 📄	
12:00 - 13:00	점심시간 🍱️				
13:00 - 14:00	UI/UX 디자인 표준 🎨	리눅스 시스템 관리 🐧	클라우드 AWS 기초 ☁️	자료구조 📊	개인 프로젝트 🚀
14:00 - 15:00		Git & GitHub 버전 관리 🌸	인공지능 머신러닝 기초 🤖		빅데이터 분석 실무 📈
15:00 - 16:00					
16:00 - 17:00					
17:00 - 18:00	취업 창업 특강 📁				주간 회고 📝

[과제] 바로가기

- index.html에 나의 수업, 휴일, 프로필 바로가기를 추가한다.
 - Folder: html
 - File: index.html
- 필수 사용 Element
 - <a>
 - CSS는 사용하지 말 것

환영 인사

스칼라에 오신 것을 환영합니다.

바로가기

- [나의 수업 \(myClass\)](#)
- [나의 휴일 \(myHoliday\)](#)
- [나의 프로필 \(myProfile\)](#)

Full-Stack Engineering - HTML, CSS, JavaScript

3. HTML Form



HTML Forms

- An HTML form is used to collect user input.
- The user input is most often sent to a server for processing.
- The HTML `<form>` element is used to create an HTML form for user input

```
<form>
.
form elements
.
</form>
```

- The `<form>` element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

```
<h2>HTML Forms</h2>
<form action="signUpResult.php" method="get">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
```

HTML Forms

First name:

Last name:

HTML Form Attributes

Attribute	Description
accept-charset	Specifies the character encodings used for form submission
action	Specifies where to send the form-data when a form is submitted
autocomplete	Specifies whether a form should have autocomplete on or off
enctype	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
method	Specifies the HTTP method to use when sending form-data
name	Specifies the name of the form
novalidate	Specifies that the form should not be validated when submitted
rel	Specifies the relationship between a linked resource and the current document
target	Specifies where to display the response that is received after submitting the form

HTML Form Elements

- The HTML <form> element can contain one or more of the following form elements

태그	설명	주요 속성
<input>	사용자로부터 다양한 형태의 입력을 받음	type, name, value, placeholder, required 등
<label>	입력 필드에 대한 텍스트 설명을 제공	for (ID와 연결)
<textarea>	여러 줄의 텍스트 입력	rows, cols, placeholder, name
<select>	드롭다운 목록을 제공	name, multiple, required
<option>	<select> 안에서 선택 가능한 항목	value, selected
<button>	클릭 가능한 버튼 생성	type (submit, reset, button)
<fieldset>	관련 입력 필드를 그룹화	없음
<legend>	<fieldset>에 대한 제목 제공	없음
<datalist>	<input>에 자동완성 옵션 제공	<option>과 함께 사용
<output>	계산 결과 등의 출력용 필드	for, name

HTML Input Types

- Here are the different input **types** you can use in HTML

태그	설명	주요 속성
text	일반 텍스트 입력 필드	<input type="text" placeholder="이름 입력">
password	입력 내용을 숨기는 비밀번호 필드	<input type="password" placeholder="비밀번호 입력">
email	이메일 주소 입력 필드. 이메일 형식 검증 제공	<input type="email" placeholder="이메일 입력">
number	숫자 입력 필드. min, max, step 속성을 사용 가능	<input type="number" min="1" max="100" step="1">
tel	전화번호 입력 필드. 전화번호 형식 검증은 없음	<input type="tel" placeholder="전화번호 입력">
url	URL 입력 필드. URL 형식 검증 제공	<input type="url" placeholder="https://example.com">
color	색상 선택기 제공	<input type="color">
date	날짜 선택 필드 (YYYY-MM-DD 형식)	<input type="date">
checkbox	체크박스. 하나 이상의 선택지를 선택할 수 있음	<input type="checkbox">
radio	라디오 버튼. 하나의 그룹 내에서 하나만 선택 가능	<input type="radio" name="group">
file	파일 선택 필드. 파일 업로드를 처리할 때 사용	<input type="file">
hidden	숨겨진 입력 필드 – 미 노출 데이터를 전송할 때	<input type="hidden" name="token" value="12345">
range	슬라이더로 범위 값을 선택	<input type="range" min="0" max="100" step="10">

HTML Input Attributes

Demo

Attribute	Description
value	Specifies an initial value for an input field
readonly	Specifies that an input field is read-only.
disabled	Specifies that an input field should be disabled.
size	Specifies the visible width, in characters, of an input field.
maxlength	Specifies the maximum number of characters allowed in an input field.
min/max	Specifies the minimum and maximum values for an input field
multiple	Specifies that the user is allowed to enter more than one value in an input field.
pattern	Specifies a regular expression that the input field's value is checked against, when the form is submitted.
placeholder	Specifies a short hint that describes the expected value of an input field
required	Specifies that an input field must be filled out before submitting the form.
step	Specifies the legal number intervals for an input field
autofocus	Specifies that an input field should automatically get focus when the page loads
height/width	Specify the height and width of an <input type="image"> element.
list	Refers to a <datalist> element that contains pre-defined options for an <input> element.
autocomplete	Specifies whether a form or an input field should have autocomplete on or off.

HTML Buttons

- The <button> element defines a clickable button.
- Button Types – **type** attribute
 - type="button" - A normal clickable button (does nothing by default)
 - type="submit" - Submits a form
 - type="reset" - Resets all form fields
- Disabled Buttons – **disabled** attribute
 - Use the disabled attribute to make a button unclickable:

HTML Form Example I

```
<form>
  <fieldset>
    <legend>계정 정보</legend>
    <p>
      <label for="userId">아이디: </label>
      <input type="text" id="userId" name="userId" minlength="4" maxlength="15" required placeholder="4~15자 영문/숫자">
      <small>(필수)</small>
    </p>
    <p>
      <label for="userPw">비밀번호: </label>
      <input type="password" id="userPw" name="userPw" minlength="8" required placeholder="8자 이상 입력">
      <small>(필수)</small>
    </p>
    <p>
      <label for="userEmail">이메일: </label>
      <input type="text" id="userEmail" name="userEmail" placeholder="example">
      @
      <select name="emailDomain">
        <option value="direct">직접 입력</option>
        <option value="naver.com">naver.com</option>
        <option value="gmail.com">gmail.com</option>
        <option value="daum.net">daum.net</option>
      </select>
    </p>
  </fieldset>
</form>
```

계정 정보

아이디: (필수)

비밀번호: (필수)

이메일: @ 직접 입력 ▼

HTML Form Example II

```

<form>
  <fieldset>
    <legend>개인 프로필 정보</legend>
    <p>
      <label for="userName">이름: </label>
      <input type="text" id="userName" name="userName" required>
    </p>
    <p>
      <label for="userBirth">생년월일: </label>
      <input type="date" id="userBirth" name="userBirth">
    </p>
    <p>
      성별:
      <label><input type="radio" name="gender" value="male" checked> 남성</label>
      <label><input type="radio" name="gender" value="female"> 여성</label>
      <label><input type="radio" name="gender" value="none"> 선택안함</label>
    </p>
    <p>
      관심 분야 (중복 선택 가능): <br>
      <label><input type="checkbox" name="interest" value="frontend"> 웹 프론트엔드 (Vue.js/HTML)</label><br>
      <label><input type="checkbox" name="interest" value="uiux"> UI/UX 디자인 표준</label><br>
      <label><input type="checkbox" name="interest" value="backend"> 백엔드 & 데이터베이스</label><br>
      <label><input type="checkbox" name="interest" value="devops"> 클라우드 & 인프라</label>
    </p>
  </fieldset>
</form>

```

개인 프로필 정보

이름:

생년월일:

성별: ☒ 남성 ☐ 여성 ☐ 선택안함

관심 분야 (중복 선택 가능):

- ☐ 웹 프론트엔드 (Vue.js/HTML)
- ☐ UI/UX 디자인 표준
- ☐ 백엔드 & 데이터베이스
- ☐ 클라우드 & 인프라

HTML Form Example III

```
<form>
  <fieldset>
    <legend>자기소개</legend>
    <p>
      <label for="intro">나를 표현하는 한 줄 소개 또는 가입 인사:</label><br>
      <textarea id="intro" name="intro" rows="5" cols="50" placeholder="여기에 내용을 입력하세요 (최대 200자)"></textarea>
    </p>
  </fieldset>
  <br>
  <input type="submit" value="동의하고 회원가입">
  <input type="reset" value="다시 작성">
</form>
```

자기소개

나를 표현하는 한 줄 소개 또는 가입 인사:

여기에 내용을 입력하세요 (최대 200자)

[과제] 회원가입

- 회원가입 페이지를 작성한다.
 - Folder: html
 - File: signUp.html
- 필수 사용 Element
 - <form> - action은 signUpResult.html, method는 get
 - <fieldset> <legend> <label>
 - <input> - placeholder, required 등 속성사용
 - <select> <option> <textarea>
 - <submit> <reset>

회원가입

SKALA-FRONT 회원가입을 환영합니다.

계정 정보

아이디: (필수)

비밀번호: (필수)

이메일: @

개인 프로필 정보

이름:

생년월일:

성별: ☒ 남성 ☐ 여성 ☐ 선택안함

관심 분야 (중복 선택 가능):

☐ 웹 프론트엔드 (Vue.js/HTML)
 ☐ UI/UX 디자인 표준
 ☐ 백엔드 & 데이터베이스
 ☐ 클라우드 & 인프라

가입 경로:

자기소개

나를 표현하는 한 줄 소개 또는 가입 인사:

여기에 내용을 입력하세요 (최대 200자)

동의하고 회원가입

다시 작성

[과제] 회원가입결과

- 회원가입결과 페이지를 작성한다.
 - Folder: html
 - File: signUpResult.html
- 회원가입에서 회원가입 버튼을 클릭하면 회원가입결과 페이지로 이동한다.



회원가입을 진심으로 축하합니다!

SKALA-FRONT 회원이 되신 것을 환영합니다.

안내 사항

- 입력하신 이메일 주소로 가입 확인 메일이 발송되었습니다.
- 보안을 위해 첫 로그인 후 비밀번호를 다시 한번 확인해 주세요.
- 작성하신 프로필 정보는 마이페이지에서 언제든지 수정이 가능합니다.

이동하기

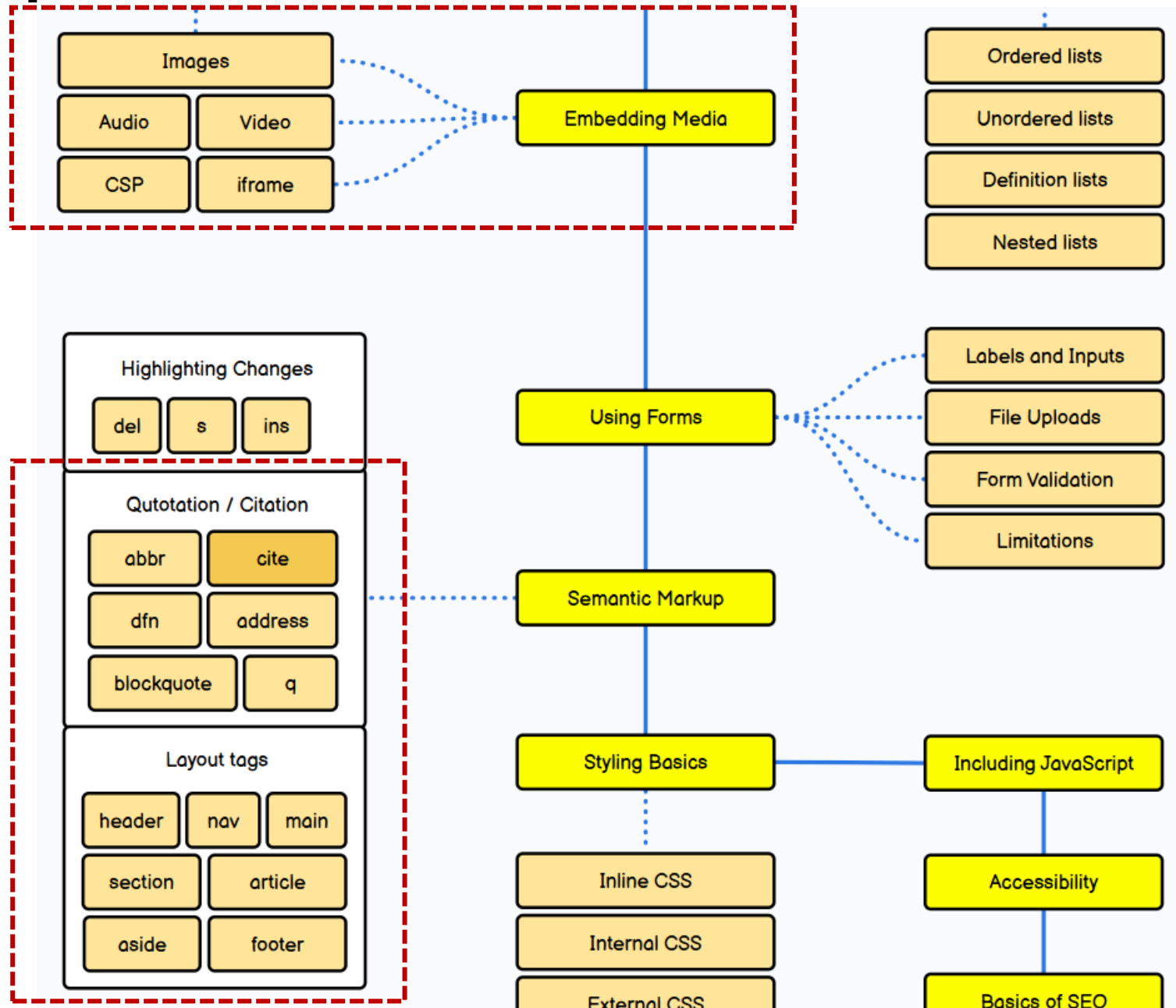
아래 링크를 클릭하면 이전에 작성한 다른 페이지로 이동할 수 있습니다.

-  [나의 프로필 보기](#)
-  [나의 수업 시간표 보기](#)
-  [나의 휴일 계획 보기](#)

Full-Stack Engineering - HTML, CSS, JavaScript

4. HTML 심화

HTML Roadmap



Embedding Media -

- The HTML tag is used to embed an image in a web page.
- The tag has two required attributes
 - src - Specifies the path to the image
 - alt - Specifies an alternate text for the image
- 이미지의 크기를 지정할 때는 width, height 속성으로 지정하는 대신 style 속성을 통해 지정하는 것을 지향한다.

```
<body>  
  <h2>Images on Another Server</h2>  
    
</body>
```

Images on Another Server



Embedding Media - <picture>

- The HTML **<picture>** element allows you to display different pictures for different devices or screen sizes.
 - The <picture> element contains one or more <source> elements.
 - Always specify an element as the last child element of the <picture> element.

```
<picture>  
  <source media="(min-width: 650px)" srcset="img_food.jpg">  
  <source media="(min-width: 465px)" srcset="img_car.jpg">  
    
</picture>
```

The picture Element



The picture Element



The picture Element



Embedding Media - <audio>

- The HTML **<audio>** element is used to play an audio file on a web page.
 - The **<source>** element allows you to specify alternative audio files which the browser may choose from. The browser will use the first recognized format.
 - The text between the **<audio>** and **</audio>** tags will only be displayed in browsers that do not support the **<audio>** element.
- Attributes
 - **controls**: play, pause, volume같은 기본 컨트롤 표시
 - **autoplay**: 자동 재생 (브라우저에서 제한될 수 있음)
 - **loop**: 반복 재생
 - **muted**: 음소거

```
<audio controls autoplay>  
  <source src="horse.ogg" type="audio/ogg">  
  <source src="horse.mp3" type="audio/mpeg">  
  Your browser does not support the audio element.  
</audio>
```



Embedding Media - <video>

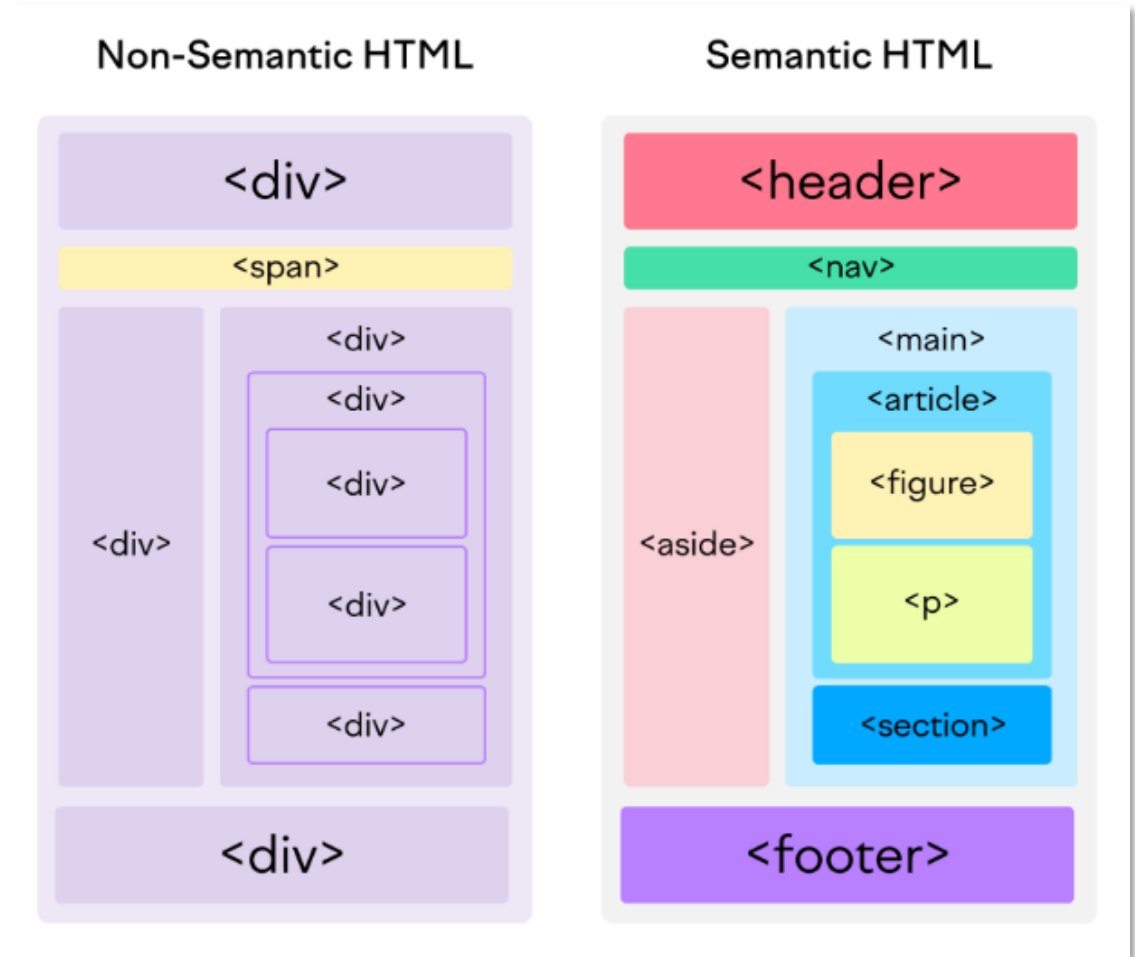
- The HTML **<video>** element is used to show a video on a web page.
 - The **<source>** element allows you to specify alternative video files which the browser may choose from. The browser will use the first recognized format.
 - The text between the **<video>** and **</video>** tags will only be displayed in browsers that do not support the **<video>** element.
- Attributes
 - **controls**: play, pause, volume같은 기본 컨트롤 표시
 - **autoplay**: 자동 재생 (브라우저에서 제한될 수 있음)
 - **poster**: 동영상 로딩 전 표시할 이미지
 - **width, height**: 비디오 플레이어의 크기 지정

```
<video width="320" height="240" autoplay>  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogv" type="video/ogg">  
  Your browser does not support the video tag.  
</video>
```



Semantic Layout Tags - What are Semantic Elements?

- Semantic elements = elements with a meaning.
- A semantic element clearly describes its meaning to both the browser and the developer.
 - Examples of non-semantic elements: `<div>` and `<span>`
 - Examples of semantic elements: `<img>`, `<table>`, and `<article>`
- semantic element의 장점
 - 가독성: 코드의 구조가 명확하여 쉽게 이해
 - 접근성: 화면 낭독기 등의 보조 기술이 구조를 더 잘 이해
 - SEO(Search Engine Optimization)
 - : 검색 엔진이 콘텐츠의 구조와 주요 정보를 효율적으로 파악



Semantic Layout Tags - Semantic Elements in HTML

태그	설명
<header>	헤더 영역을 구분하는데 사용
<nav>	내부의 다른 영역이나 외부로 연결하는 링크 영역을 구분하는데 사용
<main>	문서의 주요 콘텐츠. 페이지에서 핵심적인 내용을 포함
<section>	논리적으로 관련 있는 내용 영역을 구분할 때 사용
<article>	독립적인 영역을 구분할 때 사용
<aside>	주력 내용이나 독립적인 내용으로 보기 어려워서 article 태그나 section 태그로 영역을 구분할 수 없을 때 사용
<footer>	footer 영역을 구분할 때 사용
<figure>	이미지, 도표, 코드 등 삽입된 콘텐츠를 감쌌. <figcaption>과 함께 사용
<figcaption>	<figure>에 대한 설명을 제공

Semantic Text Markup - <blockquote><abbr>

- 인용: The HTML <blockquote> element defines a section that is quoted from another source.

```
<body>
  <p>Here is a quote from WWF's website:</p>
  <blockquote cite="http://www.worldwildlife.org/who/index.html">
    For 60 years, WWF has worked to help people and nature thrive. As the
    world's leading conservation organization, WWF works in nearly 100 countries. At
    every level, we collaborate with people around the world to develop and deliver
    innovative solutions that protect communities, wildlife, and the places in which
    they live.
  </blockquote>
</body>
```

Here is a quote from WWF's website:

For 60 years, WWF has worked to help people and nature thrive. As the world's leading conservation organization, WWF works in nearly 100 countries. At every level, we collaborate with people around the world to develop and deliver innovative solutions that protect communities, wildlife, and the places in which they live.

- 약어: The HTML <abbr> tag defines an abbreviation or an acronym, like "HTML", "CSS", "Mr.", "Dr.", "ASAP", "ATM".

```
</head>
<body>
  <p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
</body>
```

The WHO was founded in 1948.

World Health Organization

HTML Head

- The `<head>` element is a container for metadata (data about data).
- The `<head>` element is placed between the `<html>` tag and the `<body>` tag
- HTML metadata is data about the HTML document. Metadata is not displayed on the page.
- The HTML `<head>` element is a container for the following elements
 - The `<title>` element is required and it defines the title of the document
 - The `<style>` element is used to define style information for a single document
 - The `<link>` tag is most often used to link to external style sheets
 - The `<meta>` element is typically used to specify the character set, page description, keywords, author of the document, and viewport settings
 - The `<script>` element is used to define client-side JavaScripts
 - The `<base>` element specifies the base URL and/or target for all relative URLs in a page

HTML Comments

- Comments are not displayed by the browser, but they can help document your HTML source code.

```
<body>
  <!-- This is a comment -->
  <p>This is a paragraph.</p>
  <!-- Remember to add more information here -->
  <p>This <!-- great text --> is a paragraph.</p>
  <!--
  <p>Look at this cool image:</p>
  
  -->
  <p>This is a paragraph too.</p>
</body>
```

This is a paragraph.

This is a paragraph.

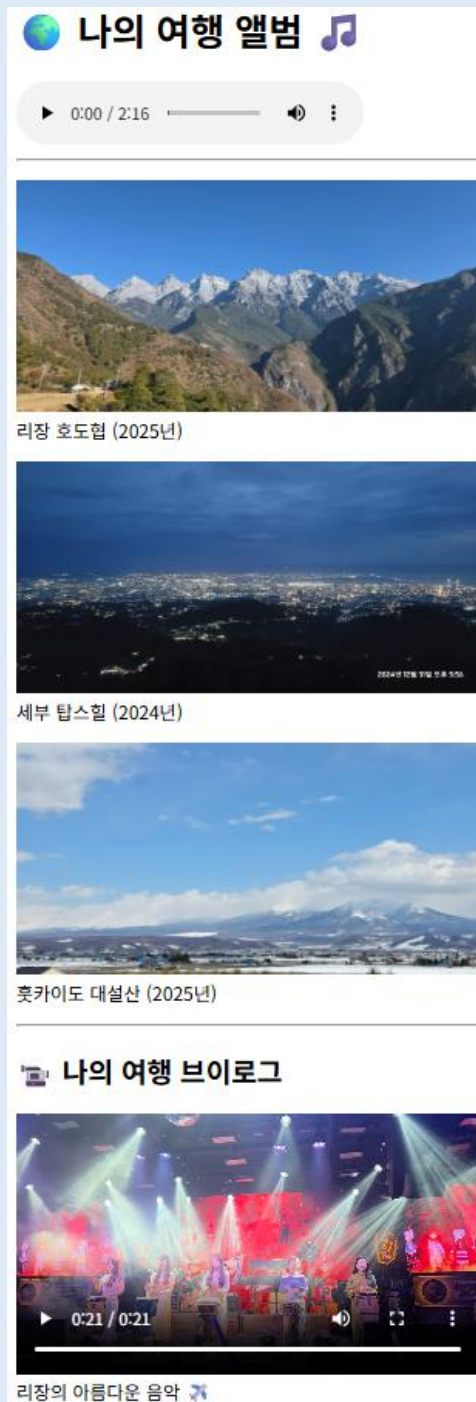
This is a paragraph too.

Accessibility

- Web Accessibility (웹 접근성)
 - Web Accessibility은 시각, 청각, 지체 장애를 가진 분들을 포함하여 "그 어떤 사용자도 웹사이트를 이용하는 데 소외되지 않도록 HTML 문서를 친절하고 표준에 맞게 작성하는 기술을 의미.
 - 화면을 눈으로 보지 못해 소리로 읽어주는 프로그램(스크린 리더)을 쓰는 시각 장애인 등 모두에게 동등한 사용자 경험을 제공하는 것이 목적.
- 웹 접근성을 높이는 HTML 작성 규칙
 - 시맨틱 태그를 쓰면 스크린 리더가 화면의 구조를 컴퓨터 공학적으로 정확하게 파악할 수 있다.
 - 문서의 구조와 각 섹션 간의 관계를 명확히 보여주기 위해 Heading Elements를 사용하는 것이 중요하다.
 - 태그를 쓸 때는 눈이 보이지 않는 사용자를 위해 "이 이미지가 어떤 내용인지" 말로 설명해 주는 alt 속성을 적어야 한다.
 - 회원가입 폼 등을 만들 때, 글자(텍스트)와 실제 입력창(input)을 눈으로만 묶어두면 안된다. 두 요소를 코드로 단단히 연결해 주어야 스크린 리더가 입력창에 포커스가 갔을 때 "어떤 내용을 적는 칸"인지 정확히 읽어준다.

[과제] 나의 여행지

- 나의 여행지를 소개하는 페이지를 작성한다.
 - Folder: html
 - File: myTrip.html
 - Media Resource Folder: media
- 필수 사용 Element
 - `<audio><source>`
 - `<img>`
 - `<video><source>`



[과제] 포털 사이트형 메인 Hub 만들기

- 기존에 만든 myClass, myHoliday, myProfile, myTrip, signUp 을 한 곳에 모아 볼수 있는 개인 메인 포털 작성
 - 수정 File: index.html
- 필수 사용 Element
 - <nav> - 다른 파일로 이동하는 메뉴 링크를 넣기
 - <main> - 본문 영역을 선언하고 콘텐츠 넣기
 - <aside> - 사이드바 넣고 부가 정보 넣기

강병호의 유니버스 🚀

저의 프로필, 학습 내용, 여행 기록을 모아둔 Hub 페이지입니다.

📁 바로가기 메뉴

- [소개 \(Profile\)](#)
- [수업 시간표 \(Class\)](#)
- [휴일 계획 \(Holiday\)](#)
- [여행 앨범 \(Album\)](#)
- [👤 회원가입 \(Sign Up\)](#)

📌 오늘의 주요 소식

최근 공지 및 업데이트

[업데이트] 여행 앨범에 동영상 추가 완료!

훗카이도 대설산 단락 아래에 멋진 브이로그 영상을 추가했습니다. 이제 소리와 영상으로 여행을 추억할 수 있습니다.

작성일: 2026년 6월 11일

🖼️ 이달의 베스트 컷



중국 리장 호도협이 웅장한 협곡 모습 (2025년 촬영)

🌤️ 실시간 정보

현재 위치: 광주광역시 광산구 📍

추천 학습 사이트: [W3Schools 온라인 명세서](#)

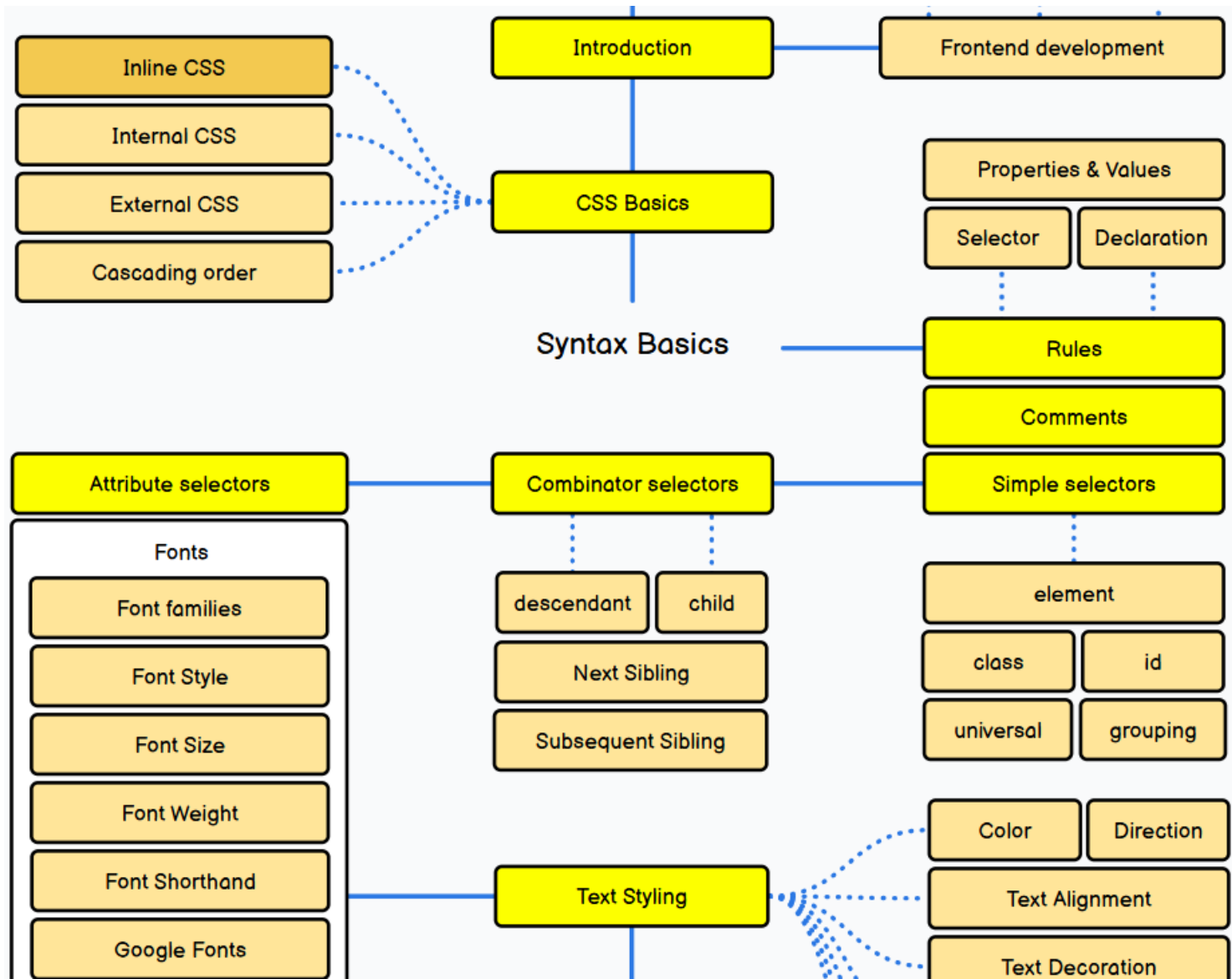
이메일 문의: medit74@sk.com

© 2026 병호 포트폴리오. 모든 태그는 웹 표준 시맨틱 태그로 작성되었습니다.

Full-Stack Engineering - HTML, CSS, JavaScript

5. CSS 기초

CSS Roadmap

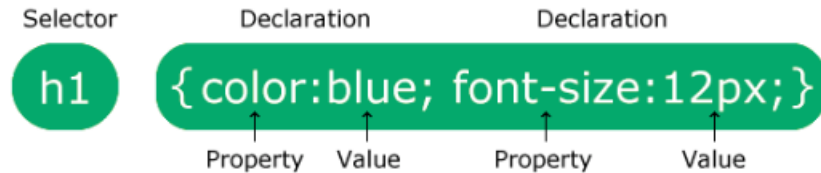


CSS 개요

- Cascading Style Sheets (CSS)는 웹 페이지의 스타일을 정의하는 스타일시트 언어
 - CSS stands for Cascading Style Sheets
 - CSS describes **how HTML elements are to be displayed** on screen, paper, or in other media
- CSS를 사용하면 색상, 글꼴, 텍스트 크기, element 간격, element 의 위치 및 레이아웃, 사용할 배경 이미지 또는 배경 색, 다양한 장치 및 화면 크기에 따른 표시 방식 등을 제어할 수 있습니다!
- CSS Demo - One HTML Page - Multiple Styles!
 - https://www.w3schools.com/css/css_intro.asp

CSS Syntax

- A CSS rule consists of a **selector** and a **declaration block**



- The **selector** points to the HTML element you want to style.
- The **declaration block** contains **one or more declarations separated by semicolons**.
- Each declaration includes a CSS **property** name and a **value**, separated by a colon.
- Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

```
p {  
  color: red;  
  text-align: center;  
}
```

Hello World!

These paragraphs are styled with CSS.

CSS How To - Inline CSS, Internal CSS

- CSS를 사용하는 3가지 방법

- Inline CSS - by using the **style** attribute inside HTML elements

```
<h1 style="color:blue;text-align:center;">This is a heading</h1>  
<p style="color:red;">This is a paragraph.</p>
```

- Internal CSS - by using a **<style>** element in the **<head>** section

```
<html>  
<head>  
  <style>  
    body {background-color: linen;}  
    h1 {color: maroon; margin-left: 40px;}  
  </style>  
</head>  
<body>  
  <h1>This is a heading</h1>  
  <p>This is a paragraph.</p>  
</body>  
</html>
```


CSS How To - External CSS

- CSS를 사용하는 3가지 방법
 - External CSS - by using a <link> element to link to an external CSS file

```
<html>
<head>
  <link rel="stylesheet" href="mystyle.css">
</head>
<body>
  <h1>This is a heading</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

```
body {background-color: lightblue;}
h1 {color: navy; margin-left: 20px;}
```

CSS Selectors - Simple Selectors

- CSS **selectors** are used to **"find" (or select) the HTML elements** you want to style.

- The CSS element Selector

```
p {  
  color: red;  
  text-align: center;  
}
```

- The CSS id Selector

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

- The CSS class Selector

```
.center {  
  text-align: center;  
  color: red;  
}
```

CSS Selectors - Grouping Selectors

- The universal selector (*) selects all HTML elements on the page.

```
* {  
  text-align: center;  
  color: blue;  
}
```

- The grouping selector selects all the HTML elements with the same style definitions.

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}
```

CSS Selectors - Attribute Selectors

- CSS **attribute selectors** are used to select and style **HTML elements with a specific attribute or attribute value, or both.**
 - [attribute] - Select elements with the specified attribute
 - [attribute="value"] - Select elements with a specific attribute and an exact value
 - [attribute~="value"] - Select elements with an attribute value containing a specific word
 - [attribute|="value"] - Select elements with the specific attribute, whose value can be exactly the value, or start with the value followed by a hyphen (-)
 - [attribute^="value"] - Select elements whose attribute value starts with a specific value
 - [attribute\$="value"] - Select elements whose attribute value ends with a specific value
 - [attribute*="value"] - Select elements whose attribute value contains a specific value

```
input[type="text"] {  
  width: 150px;  
  padding: 6px;  
  margin-bottom: 10px;  
  background-color: pink;  
}  
  
input[type="button"] {  
  width: 100px;  
  padding: 6px;  
  background-color: lightgreen;  
}
```

CSS Selectors - Combinator

- A CSS selector can contain more than one selector. Between the selectors, we can include a combinator to create a more specific selection.
 - **Descendant** combinator (space): all elements that are **descendants (children, grandchildren, etc.)** of a specified element.
 - **Child** combinator (>) : all elements that are **direct children** of a specified element
 - **Next sibling** combinator (+) : an element that is **directly after a specific element**
 - **Subsequent-sibling** combinator (~) : all elements that are **next siblings** of a specified element

CSS Selectors - Summary

- CSS **selectors** are used to **"find" (or select) the HTML elements** you want to style.

선택자 종류	설명	예시
유니버설 선택자	모든 요소를 선택	<code>* { color: blue; }</code>
요소 선택자	특정 HTML 태그를 선택	<code>p { font-size: 16px; }</code>
아이디 선택자	특정 아이디를 가진 요소를 선택	<code>#header { background-color: grey; }</code>
클래스 선택자	특정 클래스를 가진 요소를 선택	<code>.menu { display: flex; }</code>
그룹화 선택자	여러 선택자를 그룹화하여 한 번에 스타일 적용	<code>h1, h2, h3 { color: green; }</code>
자식 선택자	직접 자식 요소를 선택	<code>div > p { margin-top: 10px; }</code>
후손 선택자	자식뿐 아니라 모든 후손 요소를 선택	<code>div p { color: red; }</code>
인접 형제 선택자	특정 요소 바로 다음에 위치한 형제 요소를 선택	<code>h1 + p { font-weight: bold; }</code>
일반 형제 선택자	특정 요소 뒤에 있는 모든 형제 요소를 선택	<code>h1 ~ p { color: blue; }</code>
속성 선택자	특정 속성을 가진 요소를 선택	<code>a[href] { color: red; }</code>
속성 값 선택자	특정 속성 값이 일치하는 요소를 선택	<code>a[href="https://example.com"] { font-weight: bold; }</code>

CSS Selectors - Pseudo-classes

- A CSS pseudo-class(의사클래스 or 가상클래스) is a keyword that can be added to a selector, to define a style **for a special state of an element**.
- Some common use for pseudo-classes
 - Style an element when a user moves the mouse over it
 - Style visited and unvisited links differently
 - Style an element when it gets focus
 - Style valid/invalid/required/optional form elements
 - Style an element that is the first child of its parent
- Syntax: 선택자 뒤에 콜론(:)을 붙여 사용

```
selector:pseudo-class-name {  
    CSS properties  
}
```

- Pseudo-class Category
 - Interactive Pseudo-classes
 - Structural Pseudo-classes

CSS Selectors - Interactive Pseudo-classes

- Interactive pseudo-classes apply styles **based on user interaction with elements**.
 - **:link** - 방문한 적이 없는 링크 상태
 - **:visited** - 사용자가 이미 방문한(클릭한 적이 있는) 링크 상태
 - **:hover** - 요소 위에 마우스 커서가 올라가 있는 상태
 - **:active** - 요소를 마우스로 클릭하고 있는(누르고 있는) 상태
 - **:focus** - 키보드 탭(Tab) 키나 마우스 클릭으로 인해 해당 요소(주로 input, button 등)가 선택되어 초점이 맞춰진 상태

CSS Selectors - Structural Pseudo-classes

- Structural pseudo-classes select elements **based on their position in the document tree**.
 - **:first-child** - 부모 요소의 자식들 중 첫 번째에 위치하는 요소
 - **:last-child** - 부모 요소의 자식들 중 마지막에 위치하는 요소
 - **:nth-child(n)** - 부모 요소의 자식들 중 n번째에 위치하는 요소 (예: :nth-child(2)는 2번째 자식, :nth-child(2n)은 짝수 번째 자식들)
 - **:nth-last-child(n)** - Matches the nth child from the end
 - **:only-child** - Matches an element that is the only child
 - **:first-of-type** - Matches the first element of its type
 - **:last-of-type** - Matches the last element of its type

CSS Selectors - Pseudo-elements

- A CSS pseudo-element(의사요소 or 가상요소) is a keyword that can be added to a selector, to style a specific part of an element.
- Some common use for pseudo-elements
 - Style the first letter or first line, of an element
 - Insert content before or after an element
 - Style the markers of list items
 - Style the user-selected portion of an element
 - Style the viewbox behind a dialog box
- Syntax: 선택자 뒤에 더블콜론(::)을 붙여 사용

```
selector::pseudo-element-name {  
  CSS properties  
}
```

- Pseudo-element Category
 - Text Pseudo-elements
 - Content Pseudo-elements

CSS Selectors - Text Pseudo-elements

- Text pseudo-elements style **specific parts of text content**.
 - **::first-line** - 텍스트 블록의 첫 번째 줄만 선택. Browser 창의 너비에 따라 줄바꿈이 바뀌면 자동으로 첫 줄의 범위가 갱신된다.
 - **::first-letter** - 텍스트 블록의 첫 번째 글자만 선택
 - **::selection** - 사용자가 마우스로 드래그하여 선택한 텍스트 영역을 선택

CSS Selectors - Content Pseudo-elements

- Content pseudo-elements insert or style generated content. Use the content property to specify the content to insert.
 - **::before** - 선택한 요소의 가장 앞쪽(내부 시작점)에 가상 콘텐츠를 생성
 - **::after** - 선택한 요소의 가장 뒤쪽(내부 끝점)에 가상 콘텐츠를 생성

CSS Selectors - Selector의 조합 예

- Class가 notice인 p element

```
p.notice {font-size: 18px;}
```

- Id가 header인 div element

```
div#header {background: gray;}
```

- Class가 btn이면서 primary인 button element

```
button.btn.primary {background-color: blue;}
```

- Type attribute가 "text"이고 required attribute가 있는 input element

```
input[type="text"][required] {border: 1px solid red;}
```

- ul element 내 첫 번째 li element에 마우스를 올렸을 때

```
ul li:first-child:hover {color: red;}
```

- 마우스를 올린 모든 element

```
*:hover {background-color: yellow;}
```

- new class를 가진 section의 자식 중 featured class를 가진 article의 후손 중 title class를 가진 h2 element 바로 다음에 오는 summary class를 가진 p element

```
section.news > article.featured h2.title + p.summary {font-style: italic;}
```

CSS Specificity

- CSS specificity(우선순위 or 명시도) is an algorithm that determines which style declaration is ultimately applied to an element.
- If two or more CSS rules point to the same element, the declaration with the highest specificity will "win", and that style will be applied to the HTML element.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>
  <p class="test">Hello World!</p>
</body>
</html>
```

```
<html>
<head>
  <style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>
  <p id="demo" class="test">Hello World!</p>
</body>
</html>
```

```
<html>
<head>
  <style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>
  <p id="demo" class="test" style="color:
pink;">Hello World!</p>
</body>
</html>
```

CSS Specificity - Hierarchy

- Each type of CSS selector has a position in the specificity hierarchy, and the selector types carry different "**weights**".

Selector	Example	Description	Weight
Inline styles	<h1 style="color: pink;">	1순위	1-0-0-0
Id selectors	#navbar	2순위	0-1-0-0
Classes, attribute selectors and pseudo-classes	.test, [type="text"], :hover	3순위	0-0-1-0
Elements and pseudo-elements	h1, ::before, ::after	4순위	0-0-0-1
Universal selector and :where()	*, where()	우선순위 없음	0-0-0-0

- Specificity 작동 원칙
 - 자릿수가 높으면 무조건 승리 - [0, 0, 1, 0](클래스 1개)이 [0, 0, 0, 15](태그 15개)보다 우선순위가 높다.
 - 점수가 같으면 '가장 아래에 있는 코드'가 승리 - 두 선택자의 합산 점수가 완전히 동일하면, CSS 파일에서 가장 나중에(아래에) 작성된 규칙이 이전 규칙을 덮어쓴다. (Cascading 원칙)
 - 전체 선택자(*)는 점수가 없다 - 모든 요소를 선택하는 * 기호는 명시도 점수가 0점이므로, 항상 우선순위에서 밀린다.

CSS Specificity - !Important

■ CSS !Important Rule

- The !important rule is used to give the value of a specific property **the highest priority**.
- The !important rule will **override ALL previous styling rules** for that specific property on that element!
- The !important keyword is added to the end of a CSS declaration, before the semicolon.

```
selector {  
  property: value !important;  
}
```

■ CSS !Important Rule 유의점 ➔ 최소사용

- 디버깅 어려움: 여러 곳에서 사용하면, CSS의 우선순위를 추적하기 어려워져 디버깅이 복잡해 짐.
- 유지 보수 어려움: 스타일 우선순위를 강제로 조정하므로, 코드 유지보수 및 확장성 측면에서 문제 가능
- 가독성 감소: !important를 남용하면 코드의 가독성이 떨어지고, 규칙 간의 관계를 명확하게 파악하기 어려움

CSS Code Challenge

- 다음 실습을 진행하시오
 - 1) CSS How To - https://www.w3schools.com/css/css_challenges_howto.asp
 - 2) CSS Selectors - https://www.w3schools.com/css/css_challenges_selectors.asp
 - 3) CSS Pseudo-classes - https://www.w3schools.com/css/css_challenges_pseudo_classes.asp

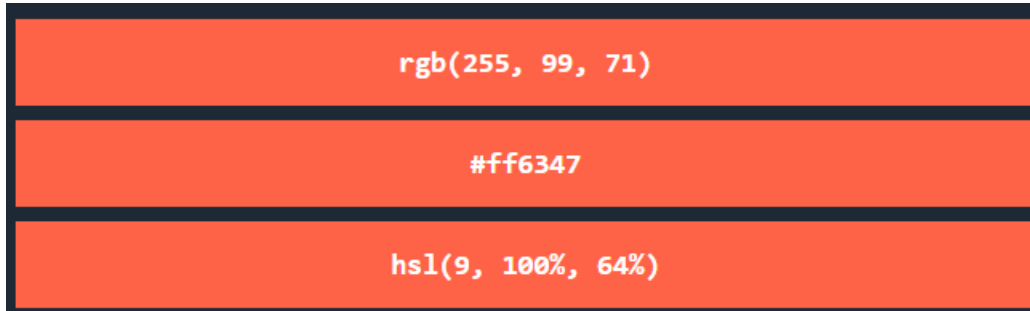
CSS Colors

- In CSS, colors are specified by using a predefined color name, or with a RGB, HEX, HSL, RGBA, HSLA value.

- Predefined color names



- RGB(Red, Green, Blue), HEX(Hexadecimal Color), HSL(Hue, Saturation, Lightness) values



CSS Units

- CSS units are used to define **the length and size of several properties**.
- Several CSS properties take "**length**" values, such as font-size, width, margin, padding, border, etc.
- Absolute Units
 - px (pixels) - The most used absolute unit for screens
- Relative Units
 - em (엠) - 부모 요소의 폰트 크기를 기준(1em = 부모의 font-size)으로 변환다
 - rem (렘 or 알엠) - 부모와 관계없이 오직 최상위 Root HTML 요소의 폰트 크기만 기준으로 작동 (Root em)
 - % - 부모 요소의 해당 속성 크기를 \$100%로 두고 상대적인 비율을 계산
 - vw (Viewport Width) - 현재 웹 브라우저 화면 전체 너비의 1%를 나타낸다. (100vw = 화면 전체 너비)
 - vh (Viewport Height) - 현재 웹 브라우저 화면 전체 높이의 1%를 나타낸다. (100vh = 화면 전체 높이)

```
body {font-size: 16px; /* Base font size */}
h1 {font-size: 2.5em; /* 2.5 * 16 = 40px */}
h2 {font-size: 1.875em; /* 1.875 * 16 = 30px */}
p {font-size: 1rem; /* 1 * 16 = 16px */}
```

* Viewport = the browser window size. 1vw = 1% of the current width of the browser's viewport. So, if the viewport is 500px wide, 1vw is 5px.

CSS Fonts

▪ Font Family – font-family

```
.p1 {font-family: "Times New Roman", Times, serif;}
```

- If the browser does not support the first font, it tries the next font. The font names should be separated with a comma.

▪ Font Style – font-style, font-weight, font-variant

```
p.italic {font-style: italic;}  
p.thick {font-weight: bold;}  
p.small {font-variant: small-caps;}
```

▪ Font Size – font-size

```
body {font-size: 16px; /* Base font size */}  
h1 {font-size: 2.5em; /* 2.5 * 16 = 40px */}  
h2 {font-size: 1.875rem; /* 1.875 * 16 = 30px */}  
p {font-size: 5vw;}
```

CSS Fonts - Google Fonts

- Google Fonts are free to use, and have more than 1000 fonts to choose from
- How to use

```
<head>  
  <link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Sofia">  
  <style>  
    body {  
      font-family: "Sofia", sans-serif;  
    }  
  </style>  
</head>
```

- Use multiple google fonts

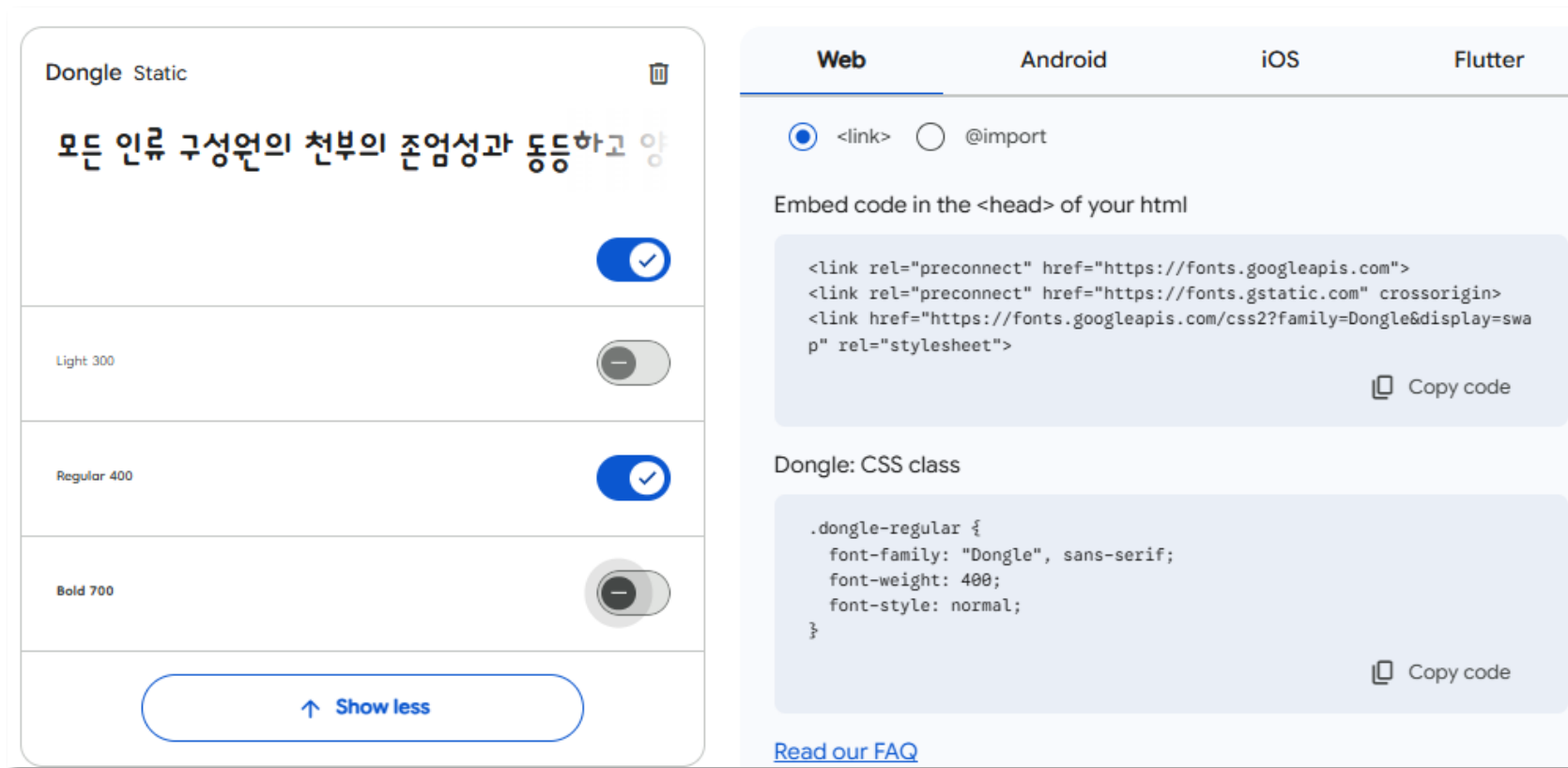
```
<head>  
  <link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Audiowide|Sofia|Trirong">  
  <style>  
    h1.a {font-family: "Audiowide", sans-serif;}  
    h1.b {font-family: "Sofia", sans-serif;}  
    h1.c {font-family: "Trirong", serif;}  
  </style>  
</head>
```

- Enabling font effects – *effect=effectname*

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Sofia&effect=fire">
```

CSS Fonts - Google 한국어폰트

- Google Fonts 공식 사이트에 접속 (<https://fonts.google.com/>)
- Language 필터에서 Korean (한국어) 선택 후 상세 페이지로 들어간다.
- 상세 페이지에서 Get font 이 후 장바구니에서 Get embed code 를 통해 확인한다.



CSS Text Styling

- Text Color – **color**

```
body {color: blue;}
h1 {color: #00ff00;}
h2 {color: rgb(255,0,0);}
```

- Text Alignment – **text-align**

```
h1 {text-align: center;}
h2 {text-align: left;}
h3 {text-align: right;}
```

- Text Transformation – **text-transform**

```
p.uppercase {text-transform: uppercase;}
p.lowercase {text-transform: lowercase;}
p.capitalize {text-transform: capitalize;}
```

- Text Decoration – **text-decoration-line, text-decoration-color, text-decoration-style, text-decoration-thickness**

```
h1 {text-decoration-line: overline; text-decoration-color: red; text-decoration-style: double; text-decoration-thickness: 15px;}
h3 {text-decoration-line: underline; text-decoration-color: green; text-decoration-style: dotted; text-decoration-thickness: 25%;}
```

- Text Spacing – **text-indent, letter-spacing, line-height, word-spacing, white-space**

```
p {text-indent: 50px; letter-spacing: 1px; line-height: 1.5; word-spacing: 5px; white-space: nowrap;}
```

- Text Shadow – **text-shadow**

```
h1 {text-shadow: 0 0 3px #ff0000, 0 0 5px #0000ff;}
```

CSS Backgrounds

- Background Color – **background-color, opacity**

```
body {background-color: lightblue; opacity: 0.3;}
```

- Background Image – **background-image**

```
body {background-image: url("paper.gif");}
```

- Background Image Repeat – **background-repeat, background-position**

```
body {background-image: url("img_tree.png"); background-repeat: no-repeat; background-position: right top;}
```

- Background Attachment – **background-attachment**

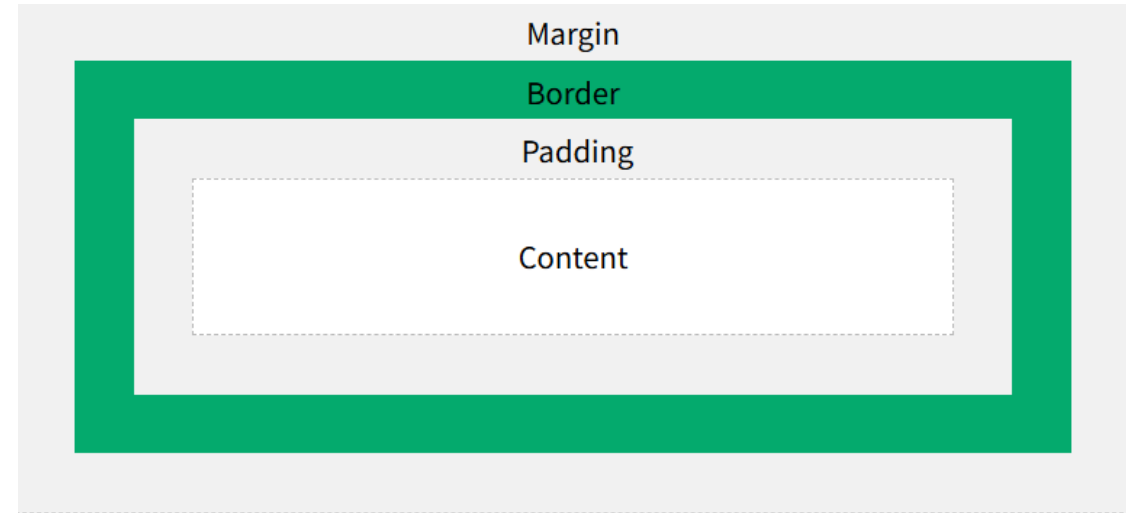
```
body {background-image: url("img_tree.png"); background-repeat: no-repeat; background-position: right top;  
background-attachment: scroll;}
```


CSS Box Model

Demo

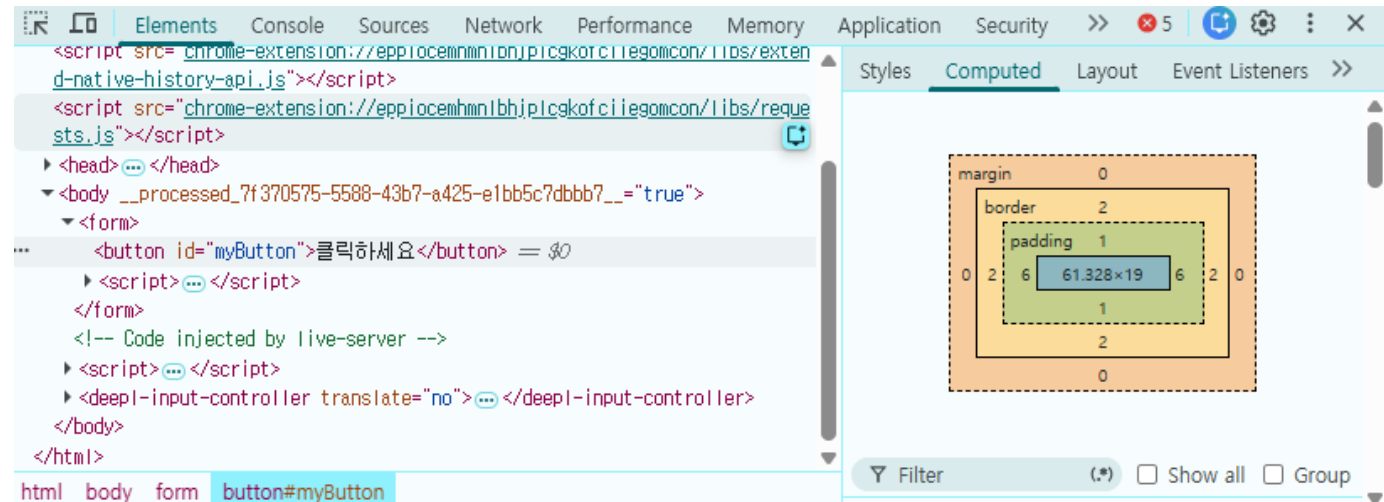
- The CSS box model is essentially a box that wraps around every HTML element.

- Margin: 요소와 다른 요소 사이의 간격을 설정
- Padding: 콘텐츠와 경계 사이의 간격을 설정
- Border: 요소의 경계를 설정
- Content: 요소의 실제 내용이 들어가는 영역



- 브라우저 개발자도구에서 확인

- Elements > Computed



CSS Box Model - Width / Height

- width/height – width, height

```
div {height: 200px; width: 50%;}
```

- The height and width do not include padding, borders, or margins.
- 'auto' is default. The browser calculates the height and width.

- 너비/높이의 최소/최대 속성 – min-width, max-width, min-height, max-height

```
.div1 {max-width: 500px;}  
.div2 {width: 500px;}
```

- 브라우저의 창의 너비가 width보다 작으면 가로 스크롤바가 생기지만, max-width를 사용하면 스크롤바가 없이 너비가 맞춰서 줄어든다.

CSS Box Model - Borders

▪ Border Style – **border-style**

```
p.dotted {border-style: dotted;}
```

- 허용 값 : dotted, dashed, solid, double, groove, ridge, inset, outset, none, hidden

▪ Border Width – **border-width**

```
p.one {border-style: solid; border-width: 5px;}  
p.two {border-style: solid; border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */}  
p.three {border-style: solid; border-width: 25px 10px 4px 35px; /* 25px top, 10px right, 4px bottom and 35px left */}
```

▪ Border Color – **border-color**

```
p.one {border-style: solid; border-color: red;}  
p.two {border-style: solid; border-color: red green blue yellow; /* red top, green right, blue bottom and yellow left */}
```

▪ Shorthand Border Property – **border, border-left, border-bottom, border-top, border-right**

```
p {border: 5px solid red;}  
p {border-left: 6px solid red;}
```

- Border-width, border-style, border-color를 한 번에 표시한다.

▪ Rounded Borders – **border-radius**

```
p {border: 2px solid red; border-radius: 5px;}
```

CSS Box Model - Margins

- Individual Margin – **margin-top, margin-right, margin-bottom, margin-left**

```
p {margin-top: 100px; margin-bottom: 100px; margin-right: 150px; margin-left: 80px;}
```

- Shorthand Margin Property – **margin**

```
p {margin: 25px 50px 75px 100px;}
```

- 순서대로 top, right, bottom, left의 값을 가진다.

- Margin Collapse (마진병합) 현상

- 여러 개의 세로 마진이 서로 만나면 각각 더해지는 것이 아니라, 그중 가장 큰 마진 값 하나로 합쳐지는 현상
- 좌우 마진은 병합되지 않는다.

```
h1 {margin-bottom: 50px;}  
h2 {margin-top: 20px;}
```

CSS Box Model - Padding

- Individual Padding – padding-top, padding-right, padding-bottom, padding-left

```
div {padding-top: 50px; padding-right: 30px; padding-bottom: 50px; padding-left: 80px;}
```

- Shorthand Padding Property – padding

```
div {padding: 25px 50px 75px 100px;}
```

- 순서대로 top, right, bottom, left의 값을 가진다.

- Element의 width와 padding

- Element의 width 속성은 element의 content 영역의 너비를 의미하므로, padding과 width가 같이 추가되어 있다면 실제 element의 총 너비는 두 값의 합과 같다.

```
div {width: 300px; padding: 25px; /* 총 너비는 300+25x2=350px */}
```

CSS Position

- **position** 속성은 element의 배치 유형을 지정하며 다음 값 중 하나를 가질 수 있다.

- static – 기본값. element는 일반적인 문서 흐름에 따라 배치된다. (top, left 등 속성 값이 적용되지 않음)

```
div.static {position: static; border: 3px solid #73AD21;}
```

- relative – element는 문서 흐름 내의 일반적인 위치에 상대적으로 배치된다. (원래 공간은 차지함)

```
div.relative {position: relative; left: 30px; border: 3px solid #73AD21;}
```

- fixed – element는 뷰포트에 상대적으로 배치됩니다. (Scroll해도 위치가 고정된다.)

```
div.fixed {position: fixed; bottom: 0; right: 0; width: 300px; border: 3px solid #73AD21;}
```

- absolute – element는 가장 가까운 조상 요소를 기준으로 배치된다. (문서 흐름에서 제거되고 공간은 차지하지 않는다.)

```
div.absolute {position: absolute; top: 80px; right: 0; width: 200px; height: 100px; border: 3px solid red;}
```

- sticky - element는 스크롤 위치에 따라 relative와 fixed 사이를 전환한다. (조건부 고정으로 주로 헤더, 메뉴 등에서 사용된다.)

```
div.sticky {position: sticky; top: 0; padding: 5px; background-color: #cae8ca; border: 2px solid #4CAF50;}
```

CSS Z-index

- **z-index** 속성은 동일 위치에 중첩(overlap)되어 있는 element의 중첩 순서를 지정한다.
 - An element can have a positive or negative stack order (z-index)

```
<head>
  <style>
    img {
      position: absolute;
      left: 0px;
      top: 0px;
      z-index: -1;
    }
  </style>
</head>
<body>
  <h1>This is a heading</h1>
  
  <p>Because the image has a z-index of -1, it will be placed behind the text.</p>
</body>
</html>
```



CSS Inheritance

- 상속(Inheritance): 부모 요소에 지정된 스타일 중 일부가 자식 요소에 자동으로 적용되는 것
 - HTML 구조가 계층적(트리 구조)이기 때문에 CSS도 이런 계층적 스타일 적용이 가능
- 상속되는 속성

텍스트	color, text-align, text-indent, letter-spacing, word-spacing, line-height, visibility, white-space
폰트	font, font-family, font-size, font-style, font-variant, font-weight
리스트	list-style, list-style-type, list-style-position, list-style-image
테이블	border-collapse, border-spacing, caption-side, empty-cells

- 상속되지 않는 속성

레이아웃	margin, padding, border, width, height, display, position
배경	background-color, background-image
박스 정렬	flex, grid, justify-content, align-items
기타	box-shadow, z-index, overflow

CSS Inheritance - inherit 키워드

- inherit 키워드를 사용하면 상속되지 않은 속성도 자식 요소로 강제로 상속이 가능하다.

```
<style>
p {
  border: 1px solid red;
}
strong {
  border: inherit;
}
</style>

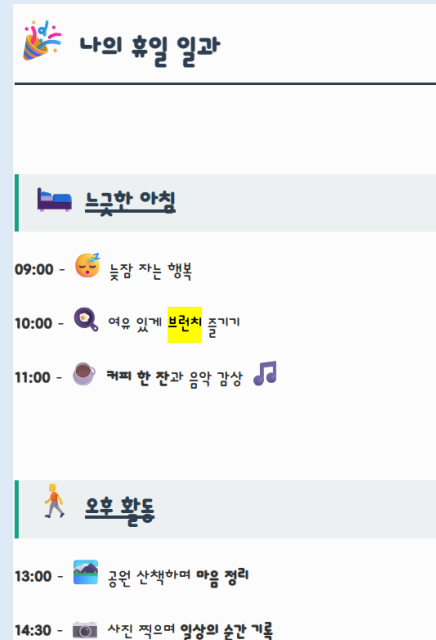
<body>
<p>This is a paragraph with some <strong>strong</strong> text.</p>
</body>
```

CSS Code Challenge

- 다음 실습을 진행하시오
 - 1) CSS Colors - https://www.w3schools.com/css/css_challenges_colors.asp
 - 2) CSS Fonts - https://www.w3schools.com/css/css_challenges_font.asp
 - 3) CSS Text - https://www.w3schools.com/css/css_challenges_text.asp
 - 4) CSS Backgrounds - https://www.w3schools.com/css/css_challenges_background.asp
 - 5) CSS Position - https://www.w3schools.com/css/css_challenges_position.asp
 - 6) CSS Inheritance - https://www.w3schools.com/css/css_challenges_inheritance.asp

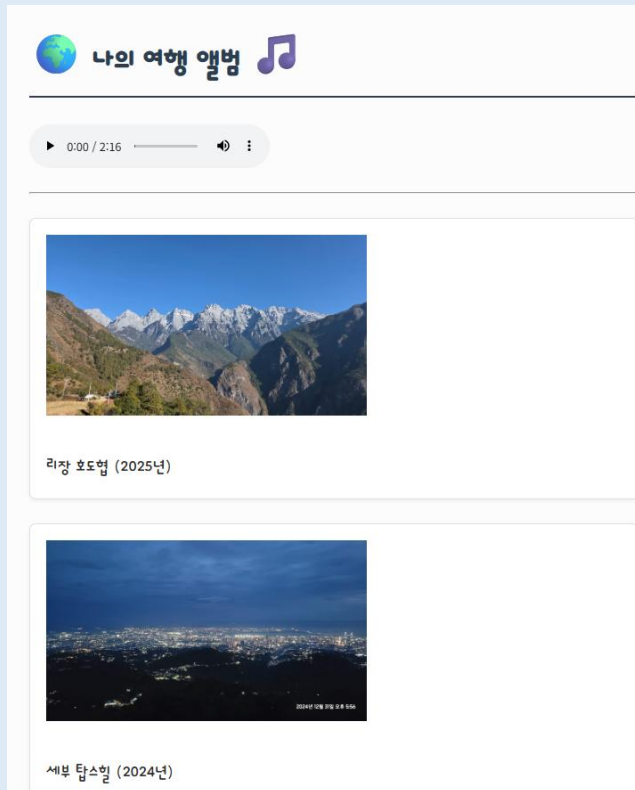
[과제] 미션1 - 전체 테마 및 텍스트 Styling

- 모든 페이지의 기본 바탕과 글꼴을 아름답게 다듬는 미션을 통해 태그 선택자와 클래스 선택자의 활용법을 익힌다.
 - 별도 CSS 파일 생성 /css/style.css
 - 전체 글꼴: 구글 폰트에서 마음에 드는 것을 선택하고 body 태그 선택자를 이용해 전체 폰트를 변경한다.
 - 전체 줄간격, Color, 배경색을 적용한다.
 - 제목 강조: h1, h2 태그에 색, 크기, Padding, Border 등을 넣어 꾸민다.
 - 링크 스타일: 링크 컬러나, Decoration을 지정한다.
 - 모든 HTML에서 style.css를 적용할 수 있도록 <link>를 추가한다.



[과제] 미션2 - 박스 모델의 이해

- 가장 중요한 박스 모델(width, padding, margin, border)을 시각적으로 이해한다.
 - body 태그 아래에 <div class="container">를 추가하고 container를 꾸민다. (모든 콘텐츠를 가운데 정렬한다.)
 - myTrip.html: 여행지 목록의 각 단락에 클래스를 추가하고(<p class="trip-card">)에 배경색, 테두리, 패딩, 마진을 조정하여 하나의 '리뷰 카드' 형태로 디자인한다.
 - myClass.html: table, th, td를 꾸며 테이블을 깔끔한 테이블을 만든다.



시간	월요일	화요일	수요일	목요일	금요일
09:00 - 10:00		네트워크		IT 트렌드 특강	
10:00 - 11:00	Vue.js 웹 프레임워크	보안 기초	웹 시스템 설계 및 분석	데이터베이스	자바스크립트
11:00 - 12:00		알고리즘 문제풀이		SQL 실습	심화 실습
12:00 - 13:00	점심 시간				
13:00 - 14:00		리눅스 시스템 관리	클라우드 AWS 기초		개인 프로젝트
14:00 - 15:00	UI/UX 디자인 표준			자료구조	
15:00 - 16:00					오픈소스 소프트웨어
16:00 - 17:00		Git & GitHub 버전 관리	인공지능 머신러닝 기초	빅데이터 분석 시론	

[과제] 미션3 - 가독성 높은 회원가입 폼

- 스타일링하기 까다로운 폼 스타일링
 - 입력창 크기 키우기
 - Fieldset 그룹 테두리 다듬기
 - 버튼 꾸미기


회원가입

SKALA-FRONT 회원가입을 환영합니다.

계정 정보

아이디:

4~15자 영문/숫자

(필수)

비밀번호:

8자 이상 입력

(필수)

이메일:

example

@

직접 입력

개인 프로필 정보

이름:

생년월일:

연도-월-일

성별: ☒ 남성 ☐ 여성 ☐ 선택안함

관심 분야 (중복 선택 가능):

☐ 웹 프론트엔드 (Vue.js/HTML)

☐ UI/UX 디자인 표준

☐ 백엔드 & 데이터베이스

☐ 클라우드 & 인프라

가입 경로:

-- 선택해 주세요 --

자기소개

나를 표현하는 한 줄 소개 또는 가입 인사:

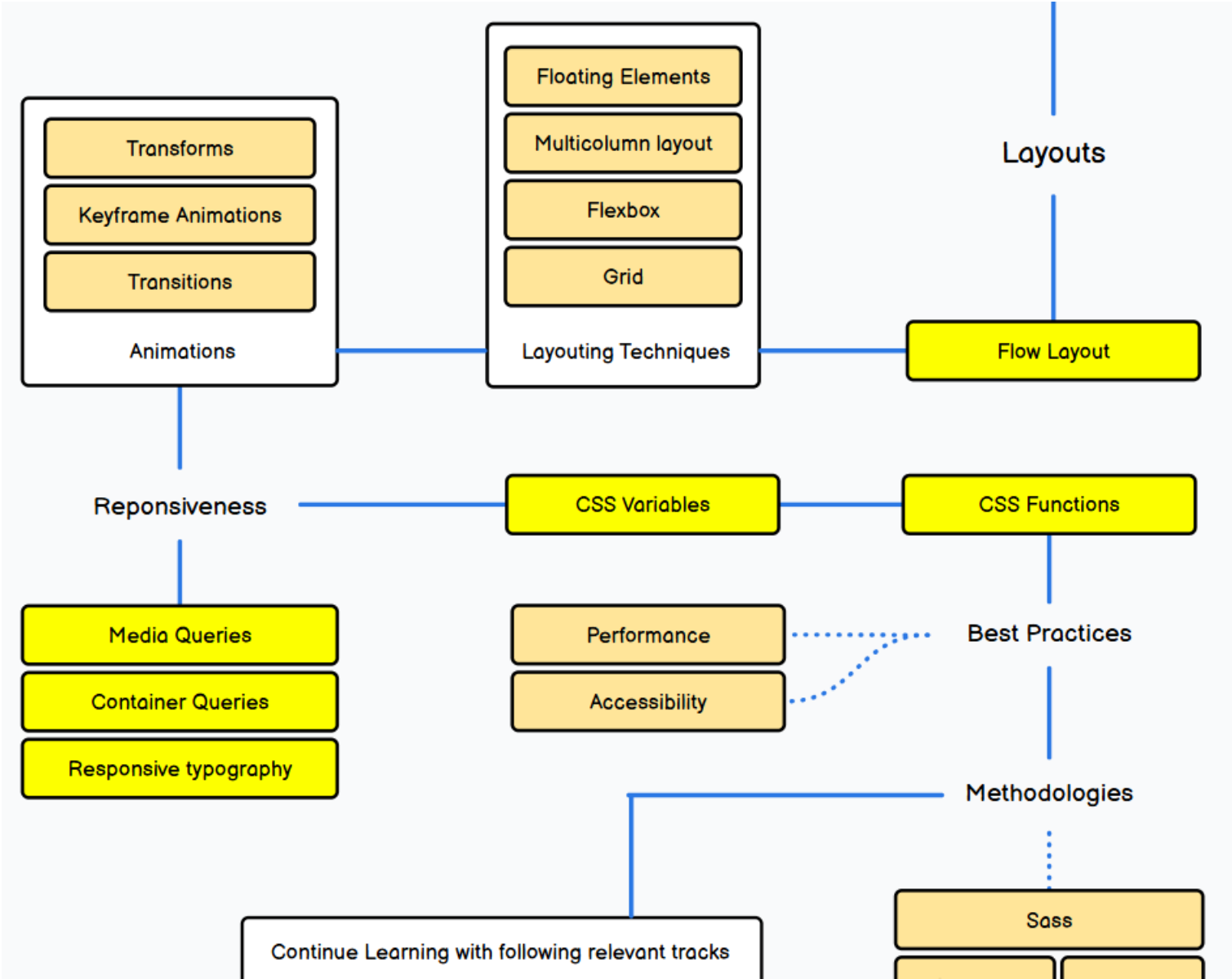
여기에 내용을 입력하세요 (최대 200자)

동의하고 회원가입
다시 작성

Full-Stack Engineering - HTML, CSS, JavaScript

6. CSS 심화

CSS Roadmap



CSS Layout

Layout 구성방식

방식	설명	장점	단점	사용 시기
float	float: left/right를 이용해 레이아웃 구성	간단한 구조에 적합	요소 겹침	구버전 브라우저 호환 목적
inline-block	display: inline-block으로 요소를 나란히 배치	float보다 자연스럽게 텍스트처럼 흐름에 따라 배치	여백(space) 문제 있음	간단한 레이아웃에 사용 가능
Flexbox	1차원(가로 또는 세로) 정렬에 특화된 현대적 레이아웃 방식	간단한 정렬/중앙정렬/비율 지정에 강력	2차원 배치에 제한	공간 내부 요소 정렬
Grid	2차원(행과 열) 레이아웃을 구성하는 최신 방식	복잡한 레이아웃도 간결하게 구성	1차원 구성이 복잡	페이지 전체 레이아웃 구성

- 일반적으로 Flexbox와 Grid를 함께 사용: 페이지 전체는 Grid, 내부 요소는 Flexbox로 정렬

CSS Layout - Float

- The **float** property specifies how an element should float within its container.
- The **float** property can have one of the following values:
 - **left** - The element floats to the left of its container
 - **right** - The element floats to the right of its container
 - **none** - Default. The element does not float and is displayed just where it occurs in the text
 - **inherit** - The element inherits the float value of its parent

```
<!DOCTYPE html>
<html>
<head>
<style>
div {float: left; padding: 15px;}
.div1 {background: red;}
.div2 {background: yellow;}
.div3 {background: green;}
</style>
</head>
<body>
<h2>Float Next To Each Other</h2>
<p>In this example, the three divs will float next to each other.</p>
<div class="div1">Div 1</div>
<div class="div2">Div 2</div>
<div class="div3">Div 3</div>
</body>
</html>
```

Float Next To Each Other

In this example, the three divs will float next to each other.



CSS Layout - Inline-block

- 모든 element는 종류에 따라 가지고 있는 기본 display 값이 있는데 그것이 바로 Block과 Inline이다.
 - Block – 혼자서 한 줄 차지. <div>, <p>, <h1>~<h6>, ,
 - Inline – 줄 바꿈 없이 다른 요소와 같은 줄에 배치되어 width/height 속성이 무시됨. , <a>, ,
- display: inline-block은 배치는 Inline처럼 나란히, 성격은 Block처럼 크기 조절 가능하게 섞어 놓은 속성이다.

```
<head>
<style>
  .nav {background-color: lightgray; list-style-type: none; margin: 0; padding: 0;}
  .nav li {display: inline-block; font-size: 18px; padding: 15px;}
</style>
</head>
<body>
  <h1>Horizontal Navigation Menu</h1>
  <p>List items are displayed vertically by default. Here, we use display: inline-block to display them horizontally (side by side).</p>
  <ul class="nav">
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About Us</a></li>
    <li><a href="#clients">Our Clients</a></li>
    <li><a href="#contact">Contact Us</a></li>
  </ul>
</body>
</html>
```

Horizontal Navigation Menu

List items are displayed vertically by default. Here, we use display: inline-block to display them horizontally (side by side).

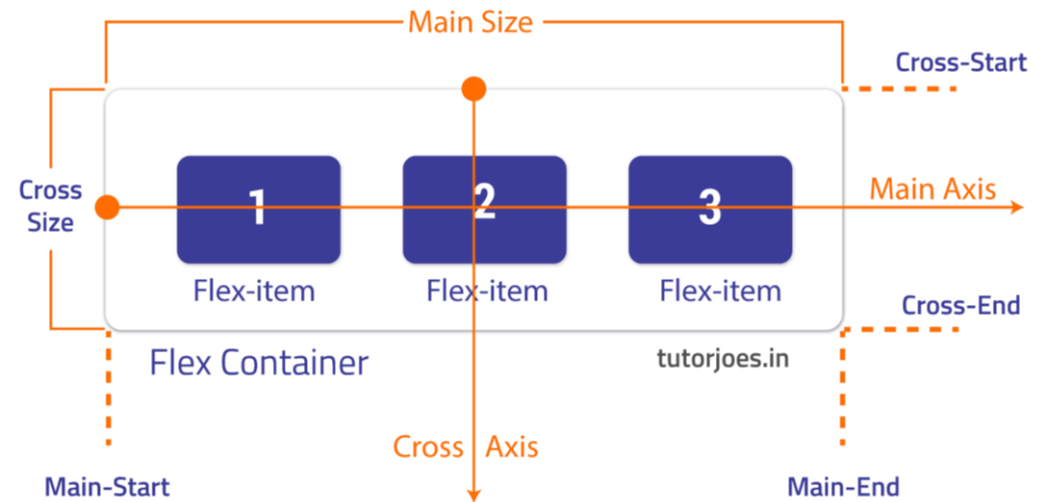
[Home](#) [About Us](#) [Our Clients](#) [Contact Us](#)

CSS Layout - Flexbox (Flexible Box Layout)

- 기존의 float나 inline-block 가지고 전체 레이아웃을 잡으려면 많이 복잡함.
- Flexbox는 가로 또는 세로 Layout을 효율적으로 구성할 수 있게 해 준다.
- Flexbox 핵심 개념

용어	설명
Flex Container	display: flex가 적용된 부모 요소
Flex Item	Flex Container의 직계 자식 요소들
Main Axis (메인 축)	Flex Item이 배치되는 중심 방향. Row (가로, 기본값) 또는 column (세로)
Cross Axis (교차 축)	Main Axis의 수직 방향

```
<style>
.container {
  display: flex;
  background-color: DodgerBlue;
}
</style>
```



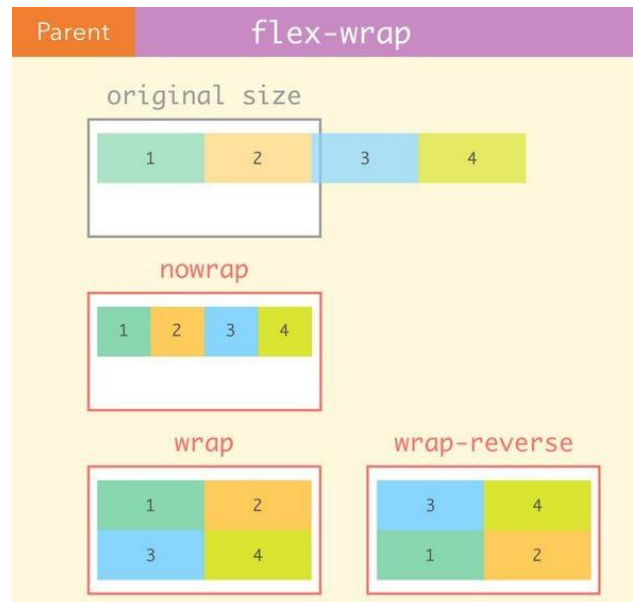
CSS Layout - Flex Container

Demo

- The flex container element can have the following properties

속성	값	설명
flex-direction	row(default), column, row-reverse, column-reverse	아이템 배치 방향 지정
flex-wrap	nowrap(default), wrap, wrap-reverse	넘치는 아이템을 줄바꿈할지 여부
justify-content	center, flex-start(default), flex-end, space-around, space-between, space-evenly	메인 축 정렬 방식
align-items	normal(default), stretch, center, flex-start, flex-end, baseline	교차 축에서 한 줄 정렬 방식
align-content	stretch(default), center, flex-start, flex-end, space-between, space-around 등	여러 줄 정렬 시 교차 축 처리

```
.container {
  display: flex;
  height: 200px;
  background-color: #f1f1f1;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
}
```



CSS Layout - Flex Item

- Flex items can have the following properties
 - order - HTML 코드 순서와 상관없이 시각적인 배치 순서를 지정
 - flex-grow - 부모 박스 안에 남은 공간이 있을 때, 자식들이 그 공간을 얼마나 나눠 가질지(비율) 결정. 기본 값은 0
 - flex-shrink - 부모 박스가 자식들보다 작아질 때, 자식들이 얼마나 줄어들지 결정. 기본 값은 1
 - flex-basis - 자식 요소의 기본 크기를 설정 (width와 유사하지만 flex 환경에 더 최적화 됨)
 - flex - flex-grow, flex-shrink, and flex-basis를 한번에 쓰는 속성
 - Align-self - flex container가 정해진 정렬 규칙을 무시하고 이 아이템만 다르게 정렬하고 싶을 때 사용

```
.item {  
  flex-grow: 1;      /* 남은 공간 비율로 커짐 */  
  flex-shrink: 1;    /* 줄어드는 비율 */  
  flex-basis: 100px; /* 기본 크기 */  
  flex: 1 1 100px;   /* 위 3개의 축약형 */  
  align-self: center; /* 개별 정렬 */  
  order: 2;          /* 순서 변경 */  
}
```

CSS Layout - Grid

- Flexbox가 1차원 레이아웃이라면 Grid는 2차원 레이아웃(행과 열)을 다룰 수 있다.
 - 페이지 전체 구조나 격자 기반 UI(갤러리, 대시보드 등)에 적합
- Grid 핵심 개념

용어	설명
Grid Container	display: grid가 선언된 부모 요소
Grid Item	Grid Container의 직계 자식 요소
Grid Line	행과 열을 나누는 선
Grid Track	행(row) 또는 열(column)
Grid Cell	하나의 셀 (row와 column이 만나는 공간)
Grid Area	여러 셀을 묶은 영역

My Header	
Link 1 Link 2 Link 3	Lorem Ipsum Lorem ipsum odor amet, consectetur adipiscing elit. Ridiculus sit nisl laoreet facilisis aliquet. Potenti dignissim litora eget montes rhoncus sapien neque urna. Cursus libero sapien integer magnis ligula lobortis quam ut.
Footer	

CSS Layout - Grid Container

Grid Container의 속성

구분	속성	예시 값	설명
Grid Tracks	grid-template-columns	200px 1fr 2fr /* 3개 열: 200px, 남은공간의 비율 1, 비율 2로 분할 */	열의 수와 너비 정의
	grid-template-rows	100px auto /* 2개 행 각각 100px, 200px 높이 */	행의 수와 높이 정의
Grid Gaps	gap, row-gap, column-gap	10px	행과 열 사이 간격 설정
Grid Align	justify-items	start, end, center, stretch	각 셀 내 요소의 수평 정렬
	align-items	start, end, center, stretch	각 셀 내 요소의 수직 정렬



* fr (Fraction): Grid Container의 남은 유연한 공간을 비율로 나누어 갖는 Grid 전용 상태 단위

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr; /* 열 정의 */
  grid-template-rows: auto 100px; /* 행 정의 */
  gap: 10px; /* 셀 사이 간격 */
  justify-items: center; /* 셀 내부 가로 정렬 */
  align-items: center; /* 셀 내부 세로 정렬 */
}
```

CSS Layout - Grid Item

Demo

- Grid items can have the following properties
 - 그리드 라인 번호로 위치 지정하기 : grid-column-start, grid-column-end, grid-column, grid-row-start, grid-row-end, grid-row (참고로, 그리드는 셀(칸)의 개수가 아니라, 칸을 나누는 **선(Grid Line)의 번호**를 기준으로 구역을 잡는다.)
 - Grid Container가 기정한 이름에 배치하기 : grid-area
 - Grid Container 정렬규칙을 따르지 않고 개별 정렬하기 : justify-self, align-self

```
.item {  
  grid-column: 1 / 3;    /* 1번 선부터 3번 선까지 열 병합 */  
  grid-row: 2 / 4;       /* 2번 선부터 4번 선까지 행 병합 */  
  justify-self: center; /* 개별 요소 수평 정렬 */  
  align-self: end;       /* 개별 요소 수직 정렬 */  
}
```


CSS Code Challenge

- 다음 실습을 진행하시오

- 1) CSS Float - https://www.w3schools.com/css/css_challenges_float.asp
- 2) CSS Inline-block - https://www.w3schools.com/css/css_challenges_inline-block.asp
- 3) Flexbox Intro - https://www.w3schools.com/css/css_challenges_flexbox_intro.asp
- 4) Flexbox Container - https://www.w3schools.com/css/css_challenges_flexbox_container.asp
- 5) Flexbox Items - https://www.w3schools.com/css/css3_flexbox_items.asp
- 6) Grid Intro - https://www.w3schools.com/css/css_challenges_grid_intro.asp
- 7) Grid Container - https://www.w3schools.com/css/css_challenges_grid_container.asp
- 8) Grid Items - https://www.w3schools.com/css/css_challenges_grid_item.asp

CSS 2D Transforms

- The CSS **transform** property applies a 2D or 3D transformation to an element.
 - transform 속성은 element를 회전하거나, 크기조정을 하거나, 이동시키거나, 기울이는데 사용한다.
- Transform Functions: transform 속성이 사용하는 함수
- 2D Transform 함수 사용 예

```
transform: rotate(45deg);    /* 45도 회전 */  
transform: scale(1.2);       /* 크기 1.2배 */  
transform: translateX(50px); /* x축으로 50px 이동 */
```

CSS 3D Transforms

- The CSS **transform** property applies a 2D or 3D transformation to an element.
 - rotateX(), rotateY(), rotateZ() 함수를 통해 X축, Y축, Z축을 기준으로 element를 회전할 수 있다.
- Transform Functions: transform 속성이 사용하는 함수
- 3D Transform 속성 목록

Property	Description
transform	Applies a 2D or 3D transformation to an element
transform-origin	Allows you to change the position on transformed elements
transform-style	Specifies how nested elements are rendered in 3D space
perspective	Specifies the perspective on how 3D elements are viewed
perspective-origin	Specifies the bottom position of 3D elements
backface-visibility	Defines whether or not an element should be visible when not facing the screen

CSS Transition

- CSS의 속성에 대한 값이 변할 때, 변화 시의 효과를 설정한다.

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background-color: red;
  transition: width 2s, height 4s, background-color 3s;
  /* width 변하는데 2초, height 변하는데 4초, 배경색 변하는데 3초 걸림 */
}
div:hover {
  width: 300px;
  height: 300px;
  background-color: orange;
}
</style>
</head>
<body>
  <h1>The transition Property</h1>
  <p>Hover over the div element below, to see the transition effect:</p>
  <div></div>
</body>
</html>
```

CSS Transition Timing

■ 전환 효과와 관련된 속성

- transition-timing-function: transition 효과가 진행되는 속도의 형태 지정
- transition-delay: transition이 시작되기 전의 지연 시간 지정
- transition-duration: transition 효과가 진행되는 시간 지정
- transition-property: transition 효과를 부여할 css property 지정

```
div {  
  transition-property: width;  
  transition-duration: 2s;  
  transition-timing-function: linear;  
  transition-delay: 1s;  
}
```

- transition: 위 4개의 transition 속성을 한번에 작성할 수 있는 속성 (shorthand property)

```
div {  
  transition: width 2s linear 1s;  
}
```

CSS Animations

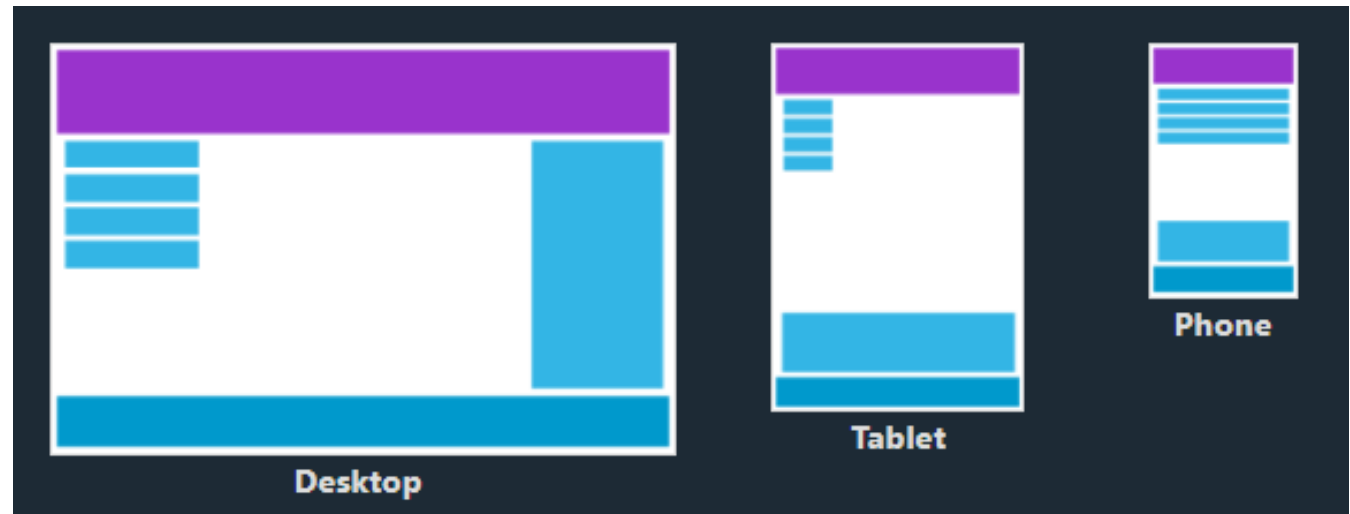
- CSS Transition은 element의 전후(시작과 종료) 상태를 부드럽게 변화한다.
- CSS Animation은 시작과 종료 뿐만 아니라 중간 상태도 고려하여 더 복잡하고 다양한 효과를 제공한다.

속성	의미	기본값
animation-name	@keyframes 의 이름	none
animation-duration	지속 시간	0s
animation-delay	대기 시간	0s
animation-timing-function	타이밍 함수	ease
animation-iteration-count	반복 횟수	1
animation-direction	반복 방향	normal
animation-fill-mode	전후 상태	none
animation-play-state	재생과 정지	running

- @keyframe은 애니메이션의 각 단계를 설정한다.

Responsive Web Design

- 반응형 웹 디자인(Responsive Web Design)은 하나의 **HTML** 소스로 **PC, 태블릿, 스마트폰** 등 다양한 크기의 기기(화면)에 맞춰 레이아웃이 자동으로 변형되도록 디자인하는 웹 개발 방식을 말한다.
- 반응형 웹의 핵심 기술
 - Viewport <meta> tag
 - Mediaqueries
 - Flexible Layout (Grid and Flex)
- 브라우저는 웹 페이지를 렌더링할 때 기기의 화면 정보를 파악
 - width, height: 디바이스 또는 브라우저의 뷰포트 크기
 - orientation: 세로(portrait) 또는 가로(landscape)
 - resolution: 해상도 (dpi 또는 dppx)
 - color, aspect-ratio 등도 감지 가능



Responsive Web Design - Viewport

- 반응형 웹이 정상적으로 작동하려면 <head> 태그 안에 반드시 Viewport 메타 태그를 넣어주어야 한다.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- width=device-width: 기기의 실제 화면 너비에 맞춰 콘텐츠 폭을 설정
- initial-scale=1.0: 초기 확대/축소 비율을 1로 설정 (100%)



Responsive Web Design - Media Query

- 디바이스의 화면 크기, 방향, 해상도 등 미디어 특성에 따라 서로 다른 스타일을 적용할 수 있게 해주는 핵심 기술
 - 화면 레이아웃이 바뀌는 기준이 되는 지점을 브레이크포인트(Breakpoint)라고 부른다.
- Breakpoints 샘플

```
/* 1. 모바일 (Mobile) */ /* 기본 스타일은 모바일 기준으로 작성 (Mobile First 방식 시) */
@media (max-width: 767px) {
    /* 스마트폰 가로/세로, 대화면 스마트폰 */
}

/* 2. 태블릿 (Tablet) */
@media (min-width: 768px) and (max-width: 1023px) {
    /* 아이패드 등 일반적인 태블릿 디바이스 */
}

/* 3. 소형 노트북 / PC (Laptop) */
@media (min-width: 1024px) and (max-width: 1439px) {
    /* 13인치, 15인치 노트북 및 작은 모니터 해상도 */
}

/* 4. 대화면 데스크톱 (Desktop) */
@media (min-width: 1440px) {
    /* 일반적인 모니터(1920px 이상) 및 대화면 디스플레이 */
}
```

Responsive Web Design - Grid View

Demo

- 많은 웹 페이지는 그리드 뷰를 기반으로 하는데, 반응형 그리드 뷰는 일반적으로 6개 또는 12개의 열을 가진다.
- Grid View 샘플 : https://www.w3schools.com/css/css_rwd_grid.asp

Responsive Web Design - Mobile First

- 반응형을 짤 때 두 가지 접근 방식
 - Mobile First: 모바일 화면 스타일을 먼저 만들고, 미디어 쿼리(min-width)를 사용해 화면이 커질 때마다 살을 붙여 나가는 방식
 - Desktop First: PC 화면을 먼저 만들고, 미디어 쿼리(max-width)를 사용해 화면이 작아질 때마다 요소를 떼어내거나 줄이는 방식
- 사용자들이 모바일 기기를 우선적으로 사용하는 시대 흐름에 맞춰 Mobile-First Design이 권장됨

항목	설명
설계 순서	모바일 → 태블릿 → 데스크탑
장점	핵심 기능에 집중, 로딩 속도 최적화, 반응형 구현에 유리
기술적 구현	CSS min-width 기반 미디어 쿼리 사용

CSS Variables (= Custom Properties)

■ 변수 선언하기: --변수명

- 보통 CSS 변수는 사이트 전체에서 전역적으로 쓰기 위해 가장 상위 요소인 :root에 선언한다.
- :root는 HTML 문서의 가장 최상단 요소(<html> 태그)를 가리키는 가상 선택자이다.

```
:root {  
  /* --변수명: 값; 구조로 선언합니다 */  
  --main-color: #007bff;  
  --danger-color: #dc3545;  
  --default-padding: 16px;  
}
```

■ 변수 사용하기: var(--변수명)

```
.button-primary {  
  background-color: var(--main-color); /* #007bff가 적용됨 */  
  padding: var(--default-padding);    /* 16px이 적용됨 */  
}  
.alert-box {  
  border: 2px solid var(--danger-color);  
}
```

■ CSS 변수 사용의 장점

- 반복되는 값의 재사용이 가능하다.
- 특히, JavaScript에서 동적으로 값을 변경하여, 테마(다크/라이트 모드) 전환 기능을 구현할 때 핵심 기술로 사용된다.

CSS Preprocessor

■ CSS Vs. SCSS/SASS

구분	CSS	SCSS (Sassy CSS)	SASS (Syntactically Awesome Style Sheets)
정의	웹 표준 스타일시트 언어	SASS의 3세대 문법 (CSS 친화적)	CSS를 확장한 전처리기 언어
구문 형태	중괄호({})와 세미콜론(;)	중괄호({})와 세미콜론(;)	들여쓰기와 줄바꿈
가독성	구조가 커질수록 복잡해짐	CSS와 유사하여 가독성 좋음	중괄호가 없어 깔끔하지만 호불호
호환성	브라우저가 직접 인식함	CSS와 100% 호환됨	전통적인 SASS 문법 (적용 필요)
사용 빈도	기본 필수	현재 업계 표준/가장 많이 씀	점차 줄어드는 추세

```
nav {
  background-color: #333;
}
nav ul {
  list-style: none;
}
nav ul li {
  display: inline-block;
}
```

```
nav {
  background-color: #333;
  ul {
    list-style: none;
    li {
      display: inline-block;
    }
  }
}
```

```
nav
  background-color: #333
  ul
    list-style: none
    li
      display: inline-block
```

CSS Preprocessor - SCSS

▪ SCSS/SASS 장점

- 변수 사용 (\$variable): 색상, 폰트 크기 등을 변수로 지정해 두고 재사용할 수 있다. (예: \$primary-color: #ff0000;)
- 중첩 (Nesting): 상위 선택자의 반복을 줄여 컴포넌트 단위로 관리가 편해진다.
- 믹스인 (Mixins): 자주 사용하는 CSS 코드 블록을 정의해 두고 필요할 때마다 불러올 수 있다. (함수와 유사)
- 모듈화 (Import): 스타일시트를 여러 파일로 쪼개어 관리하고 하나로 합칠 수 있어 **대규모 프로젝트에 필수적이다**.

▪ Convert SCSS/SASS code into standard CSS

- SCSS나 SASS로 작성한 코드는 브라우저가 직접 읽을 수 없기 때문에, 작성된 스타일시트를 일반 CSS로 바꾸어주는 과정이 필요
- 이 과정을 트랜스파일링(Transpiling)이라 한다.
- Vite, Webpack, Turbopack 같은 Frontend Build 도구에서는 설정을 통해 자동으로 변환시킨다.

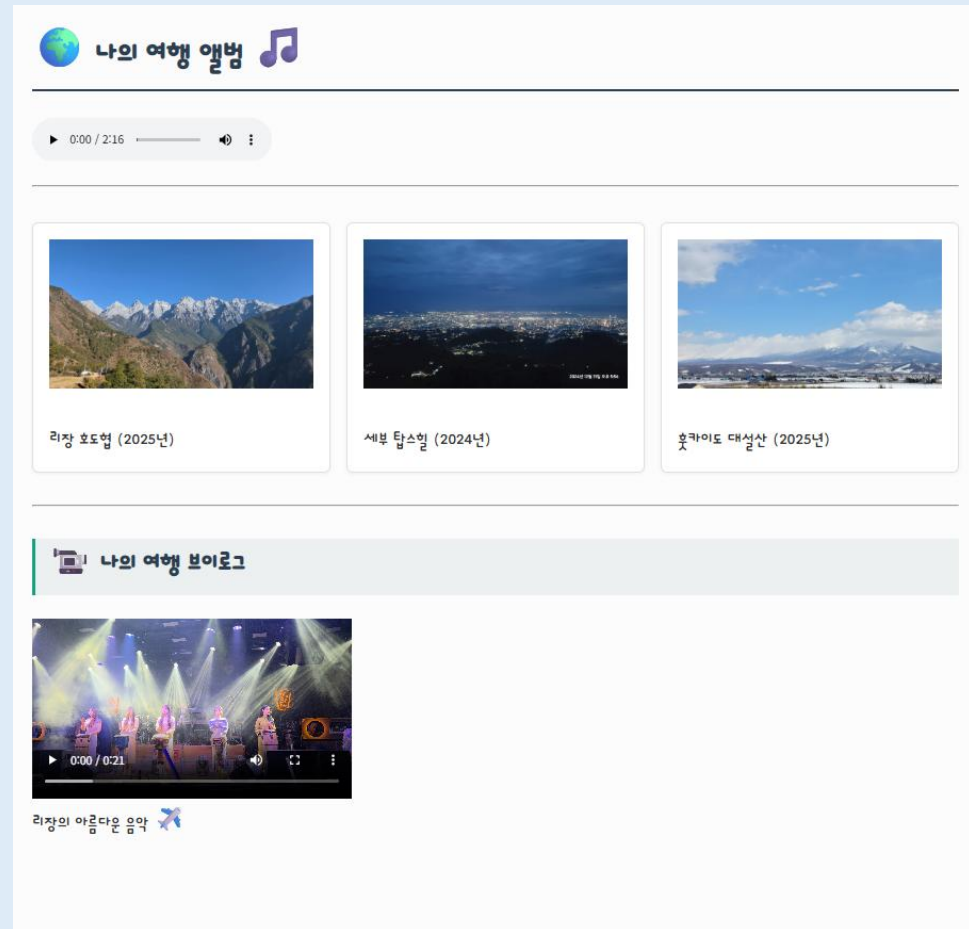
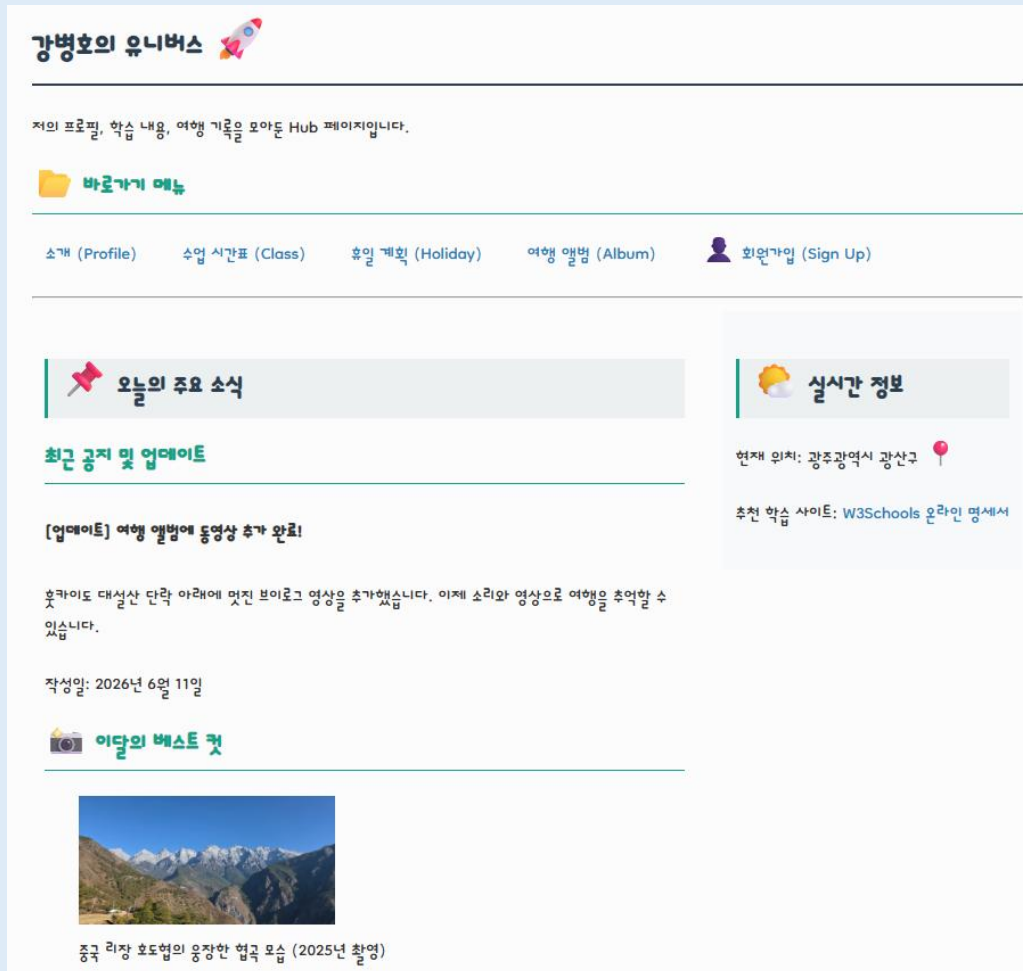
CSS Code Challenge

- 다음 실습을 진행하시오

- 1) 2D Transforms - https://www.w3schools.com/css/css_challenges_css3_transforms.asp
- 2) 3D Transforms - https://www.w3schools.com/css/css_challenges_css3_3dtransforms.asp
- 3) Transition - https://www.w3schools.com/css/css_challenges_css3_transitions.asp
- 4) Animations - https://www.w3schools.com/css/css_challenges_css3_animations.asp
- 5) RWD Intro - https://www.w3schools.com/css/css_challenges_rwd_intro.asp
- 6) RWD Viewport - https://www.w3schools.com/css/css_challenges_rwd_viewport.asp
- 7) RWD Grid View - https://www.w3schools.com/css/css_challenges_rwd_grid.asp
- 8) RWD Media Queries - https://www.w3schools.com/css/css_challenges_rwd_grid.asp
- 9) CSS Variables - https://www.w3schools.com/css/css_challenges_css3_variables.asp

[실습] 미션4 - Flex와 Grid로 레이아웃 잡기

- 위에서 아래로 일렬로만 떨어지던 답답한 배치를 현대적인 Multi-column 레이아웃과 바둑판 배열을 적용한다.
 - 1) Index.html : 바로가기에 Flexbox를 적용하고, main과 aside를 가로 배치한다.
 - 2) myTrip.html : 여행지 카드를 Grid를 사용하여 3열 바둑판 배치 한다.



[실습] 미션5 - 스마트폰에서 보기 (반응형 웹 디자인)

- 레이아웃이 화면이 작은 모바일 기기에서도 깨지지 않고 유연하게 변하도록 한다.
 - 1) Index.html : 화면 폭이 786 px 이하로 줄면 본문|사이드바 구조를 세로 1열로 변경하고 바로가기로 1열로 정렬
 - 2) myTrip.html : 3열 배열을 1열로 조정

강병호의 유니버스 

저의 프로필, 학습 내용, 여행 기록을 모아둔 Hub 페이지입니다.

바로가기 메뉴

소개 (Profile)

수업 시간표 (Class)

휴일 계획 (Holiday)

여행 앨범 (Album)



회원가입 (Sign Up)



오늘의 주요 소식

최근 공지 및 업데이트

[업데이트] 여행 앨범에 동영상 추가 완료!

후카이도 대설산 단락 아래에 멋진 브이로그 영상을 추가했습니다. 이제 소리와 영상으로 여행은 추억할 수 있습니다.



실시간 정보

현재 위치: 광주광역시 광산구

추천 학습 사이트: W3Schools

온라인 명세서

강병호의 유니버스 

저의 프로필, 학습 내용, 여행 기록을 모아둔 Hub 페이지입니다.

바로가기 메뉴

소개 (Profile)

수업 시간표 (Class)

휴일 계획 (Holiday)

여행 앨범 (Album)



회원가입 (Sign Up)



오늘의 주요 소식

나의 여행 앨범 

0:00 / 2:16



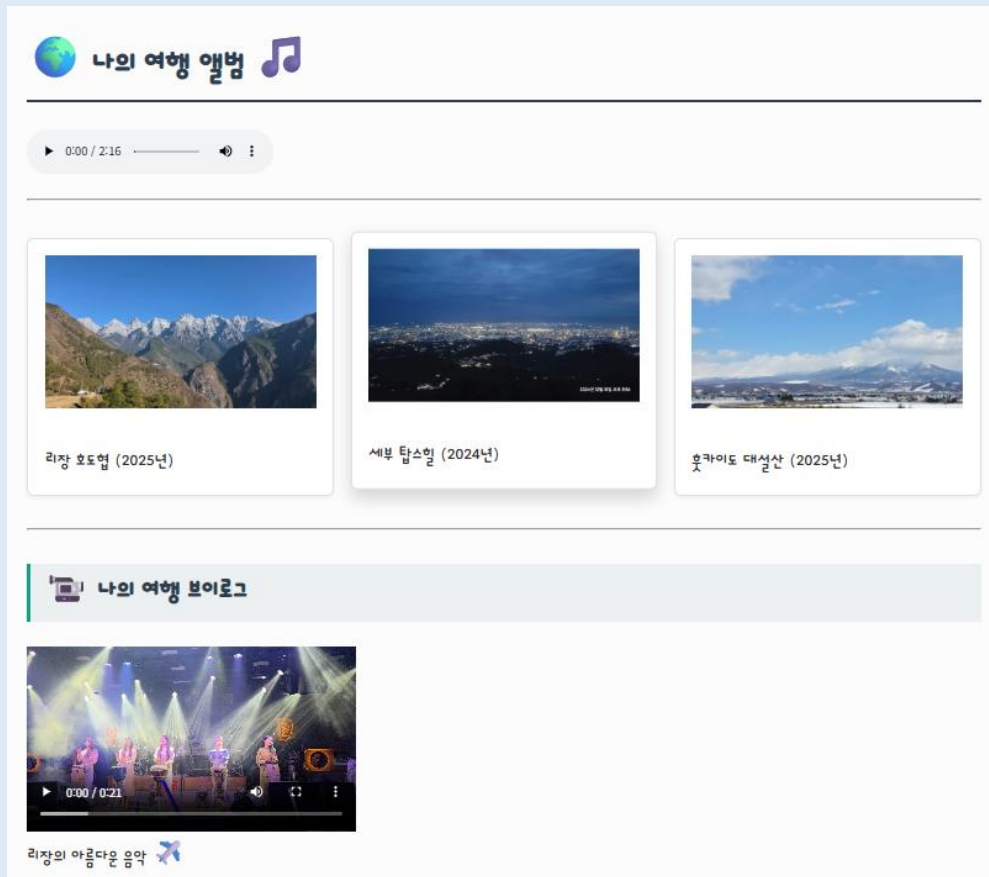
리장 호도협 (2025년)



164

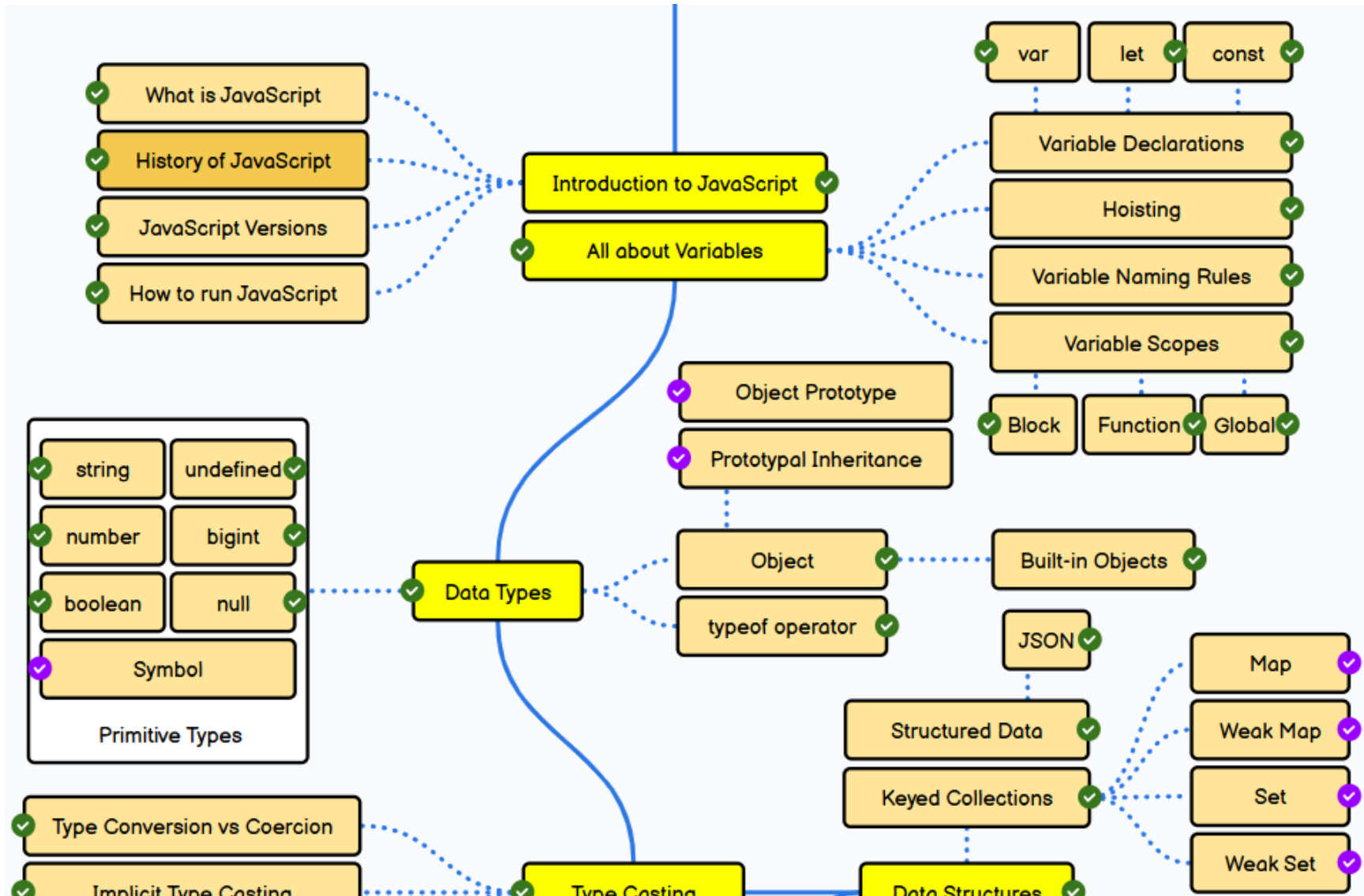
[실습] 미션6 - 생동감을 불어넣는 애니메이션

- 사용자의 마우스 움직임에 반응하고, 화면이 살아 숨 쉬는 듯한 부드러운 효과를 추가하는 미션
 - 1) 내비게이션 메뉴나 회원가입 버튼에 마우스를 올렸을 때 배경색이나 글자색이 순식간에 바뀌지 않고 부드럽게 변하도록 처리
 - 2) 여행 앨범의 카드에 마우스를 올리면 박스가 살짝 위로 떠 오르고 그림자가 더 진해지는 효과를 주어 '클릭하고 싶게' 만든다.
 - 3) index.html을 처음 열었을 때, 상단 헤더의 타이틀이 부드럽게 페이드인 되며 나타나는 등장 애니메이션을 구현한다



Full-Stack Engineering - HTML, CSS, JavaScript

7. JavaScript 기초



Introduction to JavaScript

- 웹 브라우저와 서버 모두에서 작동하는 프로그래밍 언어
- Interactive한 웹 페이지와 다양한 애플리케이션을 개발하기 위해 사용
- HTML과 CSS와 함께 웹 개발의 핵심 기술 중 하나 (자바스크립트는 '동적 제어와 논리(Behavior)'를 담당)
 - JavaScript Can Change HTML Content
 - JavaScript Can Change HTML Attribute Values
 - JavaScript Can Change HTML Styles (CSS)
 - JavaScript Can Hide/Show HTML Elements
- JavaScript의 주요 기술적 특징

특정	설명
Interpreter Language	별도의 컴파일(빌드) 과정 없이, 브라우저가 코드를 한 줄씩 위에서부터 읽으며 즉시 실행
Dynamic Typing	변수를 만들 때 데이터 타입을 미리 지정하지 않고, 값이 할당되는 순간 자동으로 타입이 결정
Single Thread	한 번에 하나의 작업만 처리하는 Single Thread를 사용하되, 사용자의 행동(이벤트)을 감지해 비동기로 처리 함으로써 화면이 멈추지 않게 제어한다.

Introduction to JavaScript - History

JavaScript의 주요 역사 및 발전 과정

시대 (연도)	주요 사건	상세 내용 및 기술적 의미
1995년	자바스크립트의 탄생	1995년 Netscape의 브렌던 아이크(Brendan Eich)가 단 10일 만에 언어를 설계 ※ Java와 JavaScript는 이름만 비슷할 뿐, 아무런 연관이 없는 별개의 언어
1996~ 2000년대 초	브라우저 전쟁과 ECMAScript 제정	1996년 MS가 IE에 복제판인 JScript를 탑재하면서 브라우저 전쟁 시작 1997년 국제 표준화 기구(ECMA)에서 공식 표준 문법인 ECMAScript(ES)를 제정
2005~2006년	AJAX의 등장과 jQuery 혁명	2005년 구글 맵스 등이 데이터만 바꾸는 AJAX(비동기 통신)를 선보임. 2006년 브라우저 간 호환성 문제를 해결해 주는 jQuery 라이브러리가 출시
2009~2015년	Node.js 탄생과 ES6(ES2015) 대개정	2009년 Node.js의 등장으로 서버 환경에서도 자바스크립트를 실행할 수 있게 됨. 2015년 ES6 문법 대개정으로 현대적이고 강력한 기능이 도입되었음
2016년~현재	모던 웹 생태계와 매년 정기 업데이트	- React, Vue, Angular 중심의 컴포넌트 기반 개발이 글로벌 표준으로 자리 잡았습니다. - 매년 소규모 기능을 정기 업데이트

※ 자바스크립트의 공식 표준명 = ECMAScript(ES)

※ Modern JavaScript라고 부르는 기준점은 바로 ES6(ES2015)

Introduction to JavaScript - How to Run

JavaScript 코드를 HTML 문서에 포함하여 작성

- HTML문서에서 JavaScript 코드는 `<script>` `</script>` Tag 사이에 작성되어야 한다.

```
<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

- JavaScript는 `<body>`나 `<head>` section 내에서 작성된다.
 - `<head>`에 스크립트를 넣으면 스크립트가 로드되는 동안 HTML 요소가 렌더링되지 않기 때문에 페이지가 늦게 표시된다.
 - `<body>` 끝에 스크립트를 넣으면 HTML 문서의 모든 콘텐츠가 먼저 렌더링 된 후 실행됨으로 사용자에게 더 빠른 경험을 제공
- External JavaScript

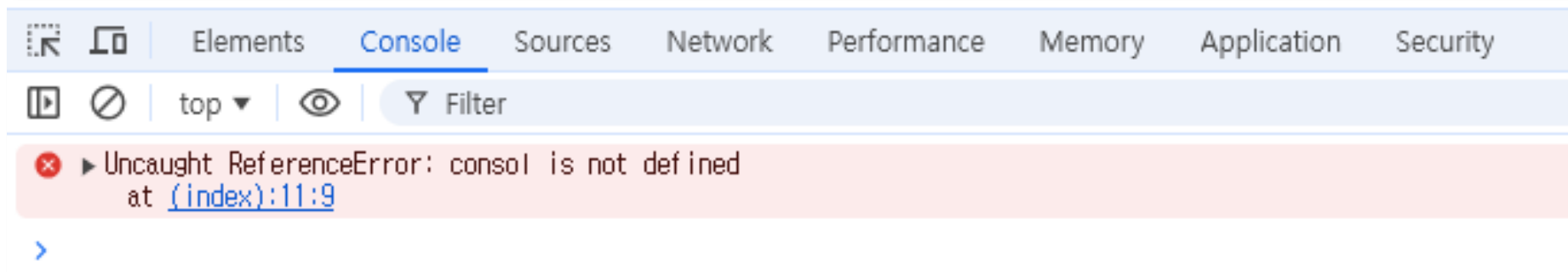
```
<script src="https://www.w3schools.com/js/myScript.js"></script>
<script src="/js/myScript.js"></script>
<script src="myScript.js"></script>
```

Browser 별 JavaScript Engine

Browser	JS Engine	핵심 기술적 특징
Google Chrome	V8	전 세계에서 가장 널리 쓰이는 오픈소스 엔진으로 속도가 매우 빠르다.
Apple Safari	JavaScriptCore	macOS, iOS 등 Apple 하드웨어 생태계에 최적화된 엔진
Mozilla Firefox	SpiderMonkey	넷스케이프 시절 처음 만든 세계 최초의 자바스크립트 엔진
Microsoft Edge	V8	구글 크롬과 동일한 오픈소스 엔진인 V8을 탑재하여 구동

Introduction to JavaScript - Output

- JavaScript가 데이터를 보여주는 방법
 - Writing into an HTML element, using innerHTML or innerText.
 - Writing into the HTML output using document.write().
 - Writing into an alert box, using window.alert().
 - Writing into the browser console, using `console.log()`.
- 개발자 도구 Console Tab
 - 단축키 : Ctrl + Shift + I
 - Console Tab에서 `console.log()`를 확인할 수 있다.
 - Console Tab에서 자바스크립트 에러를 확인할 수 있다.



Introduction to JavaScript - JS Syntax

- JavaScript Value는 Literals(Fixed Values)와 Variables(Variable Values) 2종이 존재한다.
- JavaScript Keyword는 수행할 동작을 정의하는 데 사용된다.
- JavaScript Variable은 데이터를 저장하기 위해 이름을 붙인 저장 공간이다.
- JavaScript Identifier는 변수 이름이다.
- JavaScript Operator는 변수간 연산을 하는 연산자이다.
- JavaScript Expression은 value, variables, operators의 조합이다.
- JavaScript는 Case-Sensitive 하다.

Introduction to JavaScript - Statements & Comments

- JavaScript Programs
 - A computer program is a list of "instructions" to be "executed" by a computer.
 - These programming instructions are called statements.
 - The statements are executed, one by one, in the same order as they are written.
 - In HTML, JavaScript programs are executed by the web browser.
- JavaScript **Statements**
 - JavaScript statements are composed of Values, Operators, Expressions, Keywords, and Comments.
 - Semicolons separate JavaScript statements.
 - JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.
 - JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.
 - JavaScript keywords are reserved words. Reserved words cannot be used as names for variables.
- JavaScript Comments
 - Single line comments start with //.
 - Multi-line comments start with /* and end with */.

JavaScript Variables

- JavaScript variables are containers for data.
 - Variables are identified with unique names called identifiers.
- Declaring JavaScript Variables
 - Declaring a Variable Using let
 - Declaring a Variable Using const
 - Declaring a Variable Automatically (Not Recommended)
 - Declaring a Variable Using var (Not Recommended)

Keyword	범위(Scope) = 변수 접근 가능 범위	재선언	재할당
var	함수 범위(function scope)	O	O
let	블록 범위(block scope)	X	O
const	블록 범위(block scope), 상수	X	X

JavaScript Variables - Scopes

- JavaScript variables have 3 types of scope

- Global Scope: Global variables can be accessed from anywhere in a JavaScript program.

```
let carName = "Volvo";  
// code here can use carName  
  
function myFunction() {  
  // code here can also use carName  
}
```

- Block Scope: Variables declared inside a { } block cannot be accessed from outside the block (let, const 선언에 해당)

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

- Function Scope: Inside a function all variables declared with var, let or const have Function Scope

```
function myfunction() {  
  var x = 1;  
  let y = 2;  
  const z = 3;  
}  
//x can NOT be used here  
//y can NOT be used here  
//z can NOT be used here
```

JavaScript Variables - Variable Declarations (let)

- Declaring a Variable Using **let**
 - The let keyword was introduced in ES6 (2015)
 - Variables declared with let have Block Scope
 - Variables declared with let must be Declared before use
 - Variables declared with let cannot be Redeclared in the same scope

JavaScript Variables - Variable Declarations (const)

- Declaring a Variable Using **const**
 - The const keyword was introduced in ES6 (2015)
 - Variables defined with const cannot be Redeclared
 - Variables defined with const cannot be Reassigned
 - Variables defined with const have Block Scope
- Constant Objects and Arrays
 - It does not define a constant value. It defines a constant reference to a value.

JavaScript Variables - Hoisting

- Hoisting is JavaScript's default behavior of **moving declarations to the top**.
 - JavaScript Declarations are Hoisted
 - Variables defined with `let` and `const` are hoisted to the top of the block, but not initialized.
 - JavaScript Initializations are Not Hoisted

[작성된 코드]

```
console.log(a);  
var a = 10;
```



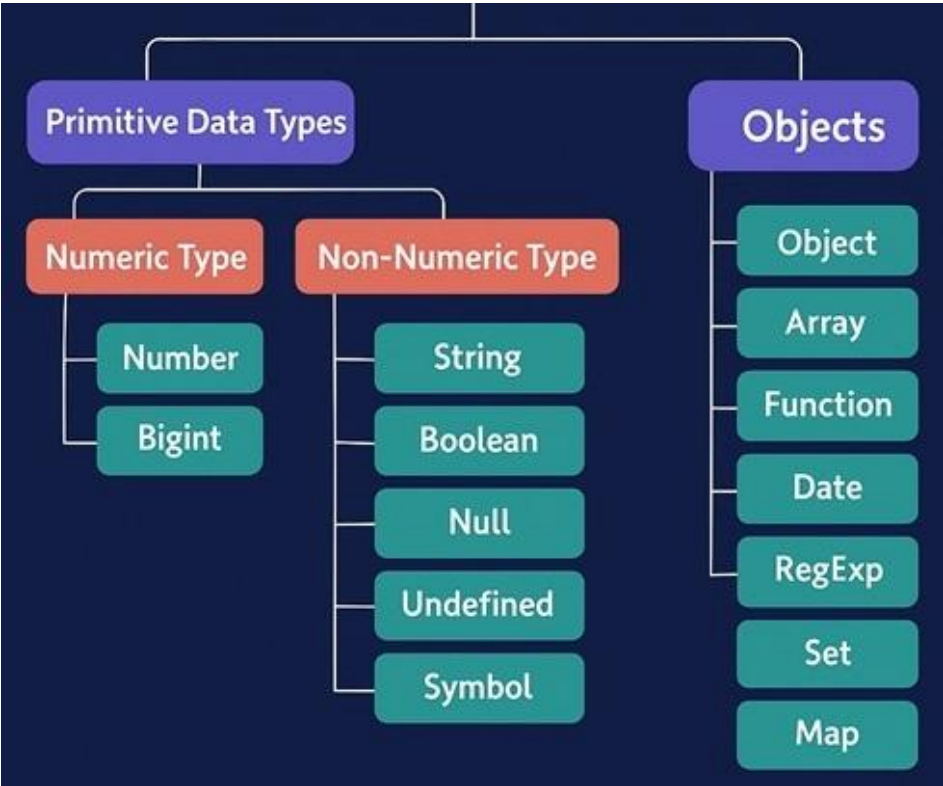
[코드의 동작]

```
var a;           // 선언이 먼저 올라감 (호이스팅)  
console.log(a);  // undefined (값이 아직 할당되지 않음)  
a = 10;
```

- Declare Your Variables At the Top !
 - Hoisting is (to many developers) an unknown or overlooked behavior of JavaScript.
 - If a developer doesn't understand hoisting, programs may contain bugs (errors).
 - To avoid bugs, always declare all variables at the beginning of every scope.
 - Since this is how JavaScript interprets the code, it is always a good rule.

Datatypes

- JavaScript Datatypes: 기본형(Primitive Data Type) + 객체형(Object Data Type, Reference Data Type, Non-Primitive Data Type)으로 구성



구분	원시 타입 (Primitive)	객체 타입 (Object)
값의 형태	단일 값 (단순함)	속성(Key)과 값(Value)의 집합 (복합함)
메모리 저장	변수 공간에 실제 값 이 직접 저장됨	메모리(Heap) 주소를 가리키는 참조값 이 저장
변경 가능성	불변성 (Immutability) : 값 수정 불가	가변성 (Mutability) : 내부 속성을 변경 가능
속성/메서드	없음 (데이터 그 자체)	있음

- JavaScript has dynamic types. This means that the same variable can be used to hold different data types:

```

let x;      // Now x is undefined
x = 5;      // Now x is a Number
x = "John"; // Now x is a String
  
```


Datatypes - String

- JavaScript Strings
 - **Using Quotes:** A JavaScript string is zero or more characters written inside quotes.
 - **Template Strings:** Templates are strings enclosed in **backticks**
 - **String Length:** To find the length of a string, use the built-in length property
 - **Escape Characters:** The backslash escape character (\) turns special characters into string characters:
 - **JavaScript Strings as Objects:** Normally, JavaScript strings are primitive values, created from literals, but strings can also be defined as objects with the keyword new
- JavaScript String Templates (=Template Strings, Template Literals)
 - **Back-Tics Syntax:** Template Strings use back-ticks (` `) rather than the quotes ("") to define a string
 - **Multiline Strings:** Template Strings allow multiline strings
 - **Interpolation:** Template strings provide an easy way to interpolate variables in strings.
 - **Expression Substitution:** Template Strings allow interpolation of expressions in strings:

Datatypes - String Methods

▪ Basic String Methods

- Javascript strings are primitive and **immutable**: All string methods produce a new string without altering the original string.
- String.concat(), String.substring(), String.substr(), String.toUpperCase(), String.trim(), String.replace(), String.split()...

▪ String Search Methods

- String.indexOf(), String.lastIndexOf(), String.search(), String.match(), String.matchAll(), String.includes(), String.startsWith()...

▪ Wrapper Object

- JavaScript 엔진은 원시 타입의 속성이나 메서드에 접근하려고 할 때, 순간적으로 그 원시 타입을 연관된 객체로 임시 변환한다.

```
const name = "gemini"; // 원시 타입인 string
console.log(name.toUpperCase()); // "GEMINI"가 정상 출력됨! 객체인가..?
```

- typeof 연산자를 통한 확인

```
const primitiveStr = "hello";
const objectStr = new String("hello"); // 명시적으로 객체로 만들

console.log(typeof primitiveStr); // "string" (원시 타입)
console.log(typeof objectStr);    // "object" (객체)
```

- Do not create String objects. The new keyword complicates the code and slows down execution speed.

Datatypes - Number

▪ JavaScript Numbers

- **Number Types:** Unlike many other programming languages, JavaScript does not define different types of numbers, like integers, short, long, floating-point etc
- **Integer Precision** : Integers (numbers without a period or exponent notation) are accurate up to 15 digits.
- **Floating Precision** : Floating point arithmetic is not always 100% accurate
- **Adding Numbers and Strings** : JavaScript uses the + operator for both addition and concatenation. Numbers are added. Strings are concatenated.
- **NaN(Not a Number)**: NaN is a JavaScript reserved word indicating that a number is not a legal number.
- **Infinity**: Infinity (or -Infinity) is the value JavaScript will return if you calculate a number outside the largest possible number. Division by 0 (zero) also generates Infinity
- **Hexadecimal**: JavaScript interprets numeric constants as hexadecimal if they are preceded by 0x.
- **JavaScript Numbers as Objects**: Normally JavaScript numbers are primitive values created from literals, But numbers can also be defined as objects with the keyword new:

Datatypes - Number Methods and Properties

- Basic Number Methods
 - **toString()**, toExponential(), toFixed(), toPrecision(), **valueOf()**
- Converting Variables to Numbers
 - Number(), **parseInt()**, **parseFloat()**
- Number Object Methods
 - Number.isInteger(), Number.isNaN(), Number.isFinite(), Number.isSafeInteger(), Number.parseInt(), Number.parseFloat()
- Number Object Properties
 - Number.EPSILON, Number.MAX_VALUE, Number.MIN_VALUE, Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER, Number.POSITIVE_INFINITY, Number.NEGATIVE_INFINITY, Number.NaN

Datatypes - BigInt

- What is JavaScript BigInt?
 - Number의 범위를 초과하는 정수를 표현하기 위해 사용 (Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER).
- How to Create a BigInt
 - Using an integer literal with an n suffix
 - Using the BigInt() constructor with a string

```
let x = 12345678901234567890n;  
let y = BigInt("12345678901234567890")
```

- Arithmetic between a BigInt and a Number is not allowed (will result in a TypeError).

```
let x = 10n;  
let y = 5;  
  
let z = x + y; // ✕ TypeError
```

Datatypes - Boolean

- Key Boolean Characteristics
 - **true** and **false** are the only possible boolean values
 - true and false must be written in lowercase
 - true and false must be written without quotes
- 비교 연산의 결과는 true/false를 return한다.

```
let boolean1 = 10 < 20;  
let boolean2 = 10 > 20;  
console.log(boolean1); // true  
console.log(boolean2); // false
```

- 조건문과 반복문에 광범위하게 사용된다.
- Everything With a "Value" is True
- Everything Without a "Value" is False

Datatypes - undefined, null

▪ Undefined

- 변수 선언은 했지만 값이 아직 할당되지 않았을 때, undefined 타입

```
let something;  
console.log(something); // undefined
```

- Accessing a non-existing object property returns undefined.

```
const person = {firstName:"John", lastName:"Doe"};  
document.getElementById("demo").innerHTML = person.age;
```

- A function without a return value returns undefined.

```
function myFunction() {  
  let x = 5;  
}  
document.getElementById("demo").innerHTML = myFunction();
```

▪ null

- null 은 “값이 없음”이나 “의도적으로 비어 있음”을 표현, null 타입

```
let nothing = null;  
console.log(nothing); // null
```

Datatypes - symbol

JavaScript Symbols

- 고유하고 변경 불가능한 값으로, 주로 객체의 고유한 속성 키로 사용
- Symbol() 함수를 사용해 생성하며, 같은 입력값을 넣어도 결과값은 달라짐
- 문자열로 자동 변환되지 않으며 + 연산도 불가능

```
let sym1 = Symbol("id");  
let sym2 = Symbol("id");  
console.log(sym1 === sym2); // false
```


Datatypes - Reference Data Types

- Reference Data Types (=Object Data Types, Non-Primitive Data Types)
 - JavaScript의 참조형(Reference) 데이터 타입은 데이터의 실제 값이 아닌, 그 값이 저장된 메모리 주소(참조값)를 변수에 저장하는 데이터 타입이다.
 - Heap Memory 저장: 크기가 정해지지 않은 복잡한 데이터이기 때문에, 실제 데이터는 메모리의 '힙'이라는 공간에 저장되고, 변수에는 그 위치를 가리키는 '주소'만 복사된다.
 - 가변성(Mutable): 기본형과 달리, 값이 복사되는 것이 아니라 주소가 공유되기 때문에 객체의 내부 프로퍼티를 변경하면 해당 주소를 참조하는 다른 변수의 값도 같이 변경된다

자료형	설명	예시
object	키-값 쌍으로 이루어진 데이터 구조	<code>const user = { name: "홍길동", age: 25 };</code>
array	순서가 있는 값들의 집합	<code>let items = [1, 2, 3];</code>
function	실행 가능한 코드 블록	<code>const greet = function() { ... };</code>
date	날짜와 시간 객체	<code>let today = new Date();</code>
Map	키-값 쌍을 저장하는 컬렉션 (key는 다양한 타입 가능)	<code>const map = new Map();</code>
Set	중복되지 않는 값의 집합	<code>const set = new Set([1, 2, 3]);</code>
Regexp	정규 표현식	<code>const regex = /abc/i;</code>

Datatypes - Wrapper Object

Wrapper Objects

Primitive	Wrapper	Auto-Boxing 예시 코드	특징 및 주의사항 (교재 가이드)
String	String	"hello".toUpperCase() "web".length	가장 빈번하게 자동 변환이 일어나는 래퍼 객체
Number	Number	(123.456).toFixed(2) (10).toString(2)	숫자에 바로 마침표를 찍으면 소수점으로 인식하므로, (10).처럼 괄호 로 감싸야 안전하게 래퍼 객체 메서드가 호출된다.
Boolean	Boolean	true.toString()	true 나 false 상태를 문자열 등으로 변환할 때 자동으로 박싱된다.
Symbol	Symbol	const s = Symbol('id'); s.description	래퍼 객체는 존재하지만, 개발자가 직접 new Symbol() 코드로 인스턴스를 생성하는 것은 금지
BigInt	BigInt	const b = 9007199254740991n; b.toString()	Symbol과 마찬가지로 new BigInt() 형태의 직접 객체 생성은 불가능하다.
null	없음 (예외)	(호출 불가)	래퍼 객체가 존재하지 않으므로 마침표(.)를 찍어 접근 시 무조건 TypeError가 발생한다.
undefined			

- Auto-Boxing: JavaScript 엔진이 편의를 위해 Primitive Type 데이터를 순간적으로 동명의 빌트인 래퍼 객체(Wrapper Object)로 자동 변환해 주는 내부 메커니즘

Operators

- JavaScript Operator(연산자)는 변수나 값을 대상으로 연산을 수행하여 새로운 값을 만들어내는 기호들이다.
 - Arithmetic Operators (산술연산자)
 - Assignment Operators (대입연산자)
 - Comparison Operators (비교연산자)
 - Logical Operators (논리연산자)
 - And more ...

Operators - Arithmetic

JavaScript Arithmetic Operators

연산자	설명	예제	결과
+	덧셈 (Addition)	5 + 3	8
-	뺄셈 (Subtraction)	5 - 3	2
*	곱셈 (Multiplication)	5 * 3	15
/	나눗셈 (Division)	6 / 2	2
%	나머지 (Modulus)	5 % 2	1
**	거듭제곱 (Exponentiation)	2 ** 3	8
++	1 증가 (Increment)	let a = 1; a++; ++a;	a = 2
--	1 감소 (Decrement)	let a = 1; a--; --a;	a = 0

- Demo : https://www.w3schools.com/js/js_arithmetic.asp

Operators - Assignment

JavaScript Assignment Operators

연산자	설명	예제	결과
=	값 할당	let a = 5	a = 5
+=	덧셈 후 할당	a += 2	a = 7
-=	뺄셈 후 할당	a -= 2	a = 3
*=	곱셈 후 할당	a *= 2	a = 10
/=	나눗셈 후 할당	a /= 2	a = 2.5
%=	나머지 후 할당	a %= 2	a = 1

- Demo : https://www.w3schools.com/js/js_assignment.asp

Operators - Comparisons

JavaScript Comparison Operators

연산자	설명	예제	결과
==	[느슨한 비교] 값이 같으면 true	5 == '5'	TRUE
===	[엄격한 비교] 값과 타입이 같으면 true	5 === '5'	FALSE
!=	[느슨한 비교] 값이 다르면 true	5 != '5'	FALSE
!==	[엄격한 비교] 값 또는 타입이 다르면 true	5 !== '5'	TRUE
<, >, <=, >=	크기 비교	5 > 3	TRUE

- Demo : https://www.w3schools.com/js/js_comparisons.asp

Operators - Logical

- JavaScript Logical Operators

연산자	설명	예제	결과
&&	논리 AND	true && false	false
	논리 OR	true false	true
!	논리 NOT	!true	false

- Demo : https://www.w3schools.com/js/js_logical.asp

Operators - Bitwise

JavaScript Bitwise Operators

연산자	설명	예제	상세	결과
&	비트 AND	5 & 1	0101 & 0001	0001 → 1
	비트 OR	5 1	0101 0001	0101 → 5
^	비트 XOR	5 ^ 1	0101 ^ 0001	0100 → 4
~	비트 NOT	~5	~0101	1010 → 10
<<, >>, >>>	비트 시프트	5 >> 1	0101 >> 1	0010 → 2

- Demo : https://www.w3schools.com/js/js_bitwise.asp

Operators - Miscellaneous

JavaScript Other Operators

연산자	설명	예제	결과
삼항(Ternary) 연산자 (? :)	조건부 연산자 (삼항 연산자)	let result = (5 > 3) ? 'Yes' : 'No'	Yes
typeof 연산자	데이터 타입 반환	typeof 42	'number'
in 연산자	객체 속성 존재 여부 확인	("firstName" in person)	true / false
instanceof 연산자	객체의 인스턴스 여부 확인	cars instanceof Array	true / false

Demo

- Ternary Operator : https://www.w3schools.com/js/js_if_ternary.asp
- typeof Operator : https://www.w3schools.com/js/js_typeof.asp
- in Operator : https://www.w3schools.com/jsref/jsref_oper_relational.asp
- instanceof Operator : https://www.w3schools.com/jsref/jsref_oper_instanceof.asp

Control Flow - if

- The JavaScript if Statement

- Use the JavaScript if statement to execute a block of code **when a condition is true**.

```
let age = 16;
let country = "USA";
let text = "You can Not drive!";

if (country == "USA" && age >= 16) {
  text = "You can drive!";
}
```

Control Flow - else

▪ The JavaScript else Statement

- Use the else statement to specify a block of code to be executed **if a condition is false**.

```
if (hour < 18) {  
  greeting = "Good day";  
} else {  
  greeting = "Good evening";  
}
```

▪ The JavaScript else if Statement

- Use the else if statement to specify a new condition if the first is false.

```
if (time < 10) {  
  greeting = "Good morning";  
} else if (time < 20) {  
  greeting = "Good day";  
} else {  
  greeting = "Good evening";  
}
```

Control Flow - switch

▪ Switch Control Flow

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- If there is no match, no code is executed.

```
switch (new Date().getDay()) {  
  case 6:  
    text = "Today is Saturday";  
    break;  
  case 0:  
    text = "Today is Sunday";  
    break;  
  default:  
    text = "Looking forward to the Weekend";  
}
```

▪ The break / default Keyword

- break 키워드는 현재 실행 중인 case 블록을 종료하고 switch 문을 빠져나간다. 만약, break가 없으면 다음 case 블록이 연속적으로 실행된다.
- default는 주어진 값이 어떤 case와도 일치하지 않을 때 실행되는 코드 블록을 명세한다. default는 선택 사항이며 생략 가능하다.

Loops and Iterations

- JavaScript loops repeat executing a block of code a number of times.

Loop Type	Description
for	Iterates over values and expressions
while	Iterates over a condition
do...while	Iterates over a condition
for...in	Iterates over the properties of an Object
for...of	Iterates over array like objects
forEach()	Iterates over each element in an Array

Loops and Iterations - for Loop

- for statement

```
for (exp 1; exp 2; exp 3) {  
  // code block to be executed  
}
```

- exp 1 is executed (one time) **before** the execution of the code block.
- exp 2 defines the **condition** for executing the code block.
- exp 3 is executed (every time) **after** the code block has been executed.

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

```
const arr = ["바나나 ", " 사과 ", " 자몽"];  
for(let I = 0; I < arr.length; i++){  
  console.log(arr[i]);  
}
```

Loops and Iterations - while Loop

▪ while statement

```
while (condition) {  
  // code block to be executed  
}
```

- 조건이 참(true)인 동안 반복

```
let count = 0;  
while (count < 3) {  
  console.log(count);  
  count++;  
}
```

▪ do while statement

```
do {  
  // code block to be executed  
}  
while (condition);
```

- 조건을 검사하기 전에 코드 블록을 한 번 실행한 후, 조건이 참(true)인 동안 반복

```
let num = 0;  
do {  
  console.log(num);  
  num++;  
} while (num < 3);
```

Loops and Iterations - break and continue

▪ break statement

- 특정 조건에서 반복을 중단해야 할 때 사용

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    break; // 반복문 종료  
  }  
  console.log(i);  
}
```

▪ continue statement

- 특정 조건에서 실행 코드를 건너뛰고 싶을 때 사용

```
for (let i = 0; i < 10; i++) {  
  if (i % 2 === 0) {  
    continue; // 짝수일 때는 건너뛴  
  }  
  console.log(i);  
}
```


Loops and Iterations - for...in

- for...in statement

```
for (let key in 객체) {  
    // 실행 코드  
}
```

- 객체의 속성(key)을 반복할 때 사용

```
let obj = { name: '홍길동', age: 28, company: '활빈당' };  
for (let key in obj) {  
    console.log(key, obj[key]);  
}
```

Loops and Iterations - for...of

- for...of statement

```
for (let value of 반복가능한객체) {  
    // 실행 코드  
}
```

- 반복 가능한 객체(배열, 문자열 등)의 엘리먼트 값을 반복

```
let arr = [10, 20, 30];  
for (let value of arr) {  
    console.log(value);  
}
```

Loops and Iterations - Nested Loop

▪ Nested Loop

```
for (let i = 0; i < 3; i++) {           // 외부 반복문
  for (let j = 0; j < 3; j++) {         // 내부 반복문
    console.log(`i: ${i}, j: ${j}`);
  }
}
```

- 반복 횟수 관리: 중첩 반복문은 실행 횟수가 기하급수적으로 늘어날 수 있으므로 조건과 횟수를 신중히 설정
- 가독성 유지: 중첩 깊이가 깊어지면 가독성이 떨어지므로, 가능한 깊이를 줄이거나 함수로 분리하여 관리

Functions

- What is Function and Why use Functions?
 - Functions are **reusable code blocks** designed to perform a particular task.
 - Functions are executed when they are called or invoked.
 - Functions help you to Reuse code (write once, run many times) and organize code into smaller parts
 - Functions help you to Make code easier to read and maintain
- JavaScript Function Syntax

```
function name( p1, p2, ... ) {  
    // code to be executed  
}
```

- A function can be created with the function keyword, a name, parentheses (), and brackets {}.
- To run a function, you call it by using its name followed by parentheses.
- A Function Can Be Used Many Times
- Variables declared within a JavaScript function, become LOCAL to the function.
- Functions can be used as variables, in all types of formulas, assignments, and calculations.

Functions - Invocation

▪ Function Invocation

- The code inside a function will execute when "something" invokes the function
- When a function returns a value, you can store the value in a variable.
- You can call the same function whenever you need it.
- The () operator invokes a function.
- You can call functions from other functions, from events, or from any code block.

▪ Calling vs Referencing a Function

```
function sayHello() {  
  return "Hello World";  
}  
let text = sayHello;
```

- Referencing a Function: sayHello refers to the function itself. It returns the function.
- Calling a Function: sayHello() refers to the function result. It returns the result.

Functions - Parameters

- Parameters allow you to pass (send) values to a function.
 - Parameters(매개 변수) are the names listed in the function definition.
 - Arguments(인수) are the real values passed to, and received by the function.
- Parameter Rules
 - JavaScript function definitions do not specify data types for parameters.
 - JavaScript functions do not perform type checking on the arguments.
 - JavaScript functions do not check the number of arguments received.
→ Accessing a function with an incorrect parameter can return an incorrect answer
- Default Parameter Values
 - The default value is used if no argument is provided.

Functions - Returns

- The return Statement
 - When a function reaches a return statement, the function stops executing.
 - Most functions return the result of a calculation or an operation.
 - The returned value can be used anywhere a value is expected.
 - A function can return any type of value (not only numbers).
 - Code written after a return statement will NOT be executed.
 - If a function does not return a value, the return value will be undefined.
 - You can use return to stop a function early, based on a condition.

Functions - Arguments

- Arguments are the real values passed to, and received by the function.
 - Arguments are assigned to parameters in the order they appear.
 - Arguments can be variables
 - If a function is called with fewer arguments than parameters, the missing values become undefined.
- Call by Value vs. Call by Sharing

구분	Call by Value (기본형 전달)	Call by Sharing (참조형 전달)
복사되는 대상	실제 데이터 값 (예: 10, "Hello")	메모리 주소 값 (예: 0x00412)
함수내부 수정	원본에 영향 없음	원본 객체의 프로퍼티 수정 시 영향 있음

```
function changeValue(x) {
  // 복사본을 수정함
  x = 20;
}

let a = 10;
changeValue(a);

console.log(a); // 10 (원본은 전혀 변하지 않음)
```

```
function changeProperty(obj) {
  // 주소를 타고 들어가 내부 속성을 수정함
  obj.name = "Lee";
}

let user = { name: "Kim" };
changeProperty(user);

console.log(user.name); // "Lee" (원본 객체가 변경됨!)
```


Functions - Function Expressions

- A function expression is a function stored in a variable

```
// Function Declaration (함수선언)
function multiply(a, b) {
  return a * b;
}

// Function Expression (함수표현식)
const multiply = function(a, b) {
  return a * b;
};
```

- Arrow Functions allow a shorter syntax for function expressions.

```
const multiply = (a, b) => a * b;
```

Functions - Function Expressions

▪ Why use Function Expression?

- 함수 선언문은 '함수 Hoisting'이 발생하여 코드 최상단으로 끌어올려져, 선언하기도 전에 호출할 수 있다. 이는 편해 보이지만, 규모가 큰 프로젝트에서는 코드의 실행 흐름을 방해하고 엉뚱한 함수가 호출되는 버그를 유발한다.

```
// 1. 함수 선언문: 선언 전 호출 가능 (혼란 유발)
sayHello(); // "Hello"
function sayHello() { console.log("Hello"); }

// 2. 함수 표현식: 선언 전 호출 불가능 (안전함)
sayHi(); // ReferenceError: Cannot access 'sayHi' before initialization
const sayHi = function() { console.log("Hi"); };
```

- 함수 선언문은 자바스크립트 엔진이 코드를 실행하기 전 구문 분석 단계에서 미리 생성되므로, if문 같은 블록 안에서 유연하게 정의하기 까다롭지만 함수 표현식은 코드가 실행되는 시점(Runtime)에 평가되어 함수를 생성하므로, 상황에 따라 전혀 다른 함수를 변수에 할당할 수 있다.

```
let guestWelcome;
const isVIP = true;
// 조건에 따라 변수에 할당되는 '함수 값'을 동적으로 결정
if (isVIP) {
  guestWelcome = function() { console.log("VIP 전용 라운지로 안내합니다."); };
} else {
  guestWelcome = function() { console.log("일반 대기실로 안내합니다."); };
}
guestWelcome(); // 실행 시점에 결정된 함수가 유연하게 호출됨
```

- 특히, 화살표 함수는 극도로 간결해지는 장점이 있다.

Data Structures - Arrays

- An Array is an object type designed for storing data collections.
- Key characteristics of JavaScript arrays are:
 - **Elements:** An array is a list of values, known as elements.
 - **Ordered:** Array elements are ordered based on their index.
 - **Zero indexed:** The first element is at index 0, the second at index 1, and so on.
 - **Dynamic size:** Arrays can grow or shrink as elements are added or removed.
 - **Heterogeneous:** Arrays can store elements of different data types (numbers, strings, objects and other arrays).

```
const cars = ["Saab", "Volvo", "BMW"];
```

Data Structures - Arrays (Creating and Accessing an Array)

■ Creating an Array

- Using an array literal : []

```
const cars = ["Saab", "Volvo", "BMW"];
```

- Using the JavaScript Keyword new

```
const cars = new Array("Saab", "Volvo", "BMW");
```

■ Accessing and Changing an Array Element

- You access an array element by referring to the index number

```
let car = cars[0];
```

- This statement changes the value of the first element in cars:

```
cars[0] = "Opel";
```

■ The length Property

- The length property of an array returns the length of an array (the number of array elements).

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];  
let length = fruits.length;
```

Data Structures - Arrays (Basic and Search Methods)

Basic Array Methods

- `toString()`, `at()`, `join()`, `pop()`, `push()`, `shift()`, `unshift()`, `isArray()`, `delete()`, `concat()`, `copyWithin()`, `flat()`, `slice()`, `splice()`, `toSpliced()`
- `toString()` - returns the elements of an array as a comma separated string.
- `pop()` - 배열의 맨 뒤 요소를 제거하고, 그 제거된 요소를 반환. 예) `let last = arr.pop();`
- `push()` - 배열의 맨 뒤에 새로운 요소를 추가. 예) `arr.push(4);`
- `shift()` - 배열의 맨 앞 요소를 제거하고, 그 제거된 요소를 반환. 예) `let first = arr.shift();`
- `unshift()` - 배열의 맨 앞에 새로운 요소를 추가. 예) `arr.unshift(0);`
- `at()` - 대괄호 `arr[arr.length - 1]` 대신 음수 인덱스를 써서 배열의 맨 마지막 원소를 가볍게 가져온다. 예) `arr.at(-1);`
- `join()` - 배열의 모든 요소를 지정한 구분자로 연결하여 하나의 문자열로 합친다. 예) `arr.join("-");`

Array Search Methods

- `indexOf()`, `lastIndexOf()`, `includes()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`
- `indexOf()` - 특정 값이 위치한 인덱스 번호를 알려준다. 없으면 -1 반환. 예) `arr.indexOf(5);`
- `lastIndexOf()` - is the same as `Array.indexOf()`, but returns the position of the last occurrence of the specified element.
- `includes()` - 배열에 특정 값이 포함되어 있는지 확인하여 true/false를 반환. 예) `arr.includes(5);`
- `find()` - 조건에 맞는 첫 번째 요소 '딱 하나'만 찾아서 반환. 없으면 undefined. 예) `const user = users.find(u => u.id === 3);`

Data Structures - Arrays (Sorting and Iteration Methods)

▪ Array Sorting Methods

- `sort()`, `reverse()`, `toSorted()`, `toReversed()`
- `sort()` - sorts an array alphabetically:
- `reverse()` - reverses the elements in an array:
- `toSorted()` - sort an array without altering the original array.
- `toReversed()` - reverse an array without altering the original array.

▪ Array Iteration Methods

- `forEach()`, `map()`, `flatMap()`, `filter()`, `reduce()`, `reduceRight()`, `every()`, `some()`, `from()`, `keys()`, `entries()`, `with()`
- `forEach()` - 배열의 모든 요소를 순회하며 내부 함수(callback 함수)를 실행. 반환값 없음 예) `arr.forEach(item => console.log(item));`
- `map()` - 모든 요소를 일정한 규칙으로 가공하여, 새로운 배열로 똑같이 만들어 반환 예) `const doubled = arr.map(item => item * 2);`
- `filter()` - 조건에 맞는 요소들만 골라내어 새로운 배열로 반환 예) `const evens = arr.filter(item => item % 2 === 0);`
- `reduce()` - 배열의 모든 요소를 누적 계산하여 하나의 결과값으로 만든다. 예) `const total = arr.reduce((sum, item) => sum + item, 0);`

▪ Spread Operator(...)

- The ... operator expands an array into individual elements

```
const numbers = [23,55,21,87,56];  
let minValue = Math.min(...numbers);
```

- `Math.min(...numbers)` → `Math.min(23, 55, 21, 87, 56)`으로 변환되어 처리됨.

Datatypes - Objects (Intro)

- What are JavaScript Objects?
 - Objects are containers for Properties and Methods
 - Properties are named Values stored as key:value pairs
 - Methods are Functions stored as key:function() pairs.

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  age      : 50,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

- How to Create a JavaScript Object
 - An **object literal {}** is the simplest and most common way to define a JavaScript object.

```
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

- Create a new JavaScript object using **new Object()**:

```
const person = new Object();  
person.firstName = "John";  
person.lastName = "Doe";  
person.age = 50;  
person.eyeColor = "blue";
```

Datatypes - Objects (Properties)

- You can access object properties in these ways
 - Dot notation (preferred)
 - Bracket notation (Bracket notation is necessary in some cases)
 - Expression
- Add, change, and delete properties using assignment and delete
 - You can change the value of a property
 - You can add a new property by simply giving it a value
 - The delete keyword deletes a property from an object
 - Use the in operator to check if a property exists in an object
- Nested Objects
 - Property values in an object can be other objects

Datatypes - Objects (Methods)

- The this Keyword
 - In an object method, this refers to the object.
- Accessing Object Methods
 - To call an object method, add parentheses ()
- Adding a Method to an Object
 - You can add a method to an object by assigning a function to a property

Datatypes - Objects (Display Objects)

- Displaying a JavaScript object will output [object Object].
- How to Display JavaScript Objects?
 - You can add a method to an object by assigning a function to a property
 - The properties of an object can be collected in a For .. In loop
 - Object.values() creates an array from the property values.
 - Object.entries() makes it simple to use objects in loops.
 - JavaScript objects can be converted to a string with JSON method JSON.stringify().

Datatypes - Objects (Constructors)

▪ Object Constructor Function

- To create an object type we use an object constructor function.

```
function Person(first, last, age, eye) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eye;  
}  
  
const myFather = new Person("John", "Doe", 50, "blue");  
const myMother = new Person("Sally", "Rally", 48, "green");  
const mySister = new Person("Anna", "Rally", 18, "green");
```

▪ Constructor Function **prototype**

- To add a new property, you must add it to the constructor function **prototype**
- Adding a new method to a constructor must be done to the constructor function **prototype**

Datatypes - Build-in Objects

- JavaScript Built-in Objects(내장 객체)는 개발자가 직접 정의하지 않아도 JavaScript 엔진(브라우저나 Node.js 환경)이 실행 단계에서 기본적으로 미리 생성하여 제공하는 전역 객체들이다.
- 핵심 Build-in Objects

객체명	주요 목적	핵심 메서드/속성	인스턴스 생성 여부 (new)
Object	모든 객체의 기본 틀 데이터 생성	Object.keys(), Object.values()	가능 (new Object()), 보통 {} 리터럴 사용
Array	순서가 있는 리스트 데이터 제어	.push(), .map(), .filter()	가능 (new Array()), 보통 [] 리터럴 사용
String	문자열 데이터 조작 및 가공	.length, .slice(), .substring()	가능 (Auto-boxing), 보통 "" 리터럴 사용
Math	수학적 연산 및 수치 처리	Math.random(), Math.floor()	불가능, 정적 메서드로 바로 호출
Date	날짜 및 시간 데이터 제어	.getFullYear(), .getMonth()	필수, 인스턴스 생성 후 사용 (new Date())
JSON	데이터 교환 형식(텍스트) 변환	JSON.stringify(), JSON.parse()	불가능, 정적 메서드로 바로 호출
RegExp	문자열 내 특정 패턴 검색 (정규식)	.test(), .exec()	가능 (new RegExp()), 보통 /패턴/ 리터럴 사용

Datatypes - Build-in Objects (Math)

- The JavaScript Math object allows you to perform mathematical tasks.
- The Math object is static. All methods and properties can be used without creating a Math object first.

```
Math.E      // 자연상수 e (오일러 수) 약 2.718
Math.PI     // 원주율 약 3.14159
Math.SQRT2  // square root of 2 (2의 제곱근) 약 1.414
Math.SQRT1_2 // square root of ½ (½의 제곱근) 약 0.707
Math.LN2    // natural logarithm of 2 (2의 자연로그) 약 0.693
Math.LN10   // natural logarithm of 10 (10의 자연로그) 약 2.302
Math.LOG2E  // base 2 logarithm of E (밑이 2인 로그에서 e의 값) 약 1.442
Math.LOG10E // base 10 logarithm of E (상용로그에서의 e의 값) 약 0.434
```

- Number to Integer

메서드 형태	기능 (한글 정의)
Math.round(x)	반올림
Math.ceil(x)	올림
Math.floor(x)	내림
Math.trunc(x)	소수점 버림

```
Math.round(4.6);
```

- https://www.w3schools.com/js/js_math_reference.asp

Datatypes - Build-in Objects (Date)

- JavaScript Date Objects let us work with dates

```
const d = new Date();  
const d = new Date("2022-03-25");
```

- `new Date()` creates a date object with the current date and time.
 - `new Date(date string)` creates a date object from a date string.
- JavaScript Stores Dates as Milliseconds
 - JavaScript stores dates as number of milliseconds since January 01, 1970.
- Displaying Dates
 - `toString()`
 - `toDateString()`
 - `toUTCString()`
 - `toISOString()`

Datatypes - Build-in Objects (JSON)

- JSON(JavaScript Object Notation) Built-in Object는 JavaScript 객체 형태의 데이터를 '텍스트 문자열'로 상호 변환해 주는 전역 유틸리티 객체이다.
- JSON.stringify(value) : 직렬화 (Serialization)

- 자바스크립트의 객체나 배열을 하나의 긴 JSON 텍스트 문자열로 변환

```
const user = { name: "홍길동", age: 15 };  
const jsonString = JSON.stringify(user);  
  
console.log(jsonString); // '{"name":"홍길동","age":15}' (순수한 문자열 타입)  
console.log(typeof jsonString); // "string"
```

- JSON.parse(text) : 역직렬화 (Deserialization)
- JSON 텍스트 문자열을 다시 자바스크립트 실제 객체/배열로 복원합니다.

```
const receivedData = '{"name":"홍길동","age":15}';  
const obj = JSON.parse(receivedData);  
  
console.log(obj.name); // "홍길동" (실제 객체로 복원되어 접근 가능)  
console.log(typeof obj); // "object"
```

Datatypes - Build-in Objects (RegExp)

- Regular Expression (Regex)
 - A Regular Expression(Regex) is a sequence of characters that forms a search pattern.
 - JavaScript RegExp is an Object for handling Regular Expressions.
 - RegExp are be used for Text searching, Text replacing and Text validation
- Regular Expression Syntax

```
/pattern/modifier flags
```

- Example: /w3schools/i is a regular expression

```
let n = text.search(/w3schools/i);
```

- w3schools is a pattern (to be used in a search).
- i is a modifier (modifies the search to be case-insensitive).

- Phone Number Validation

```
const regex = /[0-9]{3}-[0-9]{4}-[0-9]{4}/;  
const phone = "010-1234-5678";  
console.log(regex.test(phone)); // true
```


Datatypes - Build-in Objects (Set)

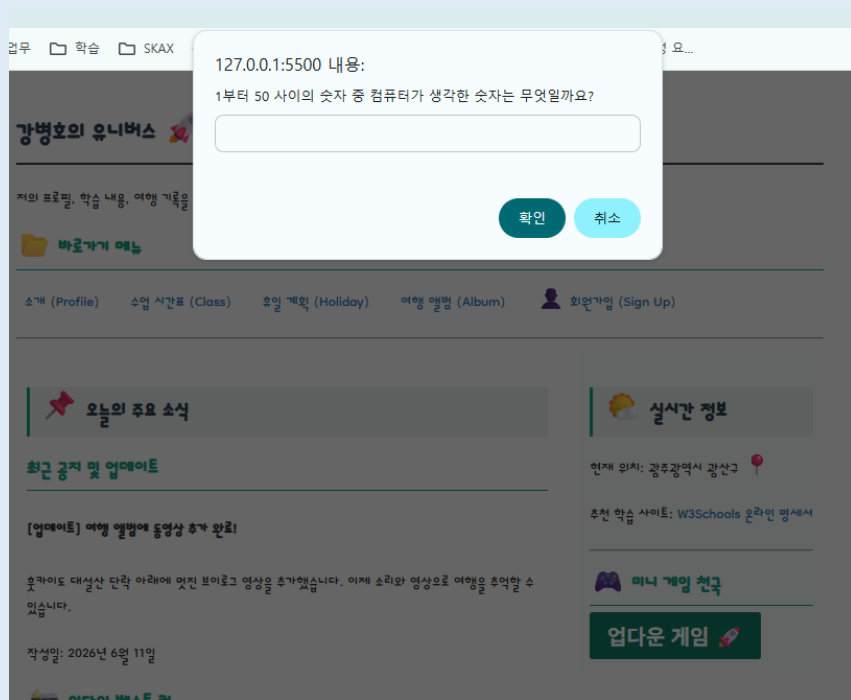
- JavaScript Set
 - A JavaScript Set is a collection of **unique** values.
 - Each value can only occur once in a Set.
- How to Create a Set
 - Passing an array to new Set()
 - Create an empty set and use add() to add values

Datatypes - Build-in Objects (Map)

- JavaScript Map
 - A JavaScript Map is an object that can store collections of **key-value** pairs
- How to Create a Map
 - Create a new Map and add elements with Map.set()
 - Passing an existing Array to the new Map() constructor

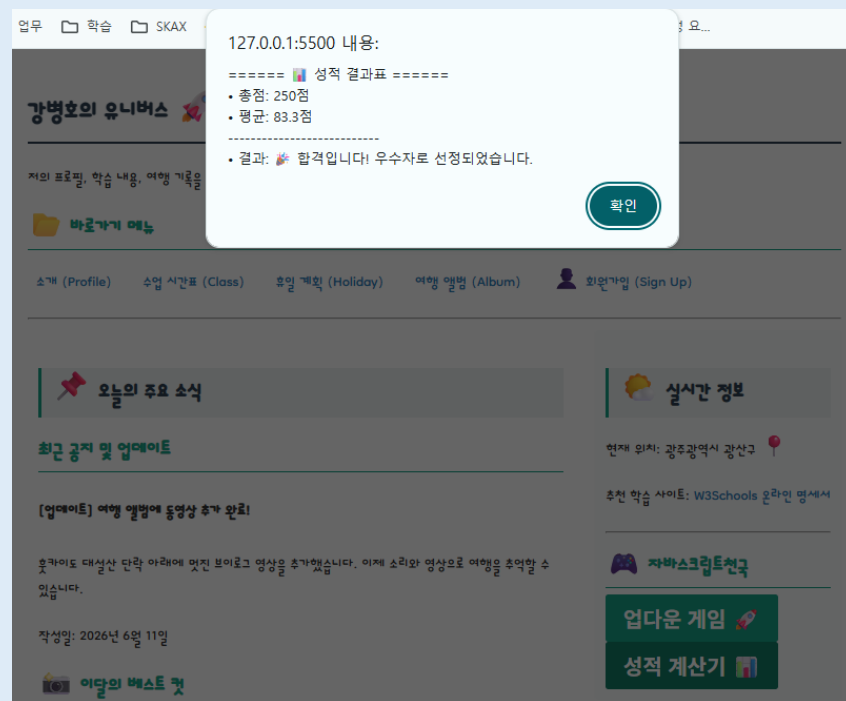
[과제] Up-Down 숫자 맞추기 게임

- 컴퓨터가 생각한 비밀 숫자를 사용자가 맞추는 게임을 만든다. (/script/upDown.js)
 - 컴퓨터가 1부터 50 사이의 무작위 숫자 하나를 생성하게 한다. (`var computerNum = Math.floor(Math.random() * 50) + 1;`)
 - `prompt()` 창을 띄워 사용자에게 숫자를 입력받는다.
 - 사용자가 맞출 때까지 반복해서 기회를 준다. (`while` 또는 `for` 반복문 실습)
 - 사용자가 정답보다 큰 값을 입력하면 `alert("Down!")`, 작은 값을 입력하면 `alert("Up!")`을 띄워준다.
 - 정답을 맞추면 `alert("축하합니다! X번 만에 맞추셨습니다.")`를 띄우고 게임을 종료한다.
- `index.html` 파일의 원하는 위치(예: `<aside>` 영역 안의 새로운 `<section>`)에 게임 시작 버튼 태그를 추가하여 실행시킨다.



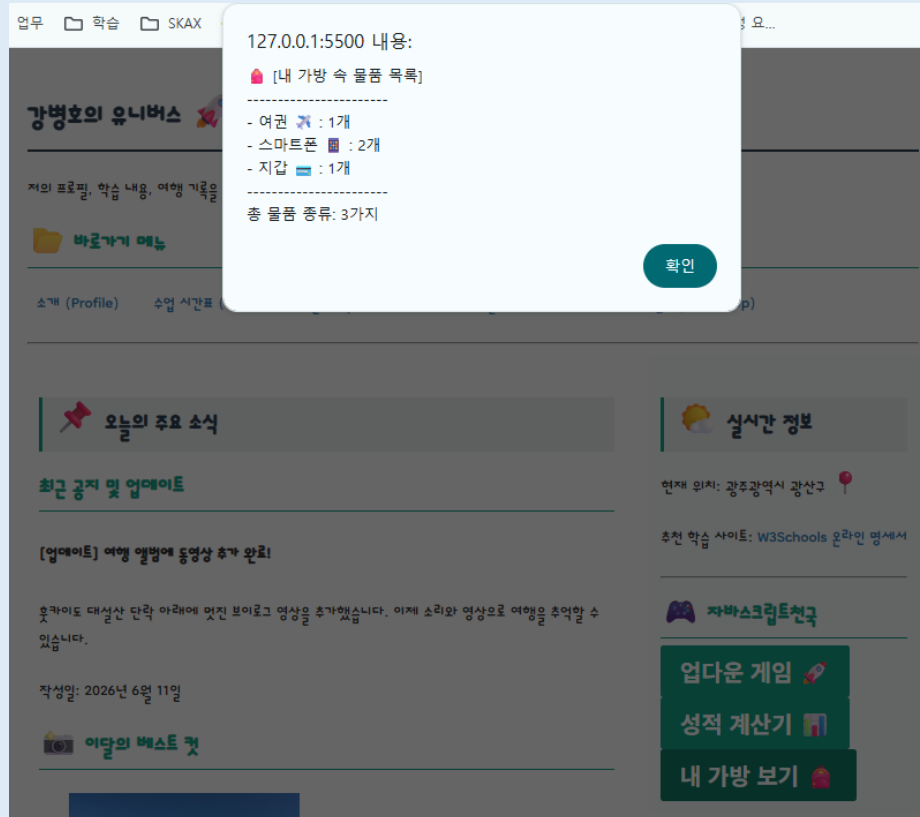
[과제] 성적 계산기

- 강의 3과목의 점수를 연속으로 입력받아 평균을 내고 합격 여부와 등급을 정한다. (/script/grade.js)
 - 과목 이름이 담긴 배열을 미리 만들어 둡니다. `var subjects = ["HTML", "CSS", "JavaScript"];`
 - 총점을 저장할 변수(`var total = 0;`)를 만듭니다.
 - for문을 배열의 길이만큼 돌리면서, 각 과목의 점수를 `prompt()`로 연속해서 입력받아 `total`에 더합니다.
 - 예: `prompt(subjects[i] + " 점수를 입력하세요.");`
 - 반복문이 끝난 후 평균 점수를 구합니다. (평균 점수가 60점 이상이면 합격, 60점 미만이면 불합격)
 - 결과를 브라우저 `alert`창으로 보여줍니다. (`alert("총점: 240점, 평균: 80, 결과: 합격입니다!")`)



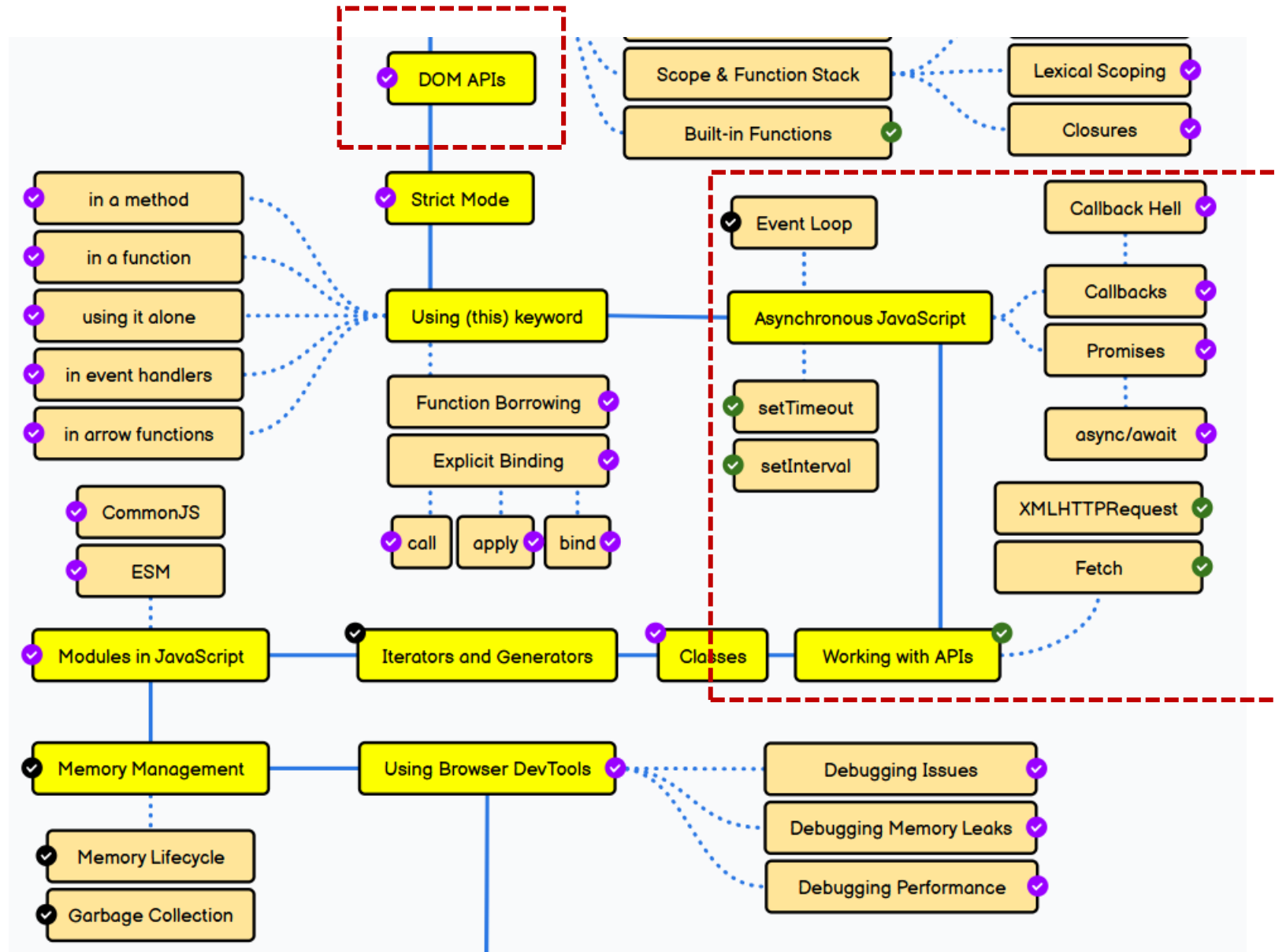
[과제] 내 가방 보기

- 가방 속 물품을 JavaScript Object로 만들고 그 내용을 보여준다.
 - /script/bag.js를 만들고 그 안에 showMyBag()함수를 생성한다.
 - myBag이라는 배열에는 소지품 객체 (소지품 명과, 소지품 수)의 임의 데이터를 만든다.
 - 반복문을 통해 소지품 객체를 출력한다.



Full-Stack Engineering - HTML, CSS, JavaScript

8. JavaScript 심화



DOM APIs - HTML DOM (Document Object Model)

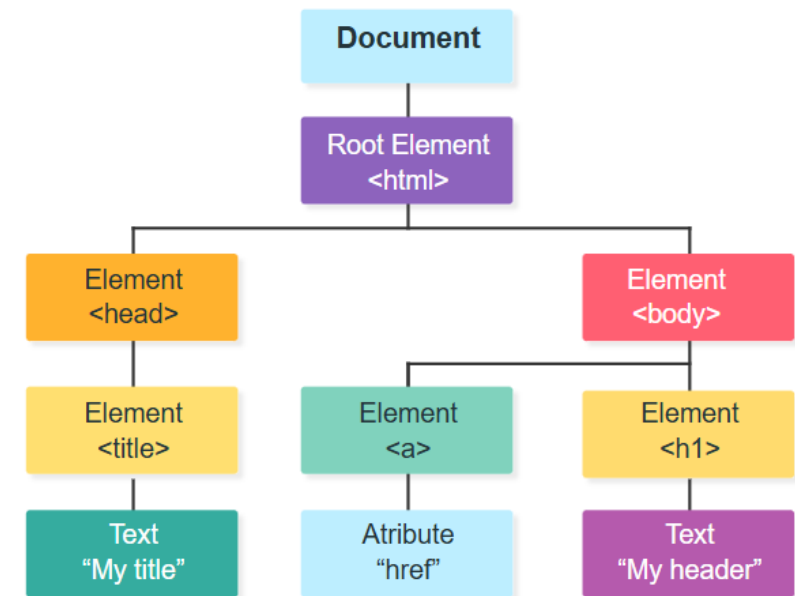
HTML Document Object Model

- 웹 문서(HTML, XML)를 구조화하여 브라우저나 프로그래밍 언어에서 쉽게 접근하고 조작할 수 있도록 하는 표준화된 인터페이스
- DOM은 웹 문서를 트리 구조로 표현하며, 문서의 각 요소를 객체로 간주
- W3C(World Wide Web Consortium)와 WHATWG(Web Hypertext Application Technology Working Group)에 의해 정의된 표준
- 자바스크립트를 통해 DOM 요소를 실시간으로 변경 가능
- 클릭, 입력, 스크롤과 같은 사용자 이벤트 처리 가능

DOM Tree

- When a web page loads, the browser creates a tree-like representation of the HTML document.

Node	Description
Document	Owner of all nodes in the document
<html>	Element Node
<head>	Element Node
<body>	Element Node
<a>	Element Node
href	Attribute Node
<h1>	Element Node
My Header	Text Node



DOM APIs - HTML DOM API

- HTML DOM API (Application Programming Interface)

- The DOM API is a set of Methods and Properties that allow JavaScript to change the content, structure, and style of any HTML elements.

- Example

```
<html>
  <body>
    <p id="demo"></p>

    <script>
      const myPara = document.getElementById("demo");
      myPara.innerHTML = "Hello World!";
    </script>

  </body>
</html>
```

- document is the HTML document.
- getElementById() is a document method.
- myPara = getElementById("demo") retrieves the "demo" element.
- innerHTML is an element property.
- myPara.innerHTML = "Hello World!" changes the property.

DOM APIs - Selecting Elements

- Finding HTML Elements
 - Finding HTML elements by id
 - Finding HTML elements by tag name
 - Finding HTML elements by class name
 - Finding HTML elements by CSS selectors
 - Finding HTML elements by HTML object collections
- Finding HTML Elements by HTML Object Collections

DOM APIs - Changing HTML

- The HTML DOM allows JavaScript to change both the text and the content of HTML elements.
 - **innerHTML** : getting or replacing the content of HTML elements
 - **attribute** : change the value of an HTML attribute
 - **style.property** : change the style of an HTML element
- document.write()
 - document.write() can be used to write directly to the HTML output stream:

DOM APIs - Events

▪ HTML Events

- HTML events are **things that happen to HTML elements**.
- Ex) An HTML button is clicked, A web page has finished loading, The mouse moves over an element, A keyboard key is pressed, An HTML input field is changed

▪ JavaScript Events

- JavaScript lets you execute code when events are detected.

```
<element event="some JavaScript">
```

- Ex)

```
<button onclick="document.getElementById('demo').innerHTML = Date()">The time is?</button>
```

- It is more common to use the event attribute to call a function:

```
<button onclick="displayDate()">The time is?</button>
```

```
<script>
function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>
```

DOM APIs - Event Types

■ 주요 이벤트 목록

이벤트	설명	예시
click	사용자가 요소를 클릭할 때 발생	<button onclick="alert('Button clicked!')">Click Me</button>
dblclick	사용자가 요소를 두 번 클릭할 때 발생	<div ondblclick="alert('Double clicked!')">Double Click Me</div>
mouseover	마우스 포인터가 요소 위에 올려졌을 때 발생	<div onmouseover="alert('Mouse over!')">Hover me</div>
mouseout	마우스 포인터가 요소에서 벗어났을 때 발생	<div onmouseout="alert('Mouse out!')">Mouse Out Me</div>
keydown	키보드의 키가 눌릴 때 발생	<input type="text" onkeydown="alert('Key down!')">
keyup	키보드에서 키가 떼어질 때 발생	<input type="text" onkeyup="alert('Key up!')">
change	입력 값이나 선택 값이 변경될 때 발생	<input type="text" onchange="alert('Value changed!')">
focus	요소가 포커스를 받았을 때 발생	<input type="text" onfocus="alert('Focused!')">
blur	요소의 포커스가 벗어났을 때 발생	<input type="text" onblur="alert('Blurred!')">
load	페이지나 이미지가 완전히 로드된 후 발생	
unload	페이지가 언로드될 때 발생	<body onunload="alert('Page unloaded!')">

DOM APIs - Event Handler

- 자바스크립트에서 이벤트를 다루는 방식(등록하는 방법)은 크게 3가지 방법이 있다

방식	설명	비고
Inline Event Handler (Not Recommended)	HTML 태그 내부에 직접 이벤트 속성(onclick, onmouseover 등)을 사용하여 JS 함수를 연결 처리	- JS와 HTML의 역할 분리 원칙 위배 - 복잡한 로직 작성에 부적합
DOM Property (Not Recommended)	JavaScript에서 DOM 요소의 이벤트 속성(예: onclick)에 함수를 할당하여 이벤트를 처리	- 이벤트 핸들러 1개만 등록 가능 - 다른 코드에서 덮어쓸 수 있어 충돌 위험
Event Listener (Highly Recommended)	addEventListener() 메서드를 사용하여 이벤트 핸들러를 등록	- 복수 핸들러 등록 가능 - 버블링/캡처 제어 가능 - 유지보수 및 확장성 우수

[Inline Event Handler - 쉽지만 비권장]

```
<button onclick="displayDate()">Time is?</button>
<p id="demo"></p>

<script>
// Function to display date
function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>
```

[Event Listener - 어렵지만 권장]

```
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
const btn = document.getElementById("myBtn");

// Add EventListener to btn
btn.addEventListener("click", function () {
  document.getElementById("demo").innerHTML = Date();
});
</script>
```

DOM APIs - Event Example

- Mouse Events : https://www.w3schools.com/js/js_events_mouse.asp
- Keyboard Events : https://www.w3schools.com/js/js_events_keyboard.asp
- Load Events : https://www.w3schools.com/js/js_events_load.asp
- Timing Events : https://www.w3schools.com/js/js_events_timing.asp
- 기타 Events : https://www.w3schools.com/js/js_htmlDOM_events.asp

DOM APIs - Event Listener

- `addEventListener()`
 - 특정 HTML 요소에 특정 event가 발생했을 때 실행할 함수(handler)를 연결(등록)한다.
 - Syntax: `element.addEventListener(event, function, useCapture);`
 - event : type of the event (like "click" or "mousedown" or any other HTML DOM Event.)
 - function : we want to call when the event occurs.
 - useCapture : boolean value specifying whether to use event bubbling (false) or event capturing (true). false가 default
- 한 요소에 동일한 이벤트 타입으로 여러 개의 Handler 함수를 중복 등록할 수 있다.
- 한 요소에 다양한 이벤트 타입으로 여러 개의 Handler 함수를 중복 등록할 수 있다.

DOM APIs - Event Propagation

- Event Propagation (이벤트 전파)
 - Event Propagation은 이벤트가 발생했을 때 요소의 처리 순서를 정의하는 방식이다.
 - 만약, <div>안에 <p>가 있고, 사용자가 <p> element를 클릭했을 때, 어느 element의 “click” 이벤트가 먼저 처리되어야 하는가?
 - Event Bubbling - 가장 안쪽 요소의 이벤트가 먼저 처리되고 그 다음 바깥쪽 요소의 이벤트가 처리된다. 즉 <p>가 먼저 처리
 - Event Capturing - 가장 바깥쪽 요소의 이벤트가 먼저 처리되고 그 다음 안쪽 요소의 이벤트가 처리된다. 즉, <div>가 먼저 처리
- Example: https://www.w3schools.com/js/tryit.asp?filename=tryjs_addeventlistener_usecapture
- Summary

구분	이벤트 버블링 (Bubbling)	이벤트 캡처링 (Capturing)
전파 방향	하위 → 상위 (자식에서 부모로)	상위 → 하위 (부모에서 자식으로)
API 구현 방법	<code>element.addEventListener('click', fn)</code> (3 번째 매개변수 생략 또는 <code>false</code>)	<code>element.addEventListener('click', fn, true)</code> (3 번째 매개변수에 <code>true</code> 또는 <code>{ capture: true }</code> 전달)
실무 활용도	매우 높음 (이벤트 위임 패턴의 핵심)	낮음 (특수한 로그 수집, 이벤트 가로채기 등에만 사용)

DOM APIs - Event Management

- JavaScript Event Listener를 등록, 제거, 차단/제어하는 함수

분류	메서드	핵심 역할	주의사항
Add	<code>addEventListener(type, handler)</code>	특정 HTML 요소에 특정 이벤트(type)가 발생했을 때 실행할 함수(handler)를 연결(등록)한다.	한 요소에 동일한 이벤트 타입으로 여러 개의 핸들러 함수를 중복 등록할 수 있다.
Remove	<code>removeEventListener(type, handler)</code>	이전에 등록했던 특정 이벤트 리스너를 요소를 대상으로 제거(삭제)한다.	등록할 때 사용한 함수와 정확히 동일한 참조를 가진 함수(이름이 있는 함수)를 인수로 전달해야만 제거된다.
Block	<code>event.stopPropagation()</code>	이벤트가 발생했을 때 부모 요소로 타고 올라가는 이벤트 버블링(전파)을 즉시 중단시킨다	내 요소의 이벤트는 정상 실행되지만, 부모 요소에 걸려 있는 동일 이벤트는 실행되지 않도록 신호를 끊어버린다.
Block	<code>event.stopImmediatePropagation()</code>	현재 요소에 걸려 있는 다른 이벤트 리스너들의 실행까지 완전히 막고, 전파도 즉시 차단한다	하나의 버튼에 3개의 클릭 이벤트를 등록했을 때, 첫 번째 이벤트에서 이 메서드를 쓰면 뒤에 올 2, 3번째 이벤트는 실행되지 않는다.
Block	<code>event.preventDefault()</code>	HTML 태그가 기본적으로 가지고 있는 고유의 기본 동작을 취소(차단)한다.	<code><a></code> 태그를 클릭할 때 이동하거나, <code><form></code> 태그의 Submit 버튼을 눌렀을 때 페이지가 새로 고침되는 기본 브라우저 행동을 막아준다.

- Code Example

- `event.preventDefault()` : https://www.w3schools.com/js/tryit.asp?filename=tryjs_events_block

Asynchronous JavaScript

Asynchronous(비동기) Code

- JavaScript Asynchronous Code는 특정 작업(예: 서버에서 데이터 가져오기, 타이머 기다리기)이 끝날 때까지 프로그램이 멈춰서 기다리지 않고, 다음 코드를 먼저 실행하는 핵심 효율화 메커니즘이다.
- JavaScript는 Single Thread언어이므로 만약 비동기가 없다면 서버에서 이미지를 다운로드 받는 동안 전체 웹페이지가 멈추게 되는데 비동기는 이러한 문제를 해결할 수 있다.

JavaScript Asynchronous 비동기 처리의 발전

발전단계	제어 방식	코드 구조 특징	장점/단점
1세대	Callback	함수 안에 함수를 계속 넣음	단점: 코드가 오른쪽으로 엄청나게 깊어지는 '콜백 지옥(Callback Hell)'이 발생하여 가독성이 최악이 되고 에러 처리가 매우 힘들다.
2세대 (ES6)	Promise	.then()과 .catch()로 연결	장점: 비동기 작업의 상태를 객체로 구조화하여 콜백 지옥을 해결 단점: 코드가 길어지면 .then() 체인이 복잡해지는 단점이 여전히 존재
3세대 (ES8)	async / await	await 키워드로 동기식처럼 작성	현대 표준: Promise 기반 위에 얹은 문법적 설탕(Syntactic Sugar)입니다. 장점: 비동기 코드를 마치 일반 동기식 코드처럼 직관적으로 읽히게 만들어 준다.

Asynchronous JavaScript - Promise

■ JavaScript Promise Object

- JavaScript 엔진에서 비동기 연산의 최종 완료 또는 실패와 그 결과 가치를 나타내는 표준 객체(Object)이다.
- Promise는 비동기 작업이 성공하거나 실패할 때 그 결과를 알려주겠다고 “약속”하는 객체

■ Promise의 3가지 상태(State)

Pending (대기)	The initial state; the operation has started but is neither fulfilled nor rejected.
Rejected (거부/실패)	The operation failed, and a reason (error) is available
Fulfilled (이행/성공)	The operation completed successfully, and a value is available.

Asynchronous JavaScript - Creating a Promise

Creating a Promise

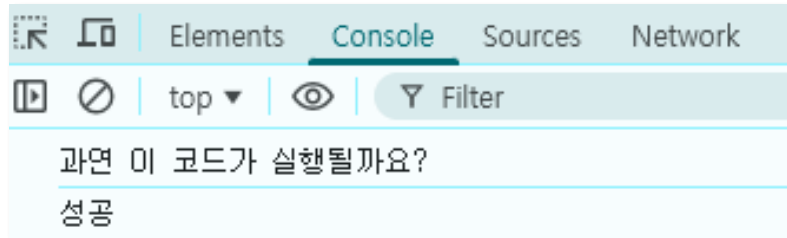
```
let myPromise = new Promise(function(resolve, reject) {
  // Code that may take some time
  resolve(value); // when successful
  reject(value);  // when error
});
```

- Promise를 만들 때 (resolve, reject) => { ... } 라는 함수를 인자로 넣는다. 여기서 resolve와 reject는 자바스크립트 엔진이 제공하는 비동기 상태 변경용 함수이다.

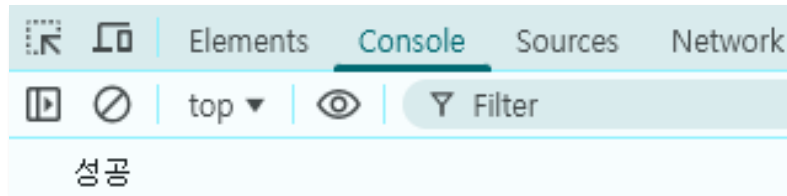
resolve	"이 비동기 작업 성공했어! 이 value 가지고 .then()으로 가!"
reject	"이 작업 실패했어! 이 error 가지고 .catch()로 가!"

- resolve와 reject는 비동기 결과값만 저장해 둘 뿐, 함수의 실행 흐름을 끊지 않는다.

```
const myPromise = new Promise((resolve, reject) => {
  resolve("성공");
  console.log("과연 이 코드가 실행될까요?");
});
myPromise.then(res => console.log(res));
```



```
const myPromise = new Promise((resolve, reject) => {
  return resolve("성공");
  console.log("과연 이 코드가 실행될까요?");
});
myPromise.then(res => console.log(res));
```



Asynchronous JavaScript - Promise Consuming Code

- A Promise contains both the producing code and calls to the consuming code

```
let myPromise = new Promise(function(resolve, reject) {  
  // "Producing Code" (May take some time)  
  resolve(value); // when successful  
  reject(value);  // when error  
});  
  
// Consuming Code  
myPromise  
  .then(function(value) {  
    console.log(value);  
  })  
  .catch(function(value) {  
    console.log(value);  
  });
```

- Promise의 초기 문법에서는 .then()의 첫 번째 인자는 성공했을 때, 두 번째 인자는 실패했을 때 실행할 콜백 함수를 받도록 설계되어 있지만, 현재는 위 코드 처럼 .then().catch() 체이닝 방식의 코드를 권장한다.

Asynchronous JavaScript - Promise Chains

- 만약 3개의 비동기 함수가 순서대로 호출되어야 하면, .then() 안에 return을 넣고 chain을 만든다.

```
// Three functions to run in steps
function step1() {
  return Promise.resolve("A");
}
function step2(value) {
  return Promise.resolve(value + "B");
}
function step3(value) {
  return Promise.resolve(value + "C");
}

// Run the three functions in steps
step1()
  .then(function(value) {
    return step2(value);
  })
  .then(function(value) {
    return step3(value);
  })
  .then(function(value) {
    myDisplayer(value);
  });
```

- 만약, "return"이 빠지면 실행순서가 보장되지 않는다.
- return 대신 .then() 안에 또.then()을 넣는 식으로 코드를 짜면 가독성이 떨어지고, 에러 처리가 까다롭다.

Asynchronous JavaScript - async and await

- Promise chains can become long. **async and await were created to reduce nesting and improve readability.**

```
<script>
function myDisplayer(some) {
  document.getElementById("demo").innerHTML = some;
}

// 3개 비동기 함수는 순서대로 실행되어야 함.
function step1() {
  return Promise.resolve("A");
}
function step2(value) {
  return Promise.resolve(value + "B");
}
function step3(value) {
  return Promise.resolve(value + "C");
}

// 체이닝으로 순서대로 실행
step1()
  .then(function(value) {
    return step2(value);
  })
  .then(function(value) {
    return step3(value);
  })
  .then(function(value) {
    myDisplayer(value);
  });
</script>
```

```
<script>
function myDisplayer(some) {
  document.getElementById("demo").innerHTML = some;
}

// 3개 비동기 함수는 순서대로 실행되어야 함.
function step1() {
  return Promise.resolve("A");
}
function step2(value) {
  return Promise.resolve(value + "B");
}
function step3(value) {
  return Promise.resolve(value + "C");
}

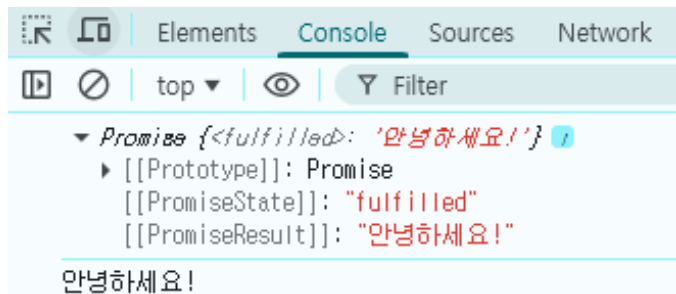
// async와 await를 이용해 구현
async function run() {
  let v1 = await step1();
  let v2 = await step2(v1);
  let v3 = await step3(v2);
  myDisplayer(v3);
}
</script>
```


Asynchronous JavaScript - async and await (syntax)

■ Async

- 이 함수 안에서 비동기 처리를 할 거야!"라고 선언하는 키워드
- async 함수는 항상 Promise를 반환한다.
- 만약 일반 값을 리턴하면 자바스크립트가 자동으로 Promise.resolve(값)으로 감싸서 내보낸다.

```
async function hello() {  
  return "안녕하세요!";  
}  
console.log(hello());  
hello().then(res => console.log(res));
```



■ Await

- "비동기 작업이 끝날 때까지 다음 줄로 넘어가지 말고 기다려!"라고 명령하는 키워드
- await는 반드시 async 함수 안에서만 사용 가능하다.

```
async function handleData() {  
  try {  
    const result = await fetchData(); // 첫 번째 비동기 기다림  
    const saveResult = await saveToDatabase(result); // 두 번째 비동기 기다림  
    console.log("저장 성공!", saveResult);  
  } catch (error) {  
    console.log("에러 발생!", error);  
  }  
}
```

Asynchronous JavaScript - async and await (Flow Control)

- 비동기 호출 시 JavaScript 엔진 내부의 흐름 제어는 Event Loop와 Microtask Queue를 기반으로 제어된다.
- Step 1: await를 만나 함수가 일시 정지된 순간
 - await 라인을 만나는 순간 await 뒤 비동기 함수는 즉시 실행시키고, 해당 async 함수의 실행을 일시 정지(Pause) 시킨다.
 - 그리고 이 함수 내부의 나머지 코드와 함수의 현재 상태(컨텍스트)를 통째로 뜯어내어 메모리의 별도 대기 구역에 보관한다.
 - 제어권은 이 async 함수를 호출했던 원래의 메인 실행 흐름(Call Stack)으로 반환되어 그 아래에 남아있던 동기식 코드들을 실행한다.
- Step 2: 메인 흐름이 실행되는 동안 비동기(await) 작업이 완료된 순간 (★ 핵심)
 - 메인 흐름의 남은 동기식 코드가 열심히 실행되는 도중에, await하고 있던 비동기 작업(예: 서버 통신 완료)이 먼저 끝날 수 있다.
 - 이때 완료되었다고 해서 실행 중이던 메인 흐름을 강제로 찢고 중간에 끼어들지 못한다. 자바스크립트는 Single Thread 언어이기 때문에 한 번에 하나의 코드만 실행할 수 있기 때문이다.
 - 비동기 작업이 완료되면, 대기실에 있던 async 함수의 나머지 실행 후속 조치들이 자바스크립트 엔진 내부의 최우선 대기 레일인 Microtask Queue라는 대기줄에 등록되어 순서를 기다린다.
- Step 3: 메인 흐름이 완전히 종료된 직후 (흐름의 복귀)
 - 메인 실행 흐름에 있던 마지막 동기식 코드까지 한 줄도 남김없이 전부 실행되어 호출 스택(Call Stack)이 완전히 텅 비게 되는 순간, 자바스크립트의 감시자인 Event Loop가 가동한다.
 - Event Loop는 Microtask Queue에서 대기 중이던 async 함수의 나머지 코드를 끄집어내어 다시 메인 호출 스택에 올려놓는다.
 - 흐름은 다시 async 함수 내부의 await 바로 다음 줄로 복귀하여, 서버에서 받아온 알맹이 데이터를 가지고 남은 하부 로직을 순차적으로 끝까지 실행한다.

Working with APIs - Browser API

- Web API vs Browser API
 - Web API : A Web API is an application programming interface for the Web. 브라우저 환경 뿐만 아니라, 서버나 스마트폰 앱 등 '웹 기술'을 쓰는 모든 곳에서 공통으로 지원하는 확장성을 담은 단어이다.
 - Web API는 실행되는 환경과 주체에 따라 Browser API와 Server API로 나뉜다.
 - **Browser API** : 크롬, 사파리, 엣지 같은 웹 브라우저가 구동되면서 자바스크립트 엔진에게 상속해 주는 내장 도구들을 집중해서 부를 때 쓰는 표현.
- Browser API의 종류

카테고리	대표적인 함수 / 객체 (Web API)	하는 일
DOM API	document.querySelector() element.addEventListener()	앞서 배운 HTML 태그 조작 및 클릭 이벤트 제어
Timer API	setTimeout(함수, 시간) setInterval(함수, 시간)	특정 시간 뒤에 코드를 실행하거나, 일정 시간마다 반복 실행 (비동기)
Storage API	localStorage.setItem() sessionStorage.getItem()	사용자의 브라우저 하드디스크에 로그인 토큰이나 설정 데이터를 영구 저장
Network API	fetch()	브라우저 새로고침 없이 서버와 데이터를 주고받는 통신 기능 (비동기)

Working with APIs - Timer API

■ Timer API

- Timer API는 특정 시간이 지난 후에 코드를 실행하거나 일정 시간마다 코드를 반복해서 실행하도록 예약하는 기능이다.
- JavaScript 엔진 자체는 타이머 기능이 없다. JavaScript가 Browser에게 부탁하면, 브라우저가 백그라운드에서 시간을 재고 있다가 정해진 시간에 함수를 실행해 주는 원리이다.
- 타이머 함수를 실행하면 브라우저는 해당 타이머의 고유 번호(Timer ID)를 반환한다. 이 번호를 잘 보관하고 있다가, 더 이상 타이머가 필요 없을 때 취소하는 메서드에 넘겨주어야 메모리 낭비나 버그를 막을 수 있다.

■ Timer API 핵심 함수

함수 이름	핵심 역할	코드 작동 방식	실무 활용 사례
setTimeout()	지정한 시간이 지난 후, 딱 1번만 함수를 실행	setTimeout(콜백함수, 밀리초); (1초 = 1000밀리초)	• 3초 뒤에 자동으로 닫히는 안내 배너 • 사용자가 입력을 멈추고 0.5초 뒤에 검색 API 호출
setInterval()	지정한 시간 간격마다, 무한 반복 하여 함수를 실행	setInterval(콜백함수, 밀리초);	• 화면에 매초 업데이트되는 디지털시계 • 5초마다 자동으로 넘어가는 이미지 슬라이드

■ Timer API Example

```
const bombId = setTimeout(function() {
  console.log("💣 폭탄이 터졌습니다!");
}, 5000);                                     // 5초 뒤에 폭탄이 터지는 타이머 예약 (ID를 변수에 저장)

button.addEventListener("click", function() {
  clearTimeout(bombId);                       // ⏸ 5초가 지나기 전에 실행하면 폭탄은 영원히 터지지 않음
  console.log("안전하게 해제되었습니다.");
});
```

Working with APIs - Storage API

▪ Web Storage

- Web Storage는 사용자의 브라우저에 데이터를 키-값(Key-Value) 쌍으로 저장할 수 있는 공간이다.
- 과거에는 웹 브라우저에 데이터를 저장할 때 'Cookie'라는 기술을 주로 썼으나, 용량이 너무 작고 서버와 통신할 때마다 불필요하게 계속 전송되는 단점이 있다.

구분	LocalStorage	SessionStorage
데이터 유지	영구적 (브라우저를 닫거나 컴퓨터를 꺼도 유지)	임시적 (탭이나 브라우저 창을 닫으면 즉시 삭제)
만료 조건	개발자가 코드로 지우거나, 사용자가 브라우저 캐시를 날리면 삭제	현재 열려 있는 탭(세션)이 종료되는 순간 자동으로 삭제됨
데이터 공유	같은 도메인이라면 여러 창/탭에서 서로 데이터 공유 가능	데이터를 저장한 현재 그 탭 내부에서만 접근 가능
실무 활용처	• 자동 로그인용 토큰 보관 • "오늘 하루 이 창을 열지 않음" 상태 저장 • 다크 모드/라이트 모드 설정 테마 유지	• 사이트의 일시적인 로그인 정보 • 일회성 입력 폼(예: 여러 페이지에 걸친 설문조사 입력 데이터)

▪ LocalStorage Example

```
localStorage.setItem("username", "홍길동"); // 1. 데이터 저장하기: setItem(Key, Value)
localStorage.setItem("userAge", "25"); // ※ 주의: 숫자를 넣어도 문자열 "25"로 저장됨

const name = localStorage.getItem("username"); // 2. 데이터 가져오기: getItem(Key)
console.log(name); // 출력: 홍길동

localStorage.removeItem("userAge"); // userAge 데이터만 삭제됨

localStorage.clear(); // 로컬스토리지 안의 모든 데이터가 소멸함
```

Working with APIs - Network API

▪ fetch()

- fetch() is the modern way to request data from a server
- fetch() is asynchronous and returns a promise.

```
fetch("data.json")
  .then(function(response) {
    console.log(response);
  });
```

▪ Fetch with async and await

- Await는 단순히 기다리는 것을 넘어, Promise 객체의 내부 메커니즘과 상호작용하며 결과물을 처리해 주는 특별한 기능들을 가지고 있다.
- await fetch()를 하면, await는 fetch를 통해 서버와 통신한 결과를 가져와 response로 만들어 준다.

```
async function loadData() {
  let response = await fetch("data.json");
  let data = await response.json();
  console.log(data);
}

loadData();
```

Modules in JavaScript

▪ What are Modules?

- ES6부터 도입(modern JavaScript)된 모듈은 다른 파일에서 함수, 객체, 변수 등을 가져오거나 내보낼 수 있게 함
- 코드의 재사용과 관리를 쉽게 하기 위한 방식으로 각 파일이나 코드 덩어리를 독립적인 유닛으로 분리

▪ Modules Export

- 모듈 외부에서 사용할 변수, 함수, 클래스, 객체를 정의할 때 사용

```
// Export an "add" function
export function add(a, b) {
  return a + b;
}
```

▪ Modules Import

- 다른 모듈에서 내보낸 요소를 가져와서 사용할 때 사용

```
<script type="module">
// Import the add function
import { add } from './math.js';
let result = add(2, 3);
</script>
```

Modules in JavaScript - Export & Import

▪ Export 방식

방식	내보내는 방법	장점	단점
Named Exports	<pre>export const foo = ...; export function bar() {} import { foo, bar } from './module.js';</pre>	- 여러 개 내보내기 가능 - 이름 기준 명확한 가져오기	- 이름을 정확히 기억해야 함 - 이름 변경 시 수정 필요
Default Exports	<pre>export default function() {...} export default class {...} const value = ...; export default value; import anyName from './module.js';</pre>	- 이름을 자유롭게 지정 가능 - 간결한 단일 내보내기	- 한 모듈에 하나만 가능 - 실제 이름 알기 어려움
Combining Default + Named	<pre>export const foo = ...; export default bar; import bar, { foo } from './module.js';</pre>	- 주요 기능은 default로, 보조 기능은 named로 구성	- 가독성이 떨어질 수 있음 - 일관성 관리 어려움

Modules in JavaScript - Module vs. External JS

- Module 과 External JS의 비교
 - External JavaScript는 코드를 단순히 분리한 것에 불과하다면, JavaScript Module은 각 파일의 경계를 안전하게 격리하고 필요한 기능만 통로(import/export)로 주고받는 현대적 설계 체계이다.

구분	External JS (일반 외부 파일)	JavaScript Module (모듈 파일)
HTML 로드 형태	<script src="file.js"></script>	<script type="module" src="file.js"></script>
Scope	전역 스코프 공유 (모든 파일이 소통함)	모듈 스코프 격리 (독립된 공간)
변수 충돌 위험	매우 높음 (이름이 같으면 버그 발생)	없음 (완벽한 은닉화)
소통 방식	전역 변수나 전역 함수를 통해 간접 소통	import와 export를 통해 명확히 소통
실행 시점	태그를 만나는 순간 즉시 실행 (HTML 중단)	HTML을 끝까지 다 읽은 후 지연 실행 (defer)

[변수 충돌 위험 - 예]

```
// external-a.js
const count = 10;

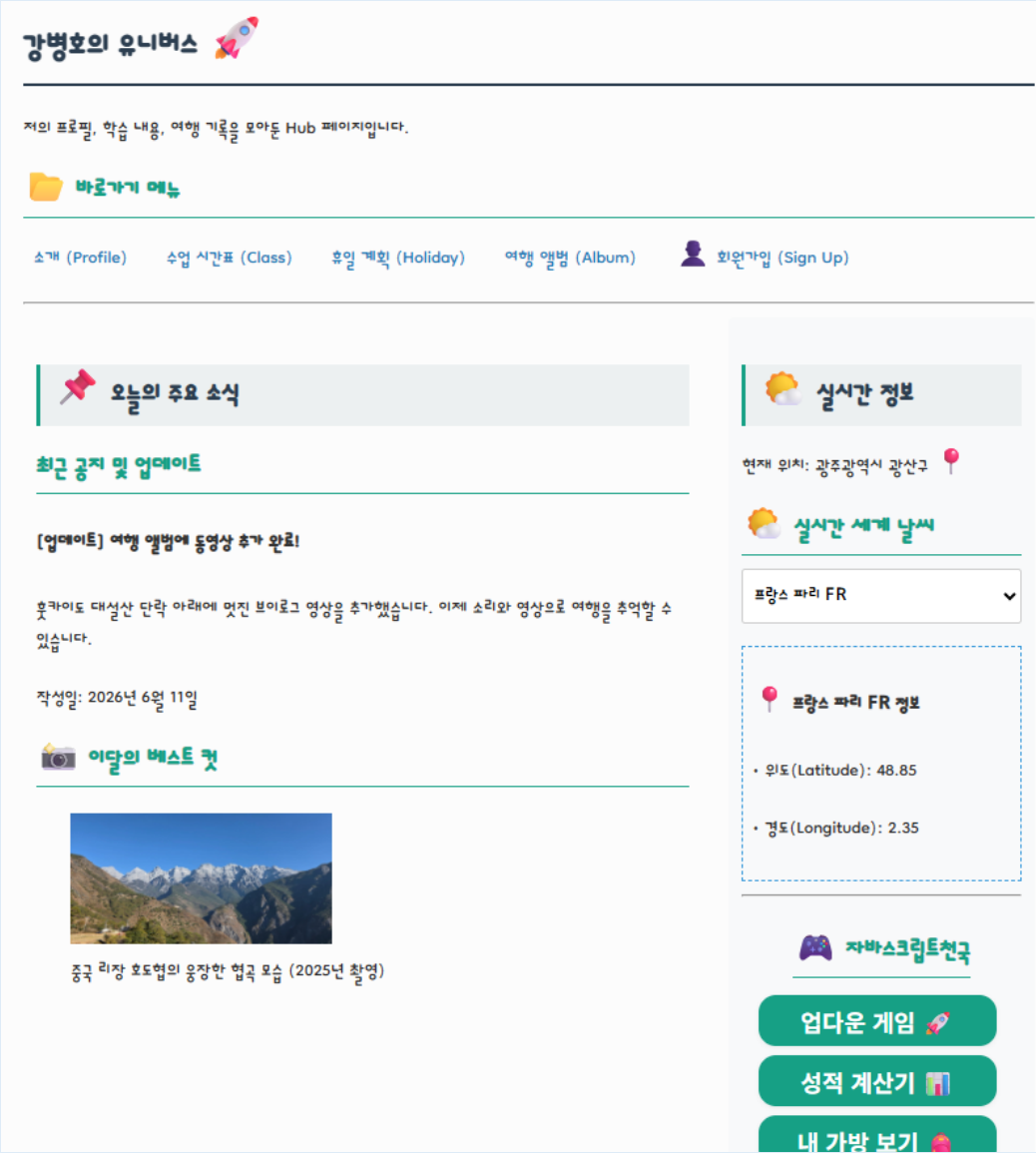
// external-b.js
const count = 20; // ✕ 에러 발생!
```

```
// module-a.js
const count = 10; // 이 파일 안에서만 살아있음

// module-b.js
const count = 20; // 아무 문제 없음!
```

[과제] 실시간 날씨 - DOM/이벤트

- 가방 속 물품을 JavaScript Object로 만들고 그 내용을 보여준다.
 - index.html: 사이드바에 도시를 고를 수 있는 <select> 태그와 결과를 보여줄 <div id="weather-box">를 만드시오
 - weather.js: 사용자가 도시를 바꿀 때마다(change 이벤트), 선택된 도시의 이름과 위도/경도 좌표를 DOM 조작(innerHTML)을 통해 화면에 실시간으로 표시하시요. (아직 날씨는 안 나옵니다!)



[과제] 실시간 날씨 - 비동기 호출

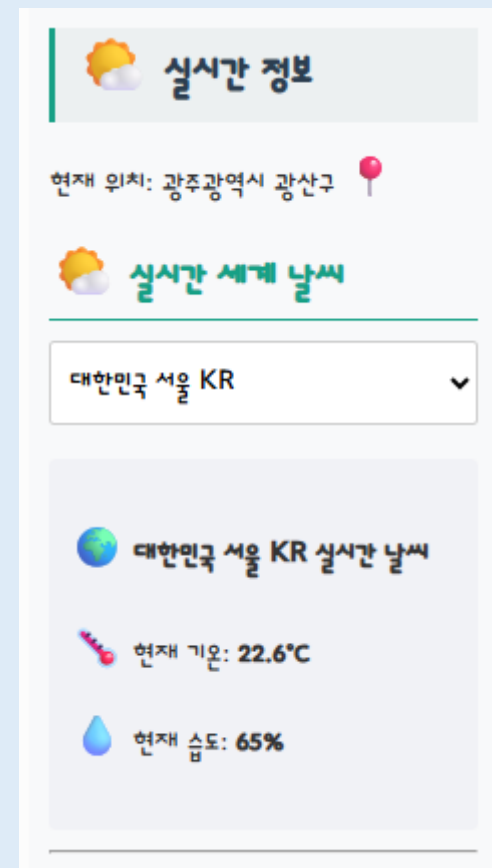
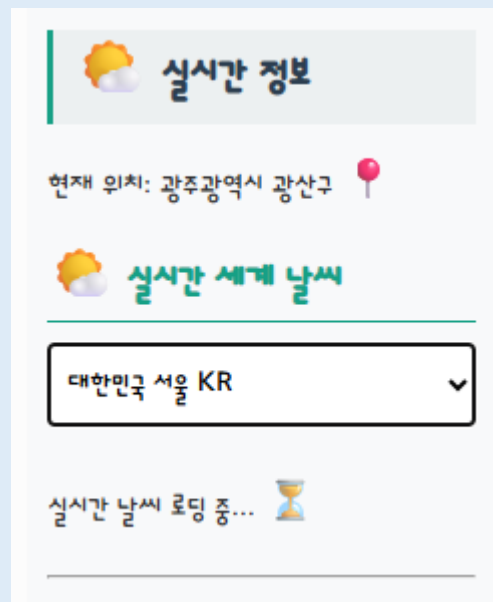
- 실시간 날씨 API를 통해 날씨데이터를 비동기로 가져온다.
 - weather.js: 도시가 변경되었을 때, 단순 좌표 출력에 그치지 않고 fetch()와 async/await 문법을 사용해 Open-Meteo 서버에 날씨 데이터를 요청한다.
 - 데이터를 받아오는 동안 화면에 "로딩 중... ⌚" 메시지를 띄우고, 다운로드가 완료되면 진짜 실시간 온도와 습도를 화면에 그린다.
- Open-Metro 무료 API
 - <https://open-meteo.com/en/docs>

Current Weather

- | | |
|---|--|
| ☒ Temperature (2 m) | ☐ Precipitation |
| ☒ Relative Humidity (2 m) | ☐ Rain |
| ☐ Apparent Temperature | ☐ Showers |
| ☐ Is Day or Night | ☐ Snowfall |
| ☐ Weather code | ☐ Wind Speed (10 m) |
| ☐ Cloud Cover Total | ☐ Wind Direction (10 m) |
| ☐ Sea Level Pressure | ☐ Wind Gusts (10 m) |
| ☐ Surface Pressure | |

Note: Current conditions are based on 15-minutely weather model data. Every weather variable available in hourly data, is available as current condition as well.

- [https://api.open-meteo.com/v1/forecast?latitude=\\${lat}&longitude=\\${lon}¤t=temperature_2m,relative_humidity_2m](https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}¤t=temperature_2m,relative_humidity_2m)



[과제] 실시간 날씨 - 모듈분리

- weather.js를 데이터를 책임지는 weatherAPI.js와 화면을 책임지는 realtimeInfo.js로 분리한다.
 - index.html: `<script type="module" src="realtimeInfo.js"></script>`
 - weatherAPI.js: export async function 분리
 - realtimeInfo.js: weatherAPI 로 부터 함수를 import 하여 처리



수고 하셨습니다!